To eliminate scenarios where the strategy fills the "Long" and "Short" orders before evaluating the strategy.close_all() call, we can instruct it to *cancel* one of the orders after it executes the other. Below, we included "Entry" as the `oca_name` argument and strategy.oca.cancel as the `oca_type` argument in both strategy.order() calls. Now, after the strategy executes either the "Long" or "Short" order, it cancels the other order and waits for strategy.close_all() to close the position:

Figure 260: image

```
//@version=6
strategy("OCA Cancel Demo", overlay=true)

float ma1 = ta.sma(close, 5)
float ma2 = ta.sma(close, 9)

if ta.cross(ma1, ma2)
    if strategy.position_size == 0
        strategy.order("Long",  strategy.long, stop = high, oca_name = "Entry", oca_type = strategy.oca.cancel
        strategy.order("Short", strategy.short, stop = low, oca_name = "Entry", oca_type = strategy.oca.cancel
    else
        strategy.close_all()

plot(ma1, "Fast MA", color.aqua)
plot(ma2, "Slow MA", color.orange)
```

strategy.oca.reduce

When an order placement command uses strategy.oca.reduce as its OCA type, the strategy *does not* cancel the resulting order entirely if another order with the same OCA name executes first. Instead, it *reduces* the order's size by the filled number of contracts/shares/lots/units, which is particularly useful for custom exit strategies.

The following example demonstrates a *long-only* strategy that generates a single stop-loss order and two take-profit orders for each new entry. When a faster moving average crosses over a slower one, the script calls strategy.entry() with `qty = 6` to create an entry order, and then it uses three strategy.order() calls to create a stop order at the `stop` price and two limit orders at the `limit1` and `limit2` prices. The strategy.order() call for the "Stop" order uses `qty = 6`, and the two calls for the "Limit 1" and "Limit 2" orders both use `qty = 3`:

```
//@version=6
strategy("Multiple TP Demo", overlay = true)

var float stop   = na
var float limit1 = na
var float limit2 = na

bool longCondition = ta.crossover(ta.sma(close, 5), ta.sma(close, 9))
if longCondition and strategy.position_size == 0
    stop   := close * 0.99
    limit1 := close * 1.01
    limit2 := close * 1.02
```

```
    strategy.entry("Long",  strategy.long, 6)
    strategy.order("Stop",  strategy.short, stop = stop, qty = 6)
    strategy.order("Limit 1", strategy.short, limit = limit1, qty = 3)
    strategy.order("Limit 2", strategy.short, limit = limit2, qty = 3)

bool showPlot = strategy.position_size != 0
plot(showPlot ? stop   : na, "Stop",    color.red,   style = plot.style_linebr)
plot(showPlot ? limit1 : na, "Limit 1", color.green, style = plot.style_linebr)
plot(showPlot ? limit2 : na, "Limit 2", color.green, style = plot.style_linebr)
```

After adding this strategy to the chart, we see it does not work as initially intended. The problem with this script is that the orders from strategy.order()**do not** belong to an OCA group by default (unlike strategy.exit(), whose orders automatically belong to a strategy.oca.reduce OCA group). Since the strategy does not assign the strategy.order() calls to any OCA group, it does not reduce any unfilled stop or limit orders after executing an order. Consequently, if the broker emulator fills the stop order and at least one of the limit orders, the traded quantity **exceeds** the open long position, resulting in an open *short* position:

Figure 261: image

For our long-only strategy to work as we intended, we must instruct it to *reduce* the sizes of the unfilled stop/limit orders after one of them executes to prevent selling a larger quantity than the open long position.

Below, we specified "Bracket" as the `oca_name` and strategy.oca.reduce as the `oca_type` in all the script's strategy.order() calls. These changes tell the strategy to reduce the sizes of the orders in the "Bracket" group each time the broker emulator fills one of them. This version of the strategy never simulates a short position because the total size of its filled stop and limit orders never *exceeds* the long position's size:

```
//@version=6
strategy("Multiple TP Demo", overlay = true)

var float stop   = na
var float limit1 = na
var float limit2 = na

bool longCondition = ta.crossover(ta.sma(close, 5), ta.sma(close, 9))
if longCondition and strategy.position_size == 0
    stop   := close * 0.99
    limit1 := close * 1.01
    limit2 := close * 1.02
    strategy.entry("Long",  strategy.long, 6)
```

Figure 262: image

```
    strategy.order("Stop",  strategy.short, stop = stop, qty = 6, oca_name = "Bracket", oca_type = strategy.oc
    strategy.order("Limit 1", strategy.short, limit = limit1, qty = 3, oca_name = "Bracket", oca_type = strate
    strategy.order("Limit 2", strategy.short, limit = limit2, qty = 6, oca_name = "Bracket", oca_type = strate

bool showPlot = strategy.position_size != 0
plot(showPlot ? stop   : na, "Stop",    color.red,   style = plot.style_linebr)
plot(showPlot ? limit1 : na, "Limit 1", color.green, style = plot.style_linebr)
plot(showPlot ? limit2 : na, "Limit 2", color.green, style = plot.style_linebr)
```

Note that:

- We also changed the `qty` value of the "Limit 2" order to 6 instead of 3 because the strategy reduces its amount by three units when it executes the "Limit 1" order. Keeping the `qty` value of 3 would cause the second limit order's size to drop to 0 after the strategy fills the first limit order, meaning it would never execute.

`strategy.oca.none`

When an order placement command uses strategy.oca.none as its `oca_type` value, all orders from that command execute *independently* of any OCA group. This value is the default `oca_type` for the strategy.order() and strategy.entry() commands.

Currency

Pine Script™ strategies can use different currencies in their calculations than the instruments they simulate trades on. Programmers can specify a strategy's account currency by including a `currency.*` variable as the `currency` argument in the strategy() declaration statement. The default value is currency.NONE, meaning the strategy uses the same currency as the current chart (syminfo.currency). Script users can change the account currency using the "Base currency" input in the script's "Settings/Properties" tab.

When a strategy script uses an account currency that differs from the chart's currency, it uses the *previous daily value* of a corresponding currency pair from the most popular exchange to determine the conversion rate. If no exchange provides the rate directly, it derives the rate using a spread symbol. The strategy multiplies all monetary values, including simulated profits/losses, by the determined cross rate to express them in the account currency. To retrieve the rate that a strategy uses to convert monetary values, call request.currency_rate() with syminfo.currency as the `from` argument and strategy.account_currency as the `to` argument.

Note that:

- Programmers can directly convert values expressed in a strategy's account currency to the chart's currency and vice versa via the strategy.convert_to_symbol() and strategy.convert_to_account() functions.

The following example demonstrates how currency conversion affects a strategy's monetary values and how a strategy's cross-rate calculations match those that `request.*()` functions use.

On each of the latest 500 bars, the strategy places an entry order with strategy.entry(), and it places a take-profit and stop-loss order one tick away from the entry price with strategy.exit(). The size of each entry order is `1.0 / syminfo.mintick`, rounded to the nearest tick, which means that the profit/loss of each closed trade is equal to *one point* in the chart's *quote currency*. We specified currency.EUR as the account currency in the strategy() declaration statement, meaning the strategy multiplies all monetary values by a cross rate to express them in Euros.

The script calculates the absolute change in the ratio of the strategy's net profit to the symbol's point value to determine the value of *one unit* of the chart's currency in Euros. It plots this value alongside the result from a request.currency_rate() call that uses syminfo.currency and strategy.account_currency as the `from` and `to` arguments. As we see below, both plots align, confirming that strategies and `request.*()` functions use the *same* daily cross-rate calculations:

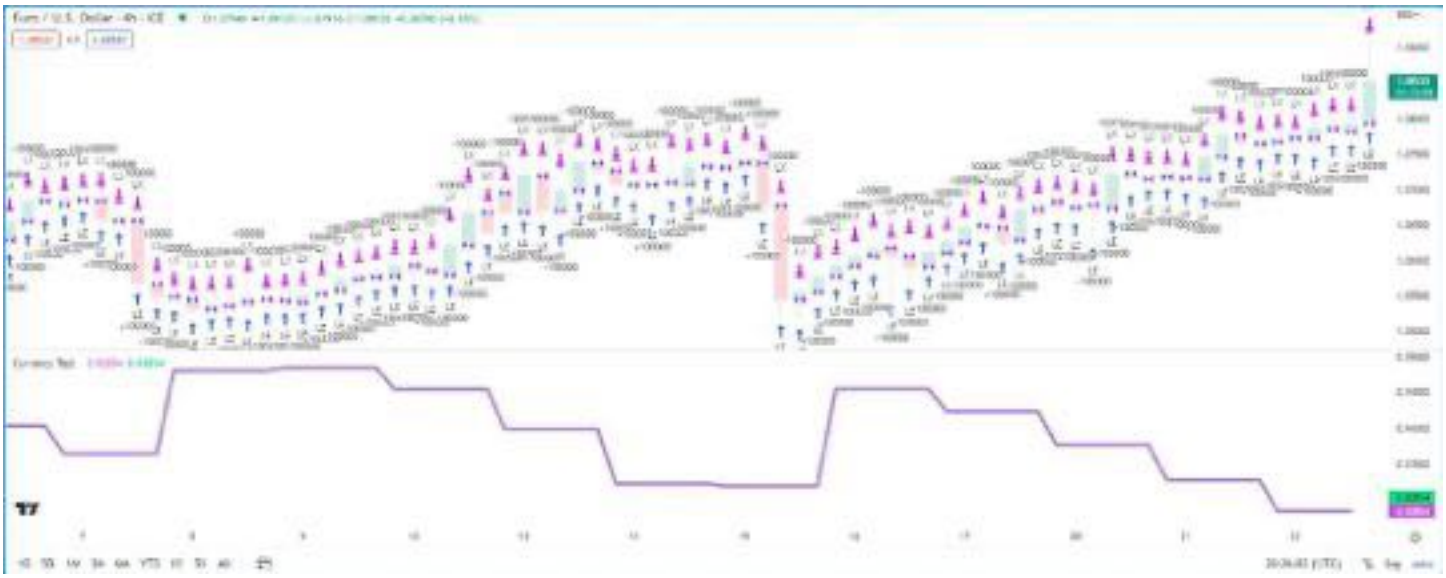


Figure 263: image

```
//@version=6
strategy("Currency Test", currency = currency.EUR)

if last_bar_index - bar_index < 500
    // Place an entry order with a size that results in a P/L of `syminfo.pointvalue` units of chart currency
    strategy.entry("LE", strategy.long, math.round_to_mintick(1.0 / syminfo.mintick))
    // Place exit orders one tick above and below the "LE" entry price,
    // meaning each trade closes with one point of profit or loss in the chart's currency.
    strategy.exit("LX", "LE", profit = 1, loss = 1)

// Plot the absolute change in `strategy.netprofit / syminfo.pointvalue`, which represents 1 chart unit of pro
plot(
     math.abs(ta.change(strategy.netprofit / syminfo.pointvalue)), "1 chart unit of profit/loss in EUR",
     color = color.fuchsia, linewidth = 4
 )
// Plot the requested currency rate.
plot(request.currency_rate(syminfo.currency, strategy.account_currency), "Requested conversion rate", color.li
```

Note that:

- When a strategy executes on a chart with a timeframe higher than "1D", it uses the data from *one day before* each *historical* bar's closing time for its cross-rate calculations. For example, on a "1W" chart, the strategy bases its cross rate on the previous Thursday's closing values. However, it still uses the latest confirmed daily rate on *realtime* bars.

Altering calculation behavior

Strategy scripts execute across all available historical chart bars and continue to execute on realtime bars as new data comes in. However, by default, strategies only recalculate their values after a bar *closes*, even on realtime bars, and the earliest point that the broker emulator fills the orders a strategy places on the close one bar is at the *open* of the following bar.

Users can change these behaviors with the `calc_on_every_tick`, `calc_on_order_fills`, and `process_orders_on_close` parameters of the strategy() declaration statement or the corresponding inputs in the "Recalculate" and "Fill orders" sections of the script's "Settings/Properties" tab. The sections below explain how these settings affect a strategy's calculations.

`calc_on_every_tick`

The `calc_on_every_tick` parameter of the strategy() function determines the frequency of a strategy's calculations on *realtime bars*. When this parameter's value is `true`, the script recalculates on each *new tick* in the realtime data feed. Its default value is `false`, meaning the script only executes on a realtime bar after it closes. Users can also toggle this recalculation behavior with the "On every tick" input in the script's "Settings/Properties" tab.

Enabling this setting can be useful in forward testing because it allows a strategy to use realtime price updates in its calculations. However, it *does not* affect the calculations on historical bars because historical data feeds *do not* contain complete tick data: the broker emulator considers each historical bar to have only four ticks (open, high, low, and close). Therefore, users should exercise caution and understand the limitations of this setting. If enabling calculation on every tick causes a strategy to behave *differently* on historical and realtime bars, the strategy will **repaint** after the user reloads it.

The following example demonstrates how recalculation on every tick can cause strategy repainting. The script uses strategy.entry() calls to place a long entry order each time the close reaches its `highest` value and a short entry order each time the close reaches its `lowest` value. The strategy() declaration statement includes `calc_on_every_tick = true`, meaning that on realtime bars, it can recalculate and place orders on new price updates *before* a bar closes:

```
//@version=6
strategy("Donchian Channel Break", overlay = true, calc_on_every_tick = true, pyramiding = 20)

int length = input.int(15, "Length")

float highest = ta.highest(close, length)
float lowest  = ta.lowest(close, length)

if close == highest
    strategy.entry("Buy", strategy.long)
if close == lowest
    strategy.entry("Sell", strategy.short)

// Highlight the background of realtime bars.
bgcolor(barstate.isrealtime ? color.new(color.orange, 80) : na)

plot(highest, "Highest", color = color.lime)
plot(lowest, "Lowest", color = color.red)
```

Note that:

- The script uses a pyramiding value of 20, allowing it to simulate up to 20 entries per position with the strategy.entry() command.
- The script highlights the chart's background orange when barstate.isrealtime is `true` to indicate realtime bars.

After applying the script to our chart and letting it run on several realtime bars, we see the following output:

The script placed a "Buy" order on *each tick* where the close was at the `highest` value, which happened *more than once* on each realtime bar. Additionally, the broker emulator filled each market order at the current realtime price rather than strictly at the open of the following chart bar.

After we reload the chart, we see that the strategy *changed* its behavior and *repainted* its results on those bars. This time, the strategy placed only *one* "Buy" order for each *closed bar* where the condition was valid, and the broker emulator filled each order at the open of the following bar. It did not generate multiple entries per bar because what were previously realtime bars became *historical* bars, which **do not** hold complete tick data:

`calc_on_order_fills`

The `calc_on_order_fills` parameter of the strategy() function enables a strategy to recalculate immediately after an *order fills*, allowing it to use more granular information and place additional orders without waiting for a bar to close. Its default value is `false`, meaning the strategy does not allow recalculation immediately after every order fill. Users can also toggle this behavior with the "After order is filled" input in the script's "Settings/Properties" tab.

Figure 264: image

Figure 265: image

Enabling this setting can provide a strategy script with additional data that would otherwise not be available until after a bar closes, such as the current average price of a simulated position on an open bar.

The example below shows a simple strategy that creates a "Buy" order with strategy.entry() whenever the strategy.position_size is 0. The script uses strategy.position_avg_price to calculate price levels for the strategy.exit() call's stop-loss and take-profit orders that close the position.

We've included `calc_on_order_fills = true` in the strategy() declaration statement, meaning that the strategy recalculates each time the broker emulator fills a "Buy" or "Exit" order. Each time an "Exit" order fills, the strategy.position_size reverts to 0, triggering a new "Buy" order. The broker emulator fills the "Buy" order on the next tick at one of the bar's OHLC values, and then the strategy uses the recalculated strategy.position_avg_price value to determine new "Exit" order prices:

Figure 266: image

```
//@version=6
strategy("Intrabar exit", overlay = true, calc_on_order_fills = true)

float stopSize   = input.float(5.0, "SL %", minval = 0.0) / 100.0
float profitSize = input.float(5.0, "TP %", minval = 0.0) / 100.0

if strategy.position_size == 0.0
    strategy.entry("Buy", strategy.long)

float stopLoss   = strategy.position_avg_price * (1.0 - stopSize)
float takeProfit = strategy.position_avg_price * (1.0 + profitSize)

strategy.exit("Exit", stop = stopLoss, limit = takeProfit)
```

Note that:

- Without enabling recalculation on order fills, this strategy would not place new orders *before* a bar closes. After an exit, the strategy would wait for the bar to close before placing a new "Buy" order, which the broker emulator would fill on the *next tick* after that, i.e., the open of the following bar.

It's important to note that enabling `calc_on_order_fills` can produce unrealistic strategy results in some cases because the broker emulator may assume order-fill prices that are *not* obtainable in real-world trading. Therefore, users should exercise caution and carefully examine their strategy logic when allowing recalculation on order fills.

For example, the following script places a "Buy" order after each new order fill and bar close over the most recent 25 historical bars. The strategy simulates *four* entries per bar because the broker emulator considers each historical bar to have *four ticks*

(open, high, low, and close). This behavior is unrealistic because it is not typically possible to fill an order at a bar's *exact* high or low price:

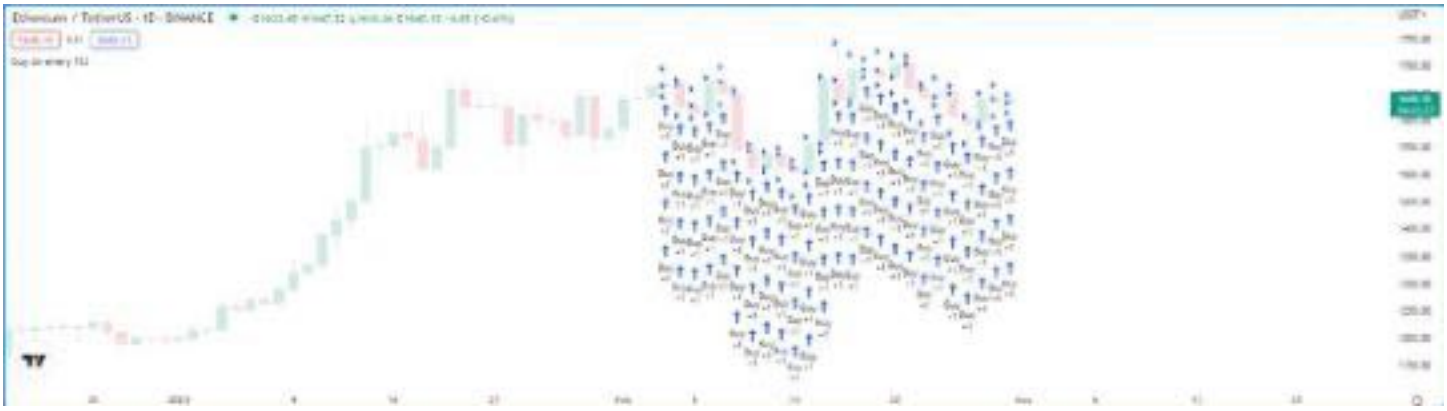


Figure 267: image

```
//@version=6
strategy("buy on every fill", overlay = true, calc_on_order_fills = true, pyramiding = 100)

if last_bar_index - bar_index <= 25
    strategy.entry("Buy", strategy.long)
```

`process_orders_on_close`

By default, strategies simulate orders at the close of each bar, meaning that the earliest opportunity to fill the orders and execute strategy calculations and alerts is on the opening of the following bar. Programmers can change this behavior to process orders on the *closing tick* of each bar by setting `process_orders_on_close` to `true` in the strategy() declaration statement. Users can set this behavior by changing the "Fill Orders/On Bar Close" setting in the "Settings/Properties" tab.

This behavior is most useful when backtesting manual strategies in which traders exit from a position before a bar closes, or in scenarios where algorithmic traders in non-24x7 markets set up after-hours trading capability so that alerts sent after close still have hope of filling before the following day.

Note that:

- Using strategies with `process_orders_on_close` enabled to send alerts to a third-party service might cause unintended results. Alerts on the close of a bar still occur after the market closes, and real-world orders based on such alerts might not fill until after the market opens again.
- The strategy.close() and strategy.close_all() commands feature an `immediately` parameter that, if `true`, allows the resulting market order to fill on the same tick where the strategy created it. This parameter provides an alternative way for programmers to selectively apply `process_orders_on_close` behavior to closing market orders without affecting the behavior of other order placement commands.

Simulating trading costs

Strategy performance reports are more relevant and meaningful when they include potential real-world trading costs. Without modeling the potential costs associated with their trades, traders may overestimate a strategy's historical profitability, potentially leading to suboptimal decisions in live trading. Pine Script™ strategies include inputs and parameters for simulating trading costs in performance results.

Commission

Commission is the fee a broker/exchange charges when executing trades. Commission can be a flat fee per trade or contract/share/lot/unit, or a percentage of the total transaction value. Users can set the commission properties of their strategies by including `commission_type` and `commission_value` arguments in the strategy() function, or by setting the "Commission" inputs in the "Properties" tab of the strategy settings.

The following script is a simple strategy that simulates a "Long" position of 2% of equity when `close` equals the `highest` value over the `length`, and closes the trade when it equals the `lowest` value:

Figure 268: image

```
//@version=6
strategy("Commission Demo", overlay=true, default_qty_value = 2, default_qty_type = strategy.percent_of_equity

length = input.int(10, "Length")

float highest = ta.highest(close, length)
float lowest  = ta.lowest(close, length)

switch close
    highest => strategy.entry("Long", strategy.long)
    lowest  => strategy.close("Long")

plot(highest, color = color.new(color.lime, 50))
plot(lowest, color = color.new(color.red, 50))
```

The results in the Strategy Tester show that the strategy had a positive equity growth of 17.61% over the testing range. However, the backtest results do not account for fees the broker/exchange may charge. Let's see what happens to these results when we include a small commission on every trade in the strategy simulation. In this example, we've included `commission_type = strategy.commission.percent` and `commission_value = 1` in the strategy() declaration, meaning it will simulate a commission of 1% on all executed orders:

```
//@version=6
strategy(
     "Commission Demo", overlay=true, default_qty_value = 2, default_qty_type = strategy.percent_of_equity,
     commission_type = strategy.commission.percent, commission_value = 1
 )

length = input.int(10, "Length")

float highest = ta.highest(close, length)
float lowest  = ta.lowest(close, length)

switch close
```

Figure 269: image

```
    highest => strategy.entry("Long", strategy.long)
    lowest  => strategy.close("Long")

plot(highest, color = color.new(color.lime, 50))
plot(lowest, color = color.new(color.red, 50))
```

As we can see in the example above, after applying a 1% commission to the backtest, the strategy simulated a significantly reduced net profit of only 1.42% and a more volatile equity curve with an elevated max drawdown. These results highlight the impact that commission can have on a strategy's hypothetical performance.

Slippage and unfilled limits

In real-life trading, a broker/exchange may fill orders at slightly different prices than a trader intended, due to volatility, liquidity, order size, and other market factors, which can profoundly impact a strategy's performance. The disparity between expected prices and the actual prices at which the broker/exchange executes trades is what we refer to as *slippage*. Slippage is dynamic and unpredictable, making it impossible to simulate precisely. However, factoring in a small amount of slippage on each trade during a backtest or forward test might help the results better align with reality. Users can model slippage in their strategy results, sized as a fixed number of *ticks*, by including a `slippage` argument in the strategy() declaration statement or by setting the "Slippage" input in the "Settings/Properties" tab.

The following example demonstrates how simulating slippage affects the fill prices of market orders in a strategy test. The script below places a "Buy" market order of 2% equity when the market price is above a rising EMA and closes the position when the price dips below the EMA while it's falling. We've included `slippage = 20` in the strategy() function, which declares that the price of each simulated order will slip 20 ticks in the direction of the trade.

The script uses strategy.opentrades.entry_bar_index() and strategy.closedtrades.exit_bar_index() to get the `entryIndex` and `exitIndex`, which it uses to obtain the `fillPrice` of the order. When the bar index is at the `entryIndex`, the `fillPrice` is the first strategy.opentrades.entry_price() value. At the `exitIndex`, `fillPrice` is the strategy.closedtrades.exit_price() value from the last closed trade. The script plots the expected fill price along with the simulated fill price after slippage to visually compare the difference:

```
//@version=6
strategy(
     "Slippage Demo", overlay = true, slippage = 20,
```

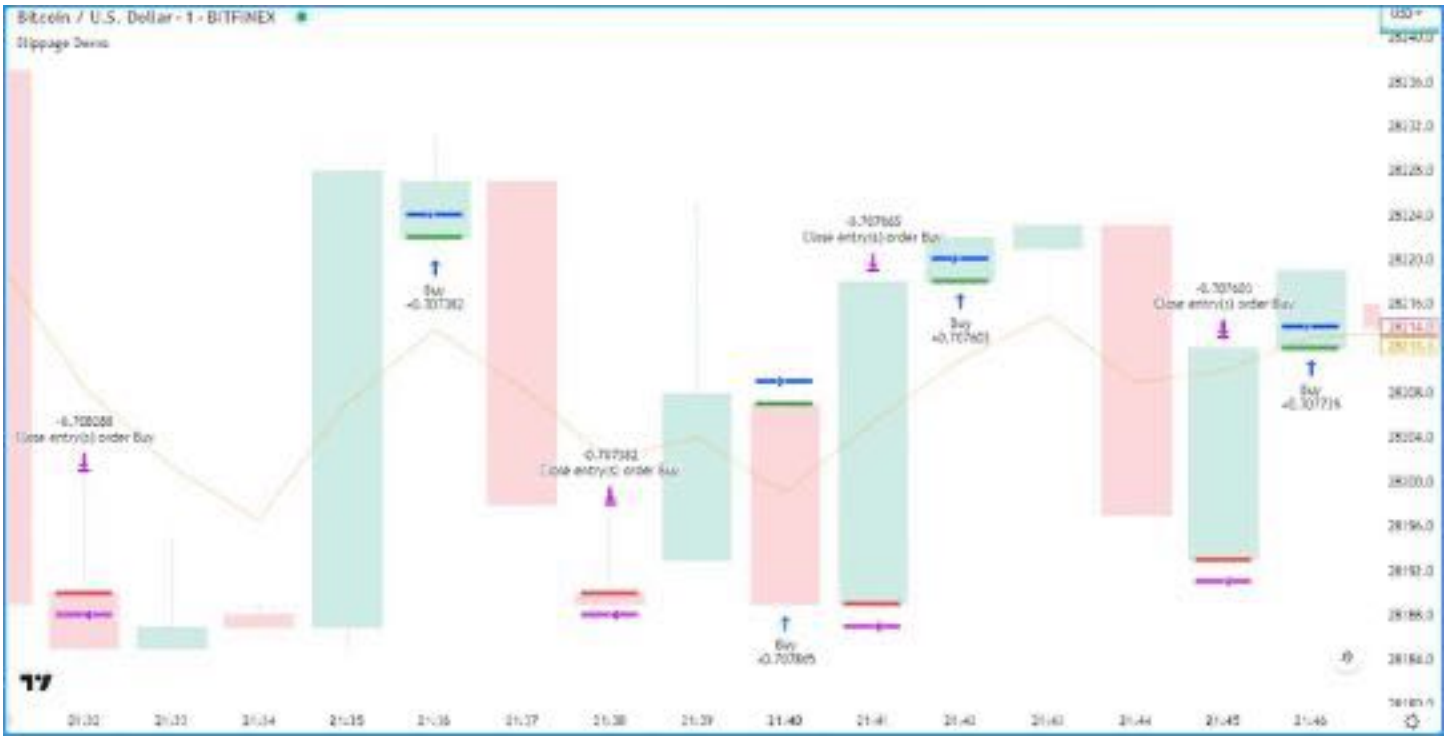


Figure 270: image

```
     default_qty_value = 2, default_qty_type = strategy.percent_of_equity
 )

int length = input.int(5, "Length")

//@variable Exponential moving average with an input `length`.
float ma = ta.ema(close, length)

//@variable Is `true` when `ma` has increased and `close` is above it, `false` otherwise.
bool longCondition = close > ma and ma > ma[1]
//@variable Is `true` when `ma` has decreased and `close` is below it, `false` otherwise.
bool shortCondition = close < ma and ma < ma[1]

// Enter a long market position on `longCondition` and close the position on `shortCondition`.
if longCondition
    strategy.entry("Buy", strategy.long)
if shortCondition
    strategy.close("Buy")

//@variable The `bar_index` of the position's entry order fill.
int entryIndex = strategy.opentrades.entry_bar_index(0)
//@variable The `bar_index` of the position's close order fill.
int exitIndex  = strategy.closedtrades.exit_bar_index(strategy.closedtrades - 1)

//@variable The fill price simulated by the strategy.
float fillPrice = switch bar_index
    entryIndex => strategy.opentrades.entry_price(0)
    exitIndex  => strategy.closedtrades.exit_price(strategy.closedtrades - 1)

//@variable The expected fill price of the open market position.
float expectedPrice = not na(fillPrice) ? open : na

color expectedColor = na
```

```
color filledColor   = na

if bar_index == entryIndex
    expectedColor := color.green
    filledColor   := color.blue
else if bar_index == exitIndex
    expectedColor := color.red
    filledColor   := color.fuchsia

plot(ma, color = color.new(color.orange, 50))

plotchar(not na(fillPrice) ? open : na, "Expected fill price", "-", location.absolute, expectedColor)
plotchar(fillPrice, "Fill price after slippage", "-", location.absolute, filledColor)
```

Note that:

- Since the strategy applies constant slippage to all order fills, some orders can fill *outside* the candle range in the simulation. Exercise caution with this setting, as adding excessive simulated slippage can produce unrealistically worse testing results.

Some traders might assume that they can avoid the adverse effects of slippage by using limit orders, as unlike market orders, they cannot execute at a worse price than the specified value. However, even if the market price reaches an order's price, there's a chance that a limit order might not fill, depending on the state of the real-life market, because limit orders can only fill if a security has sufficient liquidity and price action around their values. To account for the possibility of *unfilled* orders in a backtest, users can specify the `backtest_fill_limits_assumption` value in the declaration statement or use the "Verify price for limit orders" input in the "Settings/Properties" tab. This setting instructs the strategy to fill limit orders only after the market price moves a defined number of ticks past the order prices.

The following example places a limit order of 2% equity at a bar's hlcc4 price when the high is the `highest` value over the past `length` bars and there are no pending entries. The strategy closes the market position and cancels all orders after the low is the `lowest` value. Each time the strategy triggers an order, it draws a horizontal line at the `limitPrice`, which it updates on each bar until closing the position or canceling the order:

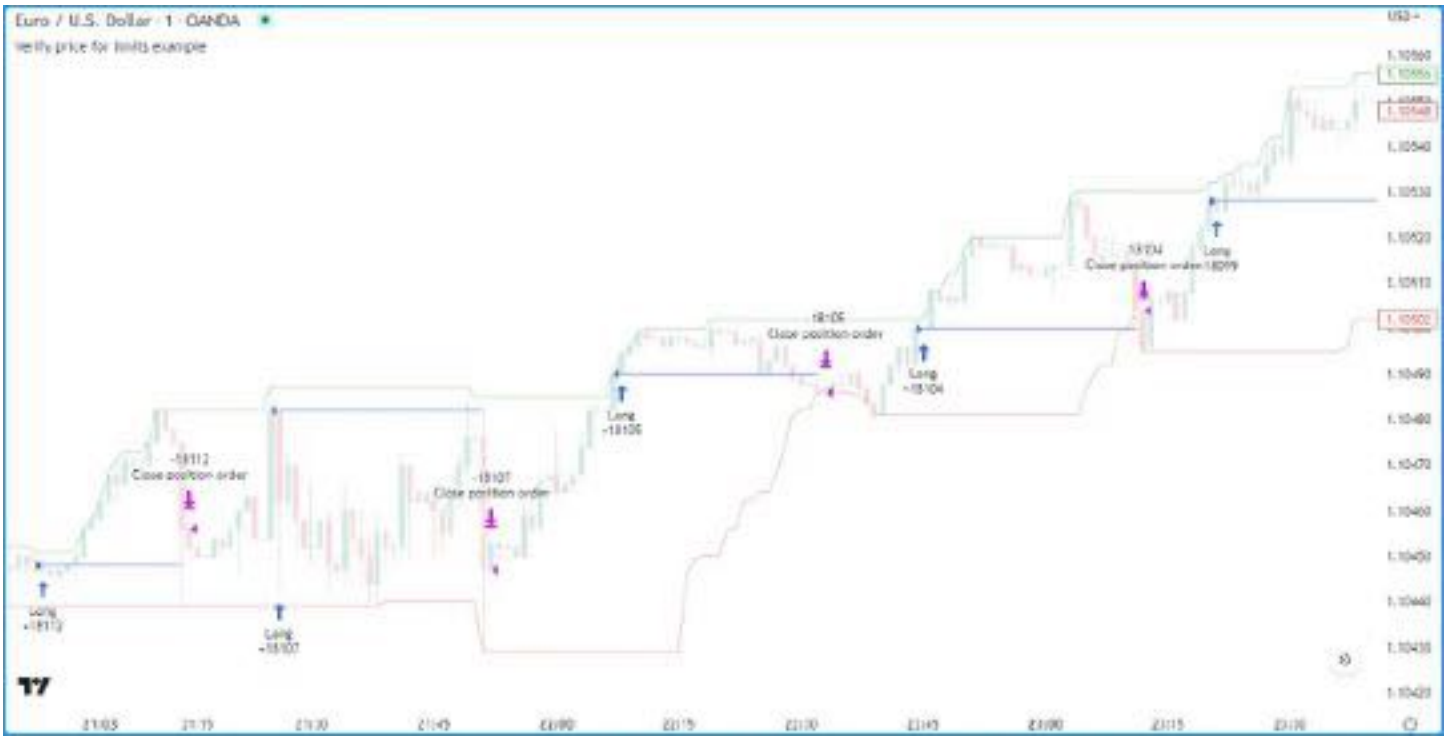


Figure 271: image

```
//@version=6
strategy(
     "Verify price for limits example", overlay = true,
```

```
     default_qty_type = strategy.percent_of_equity, default_qty_value = 2
 )

int length = input.int(25, title = "Length")

//@variable Draws a line at the limit price of the most recent entry order.
var line limitLine = na

// Highest high and lowest low
highest = ta.highest(length)
lowest  = ta.lowest(length)

// Place an entry order and draw a new line when the the `high` equals the `highest` value and `limitLine` is
if high == highest and na(limitLine)
    float limitPrice = hlcc4
    strategy.entry("Long", strategy.long, limit = limitPrice)
    limitLine := line.new(bar_index, limitPrice, bar_index + 1, limitPrice)

// Close the open market position, cancel orders, and set `limitLine` to `na` when the `low` equals the `lowes
if low == lowest
    strategy.cancel_all()
    limitLine := na
    strategy.close_all()

// Update the `x2` value of `limitLine` if it isn't `na`.
if not na(limitLine)
    limitLine.set_x2(bar_index + 1)

plot(highest, "Highest High", color = color.new(color.green, 50))
plot(lowest, "Lowest Low", color = color.new(color.red, 50))
```

By default, the script assumes that all limit orders are guaranteed to fill when the market price reaches their values, which is often not the case in real-life trading. Let's add price verification to our limit orders to account for potentially unfilled ones. In this example, we've included `backtest_fill_limits_assumption = 3` in the strategy() function call. As we can see, using limit verification omits some simulated order fills and changes the times of others, because the entry orders can now only fill after the price exceeds the limit price by *three ticks*:

Risk management

Designing a strategy that performs well, especially in a broad class of markets, is a challenging task. Most strategies are designed for specific market patterns/conditions and can produce uncontrolled losses when applied to other data. Therefore, a strategy's risk management behavior can be critical to its performance. Programmers can set risk management criteria in their strategy scripts using the `strategy.risk.*()` commands.

Strategies can incorporate any number of risk management criteria in any combination. All risk management commands execute *on every tick and order execution event*, regardless of any changes to the strategy's calculation behavior. There is no way to deactivate any of these commands on specific script executions. Irrespective of a risk management command's location, it *always* applies to the strategy unless the programmer removes the call from the code.

strategy.risk.allow_entry_in()

This command overrides the market direction allowed for all strategy.entry() commands in the script. When a user specifies the trade direction with the strategy.risk.allow_entry_in() function (e.g., long) the strategy enters trades only in that direction. If a script calls an entry command in the opposite direction while there's an open market position, the strategy simulates a market order to *close* the position.

strategy.risk.max_cons_loss_days()

This command cancels all pending orders, closes any open market position, and stops all additional trade actions after the strategy simulates a defined number of trading days with consecutive losses.

strategy.risk.max_drawdown()

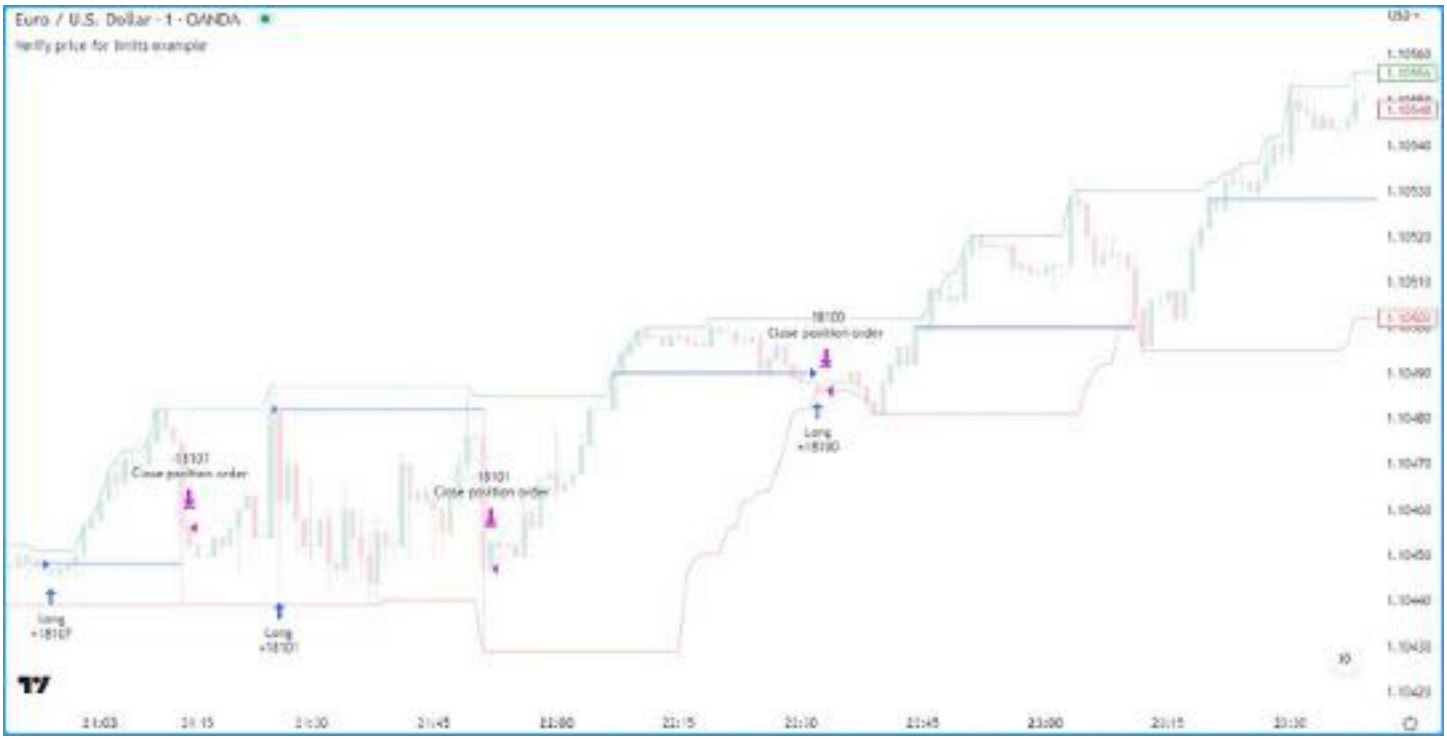


Figure 272: image

This command cancels all pending orders, closes any open market position, and stops all additional trade actions after the strategy's drawdown reaches the amount specified in the function call.

strategy.risk.max_intraday_filled_orders()

This command specifies the maximum number of filled orders per trading day (or per chart bar if the timeframe is higher than daily). If the strategy creates more orders than the maximum, the command cancels all pending orders, closes any open market position, and halts trading activity until the end of the current session.

strategy.risk.max_intraday_loss()

This command controls the maximum loss the strategy tolerates per trading day (or per chart bar if the timeframe is higher than daily). When the strategy's losses reach this threshold, it cancels all pending orders, closes the open market position, and stops all trading activity until the end of the current session.

strategy.risk.max_position_size()

This command specifies the maximum possible position size when using strategy.entry() commands. If the quantity of an entry command results in a market position that exceeds this threshold, the strategy reduces the order quantity so that the resulting position does not exceed the limit.

Margin

Margin is the minimum percentage of a market position that a trader must hold in their account as collateral to receive and sustain a loan from their broker to achieve their desired *leverage*. The `margin_long` and `margin_short` parameters of the strategy() declaration statement and the "Margin for long/short positions" inputs in the "Properties" tab of the script settings specify margin percentages for long and short positions. For example, if a trader sets the margin for long positions to 25%, they must have enough funds to cover 25% of an open long position. This margin percentage also means the trader can potentially spend up to 400% of their equity on their trades.

If a strategy's simulated funds cannot cover the losses from a margin trade, the broker emulator triggers a *margin call*, which forcibly liquidates all or part of the open position. The exact number of contracts/shares/lots/units that the emulator liquidates is *four times* the amount required to cover the loss, which helps prevent constant margin calls on subsequent bars. The emulator determines liquidated quantity using the following algorithm:

1. Calculate the amount of capital spent on the position: `Money Spent = Quantity * Entry Price`
2. Calculate the Market Value of Security (MVS): `MVS = Position Size * Current Price`

3. Calculate the Open Profit as the difference between `MVS` and `Money Spent`. If the position is short, multiply this value by -1.
4. Calculate the strategy's equity value: `Equity = Initial Capital + Net Profit + Open Profit`
5. Calculate the margin ratio: `Margin Ratio = Margin Percent / 100`
6. Calculate the margin value, which is the cash required to cover the hypothetical account's portion of the position: `Margin = MVS * Margin Ratio`
7. Calculate the strategy's available funds: `Available Funds = Equity - Margin`
8. Calculate the total amount of money lost: `Loss = Available Funds / Margin Ratio`
9. Calculate the number of contracts/shares/lots/units the account must liquidate to cover the loss, truncated to the same decimal precision as the minimum position size for the current symbol: `Cover Amount = TRUNCATE(Loss / Current Price).`
10. Multiply the quantity required to cover the loss by four to determine the margin call size: `Margin Call Size = Cover Amount * 4`

To examine this calculation in detail, let's add the built-in Supertrend Strategy to the NASDAQ:TSLA chart on the "1D" timeframe and set the "Order size" to 300% of equity and the "Margin for long positions" to 25% in the "Properties" tab of the strategy settings:

Figure 273: image

The first entry happened at the bar's opening price on 16 Sep 2010. The strategy bought 682,438 shares (Position Size) at 4.43 USD (Entry Price). Then, on 23 Sep 2010, when the price dipped to 3.9 (Current Price), the emulator forcibly liquidated 111,052 shares with a margin call. The calculations below show how the broker emulator determined this amount for the margin call event:

```
Money spent: 682438 * 4.43 = 3023200.34MVS: 682438 * 3.9 = 2661508.20pen Profit: -361692.14Equity: 1000000 + 0
```

Note that:

- The strategy.margin_liquidation_price variable's value represents the price level that will cause a margin call if the market price reaches it. For more information about how margin works and the formula for calculating a position's margin call price, see this page in our Help Center.

Using strategy information in scripts

Numerous built-ins within the `strategy.*` namespace and its *sub-namespaces* provide convenient solutions for programmers to use a strategy's trade and performance information, including data shown in the Strategy Tester, directly within their code's logic and calculations.

Several `strategy.*` variables hold fundamental information about a strategy, including its starting capital, equity, profits and losses, run-up and drawdown, and open position:

- strategy.account_currency
- strategy.initial_capital
- strategy.equity
- strategy.netprofit and strategy.netprofit_percent
- strategy.grossprofit and strategy.grossprofit_percent
- strategy.grossloss and strategy.grossloss_percent
- strategy.openprofit and strategy.openprofit_percent
- strategy.max_runup and strategy.max_runup_percent
- strategy.max_drawdown and strategy.max_drawdown_percent
- strategy.position_size
- strategy.position_avg_price
- strategy.position_entry_name

Additionally, the namespace features multiple variables that hold general trade information, such as the number of open and closed trades, the number of winning and losing trades, average trade profits, and maximum trade sizes:

- strategy.opentrades
- strategy.closedtrades
- strategy.wintrades
- strategy.losstrades
- strategy.eventrades
- strategy.avg_trade and strategy.avg_trade_percent
- strategy.avg_winning_trade and strategy.avg_winning_trade_percent
- strategy.avg_losing_trade and strategy.avg_losing_trade_percent
- strategy.max_contracts_held_all
- strategy.max_contracts_held_long
- strategy.max_contracts_held_short

Programmers can use these variables to display relevant strategy information on their charts, create customized trading logic based on strategy data, calculate custom performance metrics, and more.

The following example demonstrates a few simple use cases for these `strategy.*` variables. The script uses them in its order placement and display calculations. When the calculated `rank` crosses above 10 and the strategy.opentrades value is 0, the script calls strategy.entry() to place a "Buy" market order. On the following bar, where that order fills, it calls strategy.exit() to create a stop-loss order at a user-specified percentage below the strategy.position_avg_price. If the `rank` crosses above 80 during the open trade, the script uses strategy.close() to exit the position on the next bar.

The script draws a table on the main chart pane displaying formatted strings containing the strategy's net profit and net profit percentage, the account currency, the number of winning trades and the win percentage, the ratio of the average profit to the average loss, and the profit factor (the ratio of the gross profit to the gross loss). It also plots the total equity in a separate pane and highlights the pane's background based on the strategy's open profit:

```
//@version=6
strategy(
     "Using strategy information demo", default_qty_type = strategy.percent_of_equity, default_qty_value = 5,
     margin_long = 100, margin_short = 100
 )

//@variable The number of bars in the `rank` calculation.
int lengthInput = input.int(50, "Length", 1)
//@variable The stop-loss percentage.
float slPercentInput = input.float(4.0, "SL %", 0.0, 100.0) / 100.0

//@variable The percent rank of `close` prices over `lengthInput` bars.
float rank = ta.percentrank(close, lengthInput)
// Entry and exit signals.
bool entrySignal = ta.crossover(rank, 10) and strategy.opentrades == 0
bool exitSignal  = ta.crossover(rank, 80) and strategy.opentrades == 1

// Place orders based on the `entrySignal` and `exitSignal` occurrences.
```

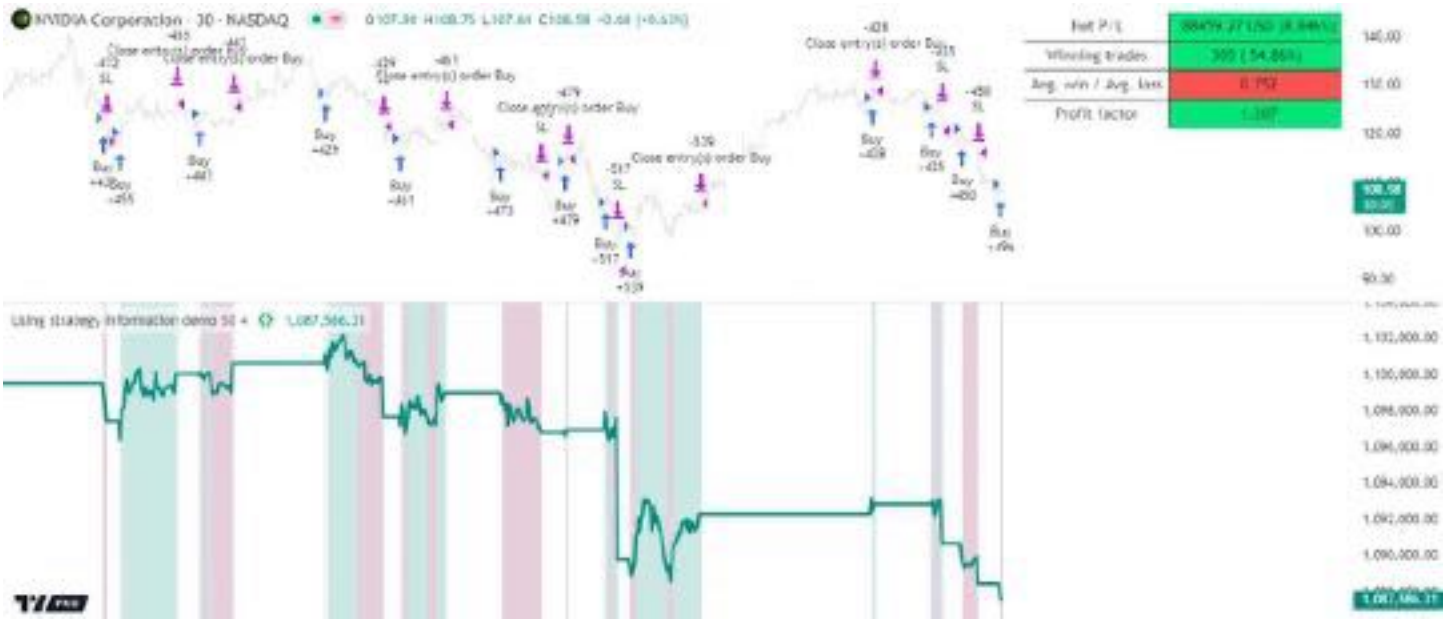


Figure 274: image

```
switch
    entrySignal    => strategy.entry("Buy", strategy.long)
    entrySignal[1] => strategy.exit("SL", "Buy", stop = strategy.position_avg_price * (1.0 - slPercentInput))
    exitSignal     => strategy.close("Buy")

if barstate.islastconfirmedhistory or barstate.isrealtime
    //@variable A table displaying strategy information on the main chart pane.
    var table dashboard = table.new(
         position.top_right, 2, 10, border_color = chart.fg_color, border_width = 1, force_overlay = true
     )
    //@variable The strategy's currency.
    string currency = strategy.account_currency
    // Display the net profit as a currency amount and percentage.
    dashboard.cell(0, 1, "Net P/L")
    dashboard.cell(
         1, 1, str.format("{0, number, 0.00} {1} ({2}%)", strategy.netprofit, currency, strategy.netprofit_per
         text_color = chart.fg_color, bgcolor = strategy.netprofit > 0 ? color.lime : color.red
     )
    // Display the number of winning trades as an absolute value and percentage of all completed trades.
    dashboard.cell(0, 2, "Winning trades")
    dashboard.cell(
         1, 2, str.format("{0} ({1, number, #.##%})", strategy.wintrades, strategy.wintrades / strategy.closed
         text_color = chart.fg_color, bgcolor = strategy.wintrades > strategy.losstrades ? color.lime : color.
     )
    // Display the ratio of average trade profit to average trade loss.
    dashboard.cell(0, 3, "Avg. win / Avg. loss")
    dashboard.cell(
         1, 3, str.format("{0, number, #.###}", strategy.avg_winning_trade / strategy.avg_losing_trade),
         text_color = chart.fg_color,
         bgcolor = strategy.avg_winning_trade > strategy.avg_losing_trade ? color.lime : color.red
     )
    // Display the profit factor, i.e., the ratio of gross profit to gross loss.
    dashboard.cell(0, 4, "Profit factor")
    dashboard.cell(
         1, 4, str.format("{0, number, #.###}", strategy.grossprofit / strategy.grossloss), text_color = chart
         bgcolor = strategy.grossprofit > strategy.grossloss ? color.lime : color.red
     )
```

```
// Plot the current equity in a separate pane and highlight the pane's background while there is an open posit
plot(strategy.equity, "Total equity", strategy.equity > strategy.initial_capital ? color.teal : color.maroon,
bgcolor(
     strategy.openprofit > 0 ? color.new(color.teal, 80) : strategy.openprofit < 0 ? color.new(color.maroon, 8
     title = "Open position highlight"
 )
```

Note that:

- This script creates a stop-loss order one bar after the entry order because it uses strategy.position_avg_price to determine the price level. This variable has a non-na value only when the strategy has an *open position*.
- The script only draws the table on the last historical bar and all realtime bars because the historical states of tables are **never visible**. See the Reducing drawing updates section of the Profiling and optimization page for more information.
- We included `force_overlay = true` in the table.new() call to display the table on the main chart pane.

Individual trade information

The `strategy.*` namespace features two sub-namespaces that provide access to *individual trade* information: `strategy.opentrades.*` and `strategy.closedtrades.*`. The `strategy.opentrades.*` built-ins return data for *incomplete* (open) trades, and the `strategy.closedtrades.*` built-ins return data for *completed* (closed) trades. With these built-ins, programmers can use granular trade data in their scripts, allowing for more detailed strategy analysis and advanced calculations.

Both sub-namespaces contain several similar functions that return information about a trade's orders, simulated costs, and profit/loss, including:

- strategy.opentrades.entry_id() / strategy.closedtrades.entry_id()
- strategy.opentrades.entry_price() / strategy.closedtrades.entry_price()
- strategy.opentrades.entry_bar_index() / strategy.closedtrades.entry_bar_index()
- strategy.opentrades.entry_time() / strategy.closedtrades.entry_time()
- strategy.opentrades.entry_comment() / strategy.closedtrades.entry_comment()
- strategy.opentrades.size() / strategy.closedtrades.size()
- strategy.opentrades.profit() / strategy.closedtrades.profit()
- strategy.opentrades.profit_percent() / strategy.closedtrades.profit_percent()
- strategy.opentrades.commission() / strategy.closedtrades.commission()
- strategy.opentrades.max_runup() / strategy.closedtrades.max_runup()
- strategy.opentrades.max_runup_percent() / strategy.closedtrades.max_runup_percent()
- strategy.opentrades.max_drawdown() / strategy.closedtrades.max_drawdown()
- strategy.opentrades.max_drawdown_percent() / strategy.closedtrades.max_drawdown_percent()
- strategy.closedtrades.exit_id()
- strategy.closedtrades.exit_price()
- strategy.closedtrades.exit_time()
- strategy.closedtrades.exit_bar_index()
- strategy.closedtrades.exit_comment()

Note that:

- Most built-ins within these namespaces are *functions*. However, the `strategy.opentrades.*` namespace also features a unique *variable*: strategy.opentrades.capital_held. Its value represents the amount of capital reserved by *all* open trades.
- Only the `strategy.closedtrades.*` namespace has `.exit_*()` functions that return information about *exit orders*.

All `strategy.opentrades.*()` and `strategy.closedtrades.*()` functions have a `trade_num` parameter, which accepts an "int" value representing the index of the open or closed trade. The index of the first open/closed trade is 0, and the last trade's index is *one less* than the value of the strategy.opentrades/strategy.closedtrades variable.

The following example places up to five long entry orders per position, each with a unique ID, and it calculates metrics for specific closed trades.

The strategy places a new entry order when the close crosses above its `median` without reaching the `highest` value, but only if the number of open trades is less than five. It exits each position using stop-loss orders from strategy.exit() or a market order from strategy.close_all(). Each successive entry order's ID depends on the number of open trades. The first entry ID in each position is "Buy0", and the last possible entry ID is "Buy4".

The script calls `strategy.closedtrades.*()` functions within a for loop to access closed trade entry IDs, profits, entry bar indices, and exit bar indices. It uses this information to calculate the total number of closed trades with the specified entry ID, the number of winning trades, the average number of bars per trade, and the total profit from all the trades. The script then organizes this information in a formatted string and displays it in a single-cell table:

Figure 275: image

```
//@version=6
strategy(
     "Individual trade information demo", pyramiding = 5, default_qty_type = strategy.percent_of_equity,
     default_qty_value = 1, margin_long = 100, margin_short = 100
 )

//@variable The number of bars in the `highest` and `lowest` calculation.
int lengthInput = input.int(50, "Length", 1)
string idInput = input.string("Buy0", "Entry ID to analyze", ["Buy0", "Buy1", "Buy2", "Buy3", "Buy4"])

// Calculate the highest, lowest, and median `close` values over `lengthInput` bars.
float highest = ta.highest(close, lengthInput)
float lowest  = ta.lowest(close, lengthInput)
float median  = 0.5 * (highest + lowest)

// Define entry and stop-loss orders when the `close` crosses above the `median` without touching the `highest
if ta.crossover(close, median) and close != highest and strategy.opentrades < 5
    strategy.entry("Buy" + str.tostring(strategy.opentrades), strategy.long)
    if strategy.opentrades == 0
        strategy.exit("SL", stop = lowest)
// Close the entire position when the `close` reaches the `lowest` value.
if close == lowest
    strategy.close_all()

// The total number of closed trades with the `idInput` entry, the number of wins, the average number of bars,
// and the total profit.
int   trades  = 0
int   wins    = 0
float avgBars = 0
float totalPL = 0.0

if barstate.islastconfirmedhistory or barstate.isrealtime
    //@variable A single-cell table displaying information about closed trades with the `idInput` entry ID.
    var table infoTable = table.new(position.middle_center, 1, 1, color.purple)
    // Iterate over closed trade indices.
```

```
    for tradeNum = 0 to strategy.closedtrades - 1
        // Skip the rest of the current iteration if the `tradeNum` closed trade didn't open with an `idInput`
        if strategy.closedtrades.entry_id(tradeNum) != idInput
            continue
        // Accumulate `trades`, `wins`, `avgBars`, and `totalPL` values.
        float profit = strategy.closedtrades.profit(tradeNum)
        trades  += 1
        wins    += profit > 0 ? 1 : 0
        avgBars += strategy.closedtrades.exit_bar_index(tradeNum) - strategy.closedtrades.entry_bar_index(trad
        totalPL += profit
    avgBars /= trades

    //@variable A formatted string containing the calculated closed trade information.
    string displayText = str.format(
         "ID: {0}\n\nTotal trades: {1}\nWin trades: {2}\nAvg. bars: {3}\nTotal P/L: {4} {5}",
         idInput, trades, wins, avgBars, totalPL, strategy.account_currency
     )
    // Populate the table's cell with `displayText`.
    infoTable.cell(0, 0, displayText, text_color = color.white, text_halign = text.align_left, text_size = siz

// Plot the highest, median, and lowest values on the main chart pane.
plot(highest, "Highest close", force_overlay = true)
plot(median, "Median close", force_overlay = true)
plot(lowest, "Lowest close", force_overlay = true)
```

Note that:

- This strategy can open up to five long trades per position because we included `pyramiding = 5` in the strategy() declaration statement. See the pyramiding section for more information.
- The strategy.exit() instance in this script persists and generates exit orders for every entry in the open position because we did not specify a `from_entry` ID. See the Exits for multiple entries section to learn more about this behavior.

Strategy alerts

Pine Script™ indicators (not strategies) have two different mechanisms to set up custom alert conditions: the alertcondition() function, which tracks one specific condition per function call, and the alert() function, which tracks all its calls simultaneously, but provides greater flexibility in the number of calls, alert messages, etc.

Pine Script™ strategies cannot create alert triggers using the alertcondition() function, but they can create triggers with the alert() function. Additionally, each order placement command comes with its own built-in alert functionality that does not require any additional code to implement. As such, any strategy that uses an order placement command can issue alerts upon order execution. The precise mechanics of such built-in strategy alerts are described in the Order Fill events section of the Alerts page.

When a strategy uses both the alert() function and functions that create orders in the same script, the "Create Alert" dialog box provides a choice between the conditions to use as a trigger: alert() events, order fill events, or both.

For many trading strategies, the delay between a triggered alert and a live trade can be a critical performance factor. By default, strategy scripts can only execute alert() function calls on the close of realtime bars, as if they used alert.freq_once_per_bar_close, regardless of the `freq` argument in the call. Users can change the alert frequency by including `calc_on_every_tick = true` in the strategy() call or selecting the "Recalculate/On every tick" option in the "Settings/Properties" tab before creating the alert. However, depending on the script, this setting can adversely impact the strategy's behavior. See the `calc_on_every_tick` section for more information.

Order fill alert triggers do not suffer the same limitations as the triggers from alert() calls, which makes them more suitable for sending alerts to third parties for automation. Alerts from order fill events execute *immediately*, unaffected by a script's `calc_on_every_tick` setting. Users can set the default message for order fill alerts via the `//@strategy_alert_message` compiler annotation. The text provided with this annotation populates the "Message" field in the "Create Alert" dialog box.

The following script shows a simple example of a default order fill alert message. Above the strategy() declaration statement, the script includes `@strategy_alert_message` with *placeholders* for the trade action, current position size, ticker name, and fill price values in the message text:

```
//@version=6
```

```
//@strategy_alert_message {{strategy.order.action}} {{strategy.position_size}} {{ticker}} @ {{strategy.order.p
strategy("Alert Message Demo", overlay = true)
float fastMa = ta.sma(close, 5)
float slowMa = ta.sma(close, 10)

if ta.crossover(fastMa, slowMa)
    strategy.entry("buy", strategy.long)

if ta.crossunder(fastMa, slowMa)
    strategy.entry("sell", strategy.short)

plot(fastMa, "Fast MA", color.aqua)
plot(slowMa, "Slow MA", color.orange)
```

This script populates the "Create Alert" dialog box with its default message when the user selects its name from the "Condition" dropdown tab:

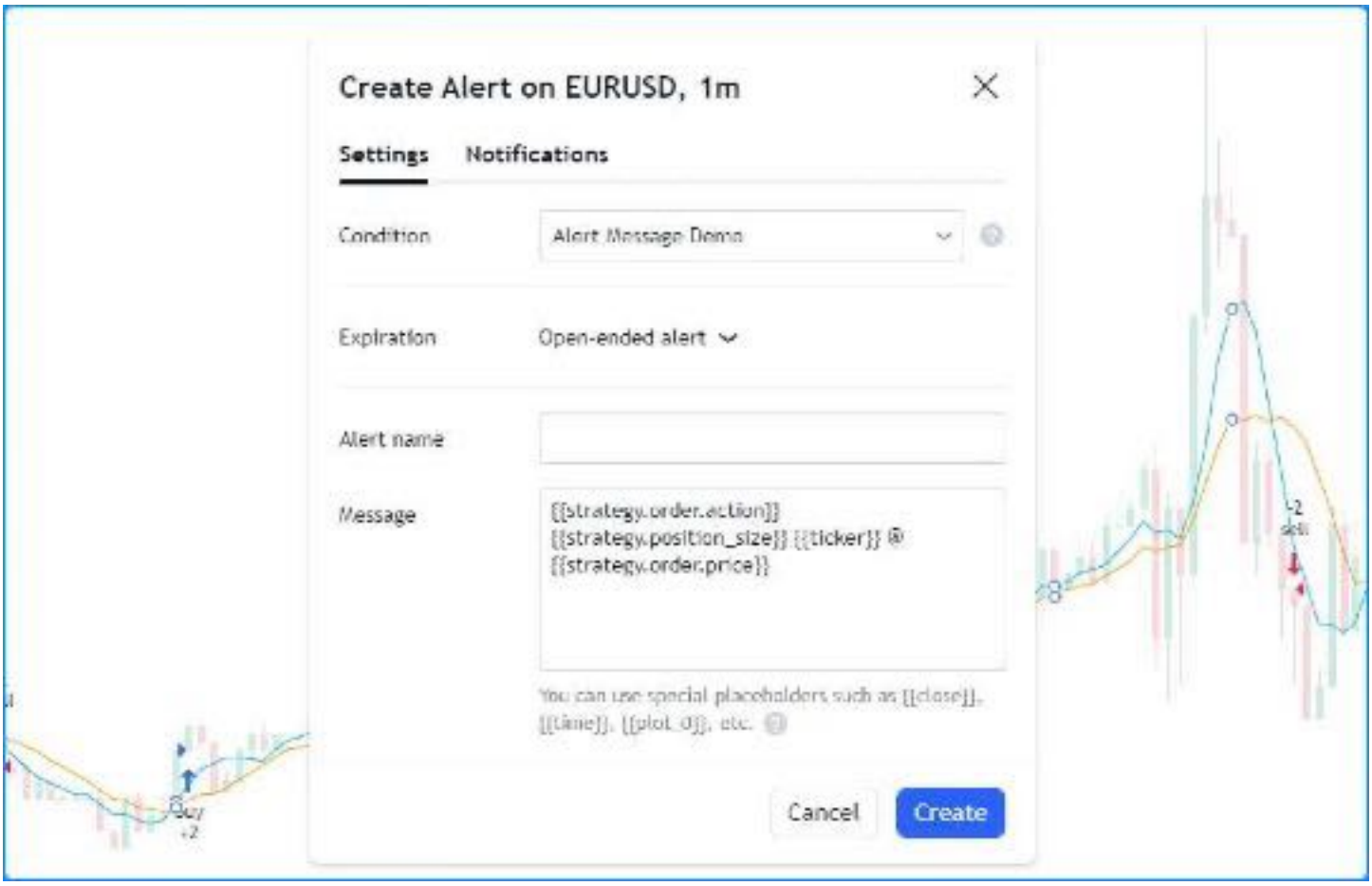


Figure 276: image

When the alert fires, the strategy populates the placeholders in the alert message with their corresponding values. For example:

Notes on testing strategies

Testing and tuning strategies in historical and live market conditions can provide insight into a strategy's characteristics, potential weaknesses, and *possibly* its future potential. However, traders should always be aware of the biases and limitations of simulated strategy results, especially when using the results to support live trading decisions. This section outlines some caveats associated with strategy validation and tuning and possible solutions to mitigate their effects.

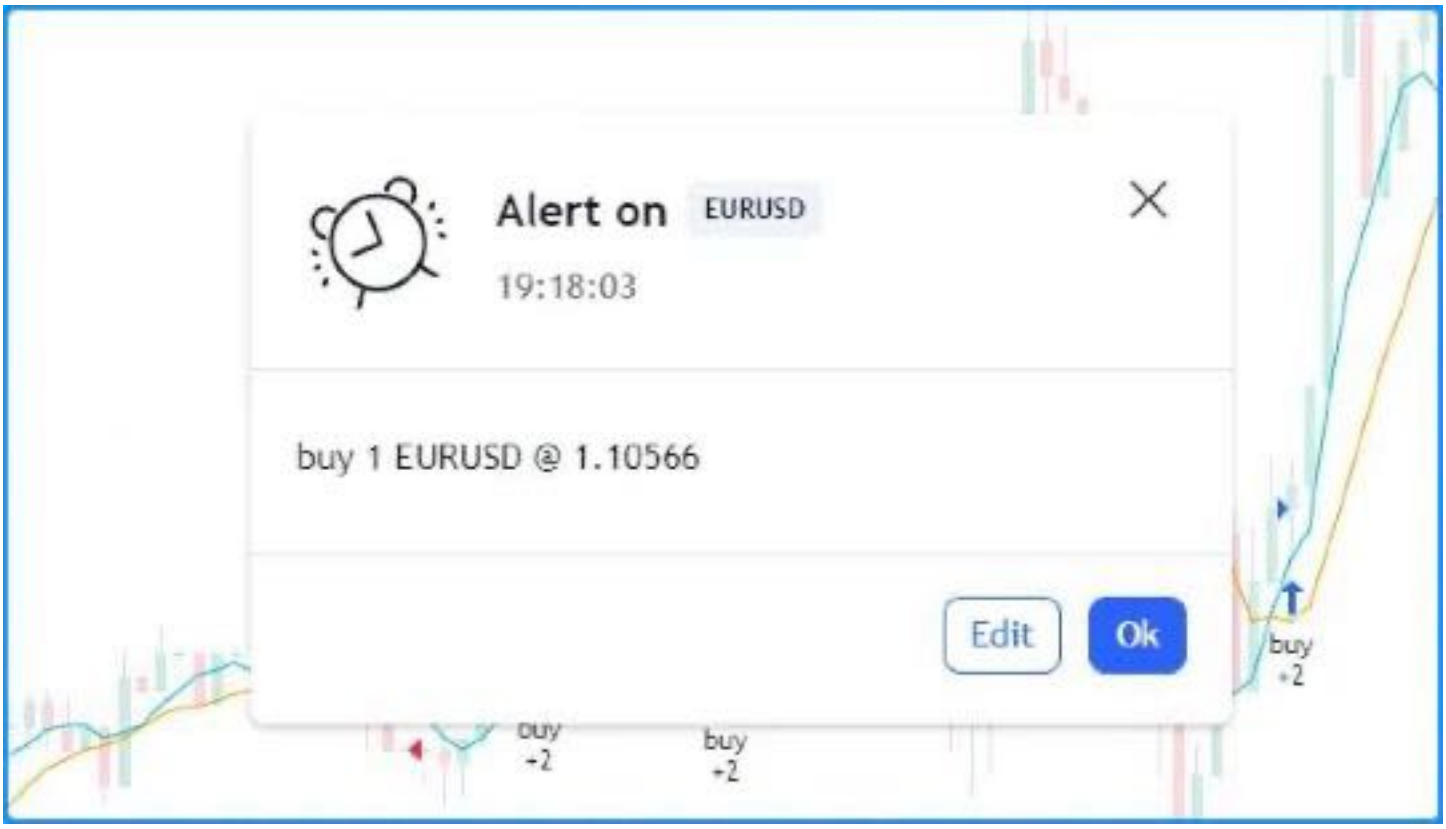


Figure 277: image

Backtesting and forward testing

Backtesting is a technique to evaluate the historical performance of a trading strategy or model by simulating and analyzing its past results on historical market data. This technique assumes that a strategy's results on past data can provide insight into its strengths and weaknesses. When backtesting, many traders adjust the parameters of a strategy in an attempt to optimize its results. Analysis and optimization of historical results can help traders to gain a deeper understanding of a strategy. However, traders should always understand the risks and limitations when basing their decisions on optimized backtest results.

It is prudent to also use realtime analysis as a tool for evaluating a trading system on a forward-looking basis. *Forward testing* aims to gauge the performance of a strategy in live market conditions, where factors such as trading costs, slippage, and liquidity can meaningfully affect its performance. While forward testing has the distinct advantage of not being affected by certain types of biases (e.g., lookahead bias or "future data leakage"), it does carry the disadvantage of being limited in the quantity of data to test. Therefore, although it can provide helpful insights into a strategy's performance in current market conditions, forward testing is not typically used on its own.

Lookahead bias

One typical issue in backtesting strategies that request alternate timeframe data, use repainting variables such as timenow, or alter calculation behavior for intrabar order fills, is the leakage of future data into the past during evaluation, which is known as *lookahead bias*. Not only is this bias a common cause of unrealistic strategy results, since the future is never actually knowable beforehand, but it is also one of the typical causes of strategy repainting.

Traders can often confirm whether a strategy has lookahead bias by forward testing it on realtime data, where no known data exists beyond the latest bar. Since there is no future data to leak into the past on realtime bars, the strategy will behave differently on historical and realtime bars if its results have lookahead bias.

To eliminate lookahead bias in a strategy:

- Do not use repainting variables that leak future values into the past in the order placement or cancellation logic.
- Do not include barmerge.lookahead_on in `request.*()` calls without offsetting the data series, as described in this section of the Repainting page.
- Use realistic strategy calculation behavior.

Selection bias

Selection bias occurs when a trader analyzes only results on specific instruments or timeframes while ignoring others. This bias can distort the perspective of the strategy's robustness, which can impact trading decisions and performance optimizations. Traders can reduce the effects of selection bias by evaluating their strategies on multiple, ideally diverse, symbols and timeframes, and ensuring not to ignore poor performance results or "cherry-pick" testing ranges.

Overfitting

A common problem when optimizing a strategy based on backtest results is overfitting ("curve fitting"), which means tailoring the strategy for specific data. An overfitted strategy often fails to generalize well on new, unseen data. One widely-used approach to help reduce the potential for overfitting and promote better generalization is to split an instrument's data into two or more parts to test the strategy outside the sample used for optimization, otherwise known as "in-sample" (IS) and "out-of-sample" (OOS) backtesting.

In this approach, traders optimize strategy parameters on the IS data, and they test the optimized configuration on the OOS data without additional fine-tuning. Although this and other, more robust approaches might provide a glimpse into how a strategy might fare after optimization, traders should still exercise caution. No trading strategy can guarantee future performance, regardless of the data used for optimization and testing, because the future is inherently unknowable.

Order limit

Outside of Deep Backtesting, a strategy can keep track of up to 9000 orders. If a strategy creates more than 9000 orders, the earliest orders are *trimmed* so that the strategy stores the information for only the most recent orders.

Trimmed orders do **not** appear in the Strategy Tester. Referencing the trimmed order IDs using `strategy.closedtrades.*` functions returns na.

The strategy.closedtrades.first_index variable holds the index of the oldest *untrimmed* trade, which corresponds to the first trade listed in the List of Trades. If the strategy creates less than 9000 orders, there are no trimmed orders, and this variable's value is 0.

[Previous

Sessions](#sessions)[Next

Tables](#tables) User Manual/Concepts/Tables

Tables

Introduction

Tables are objects that can be used to position information in specific and fixed locations in a script's visual space. Contrary to all other plots or objects drawn in Pine Script™, tables are not anchored to specific bars; they *float* in a script's space, whether in overlay or pane mode, in studies or strategies, independently of the chart bars being viewed or the zoom factor used.

Tables contain cells arranged in columns and rows, much like a spreadsheet. They are created and populated in two distincts steps:

1. A table's structure and key attributes are defined using table.new(), which returns a table ID that acts like a pointer to the table, just like label, line, or array IDs do. The table.new() call will create the table object but does not display it.
2. Once created, and for it to display, the table must be populated using one table.cell() call for each cell. Table cells can contain text, or not. This second step is when the width and height of cells are defined.

Most attributes of a previously created table can be changed using `table.set_*()` setter functions. Attributes of previously populated cells can be modified using `table.cell_set_*()` functions.

A table is positioned in an indicator's space by anchoring it to one of nine references: the four corners or midpoints, including the center. Tables are positioned by expanding the table from its anchor, so a table anchored to the position.middle_right reference will be drawn by expanding up, down and left from that anchor.

Two modes are available to determine the width/height of table cells:

- A default automatic mode calculates the width/height of cells in a column/row using the widest/highest text in them.
- An explicit mode allows programmers to define the width/height of cells using a percentage of the indicator's available x/y space.

Displayed table contents always represent the last state of the table, as it was drawn on the script's last execution, on the dataset's last bar. Contrary to values displayed in the Data Window or in indicator values, variable contents displayed in tables will thus not change as a script user moves his cursor over specific chart bars. For this reason, it is strongly recommended to always restrict execution of all `table.*()` calls to either the first or last bars of the dataset. Accordingly:

- Use the var keyword to declare tables.
- Enclose all other calls inside an ifbarstate.islast block.

Multiple tables can be used in one script, as long as they are each anchored to a different position. Each table object is identified by its own ID. Limits on the quantity of cells in all tables are determined by the total number of cells used in one script.

Creating tables

When creating a table using table.new(), three parameters are mandatory: the table's position and its number of columns and rows. Five other parameters are optional: the table's background color, the color and width of the table's outer frame, and the color and width of the borders around all cells, excluding the outer frame. All table attributes except its number of columns and rows can be modified using setter functions: table.set_position(), table.set_bgcolor(), table.set_frame_color(), table.set_frame_width(), table.set_border_color() and table.set_border_width().

Tables can be deleted using table.delete(), and their content can be selectively removed using table.clear().

When populating cells using table.cell(), you must supply an argument for four mandatory parameters: the table id the cell belongs to, its column and row index using indices that start at zero, and the text string the cell contains, which can be null. Other parameters are optional: the width and height of the cell, the text's attributes (color, horizontal and vertical alignment, size, formatting), and the cell's background color. All cell attributes can be modified using setter functions: table.cell_set_text(), table.cell_set_width(), table.cell_set_height(), table.cell_set_text_color(), table.cell_set_text_halign(), table.cell_set_text_valign(), table.cell_set_text_size(), table.cell_set_text_formatting(), and table.cell_set_bgcolor().

Keep in mind that each successive call to table.cell() redefines **all** the cell's properties, deleting any properties set by previous table.cell() calls on the same cell.

Placing a single value in a fixed position

Let's create our first table, which will place the value of ATR in the upper-right corner of the chart. We first create a one-cell table, then populate that cell:

```
//@version=6
indicator("ATR", "", true)
// We use `var` to only initialize the table on the first bar.
var table atrDisplay = table.new(position.top_right, 1, 1)
// We call `ta.atr()` outside the `if` block so it executes on each bar.
myAtr = ta.atr(14)
if barstate.islast
    // We only populate the table on the last bar.
    table.cell(atrDisplay, 0, 0, str.tostring(myAtr))
```

Note that:

- We use the var keyword when creating the table with table.new().
- We populate the cell inside an ifbarstate.islast block using table.cell().
- When populating the cell, we do not specify the `width` or `height`. The width and height of our cell will thus adjust automatically to the text it contains.
- We call `ta.atr(14)` prior to entry in our if block so that it evaluates on each bar. Had we used `str.tostring(ta.atr(14))` inside the if block, the function would not have evaluated correctly because it would be called on the dataset's last bar without having calculated the necessary values from the previous bars.

Let's improve the usability and aesthethics of our script:

```
//@version=6
indicator("ATR", "", true)
```

Figure 278: image

```
atrPeriodInput = input.int(14,  "ATR period", minval = 1, tooltip = "Using a period of 1 yields True Range.")

var table atrDisplay = table.new(position.top_right, 1, 1, bgcolor = color.gray, frame_width = 2, frame_color =
myAtr = ta.atr(atrPeriodInput)
if barstate.islast
    table.cell(atrDisplay, 0, 0, str.tostring(myAtr, format.mintick), text_color = color.white)
```

Figure 279: image

Note that:

- We used table.new() to define a background color, a frame color and its width.
- When populating the cell with table.cell(), we set the text to display in white.
- We pass format.mintick as a second argument to the str.tostring() function to restrict the precision of ATR to the chart's tick precision.
- We now use an input to allow the script user to specify the period of ATR. The input also includes a tooltip, which the user can see when he hovers over the "i" icon in the script's "Settings/Inputs" tab.

Coloring the chart's background

This example uses a one-cell table to color the chart's background on the bull/bear state of RSI:

```
//@version=6
indicator("Chart background", "", true)
bullColorInput = input.color(color.new(color.green, 95), "Bull", inline = "1")
bearColorInput = input.color(color.new(color.red, 95), "Bear", inline = "1")
// ----- Function colors chart bg on RSI bull/bear state.
colorChartBg(bullColor, bearColor) =>
    var table bgTable = table.new(position.middle_center, 1, 1)
    float r = ta.rsi(close, 20)
    color bgColor = r > 50 ? bullColor : r < 50 ? bearColor : na
    if barstate.islast
        table.cell(bgTable, 0, 0, width = 100, height = 100, bgcolor = bgColor)
```

```
colorChartBg(bullColorInput, bearColorInput)
```

Note that:

- We provide users with inputs allowing them to specify the bull/bear colors to use for the background, and send those input colors as arguments to our `colorChartBg()` function.
- We create a new table only once, using the var keyword to declare the table.
- We use table.cell() on the last bar only, to specify the cell's properties. We make the cell the width and height of the indicator's space, so it covers the whole chart.

Creating a display panel

Tables are ideal to create sophisticated display panels. Not only do they make it possible for display panels to always be visible in a constant position, they provide more flexible formatting because each cell's properties are controlled separately: background, text color, size and alignment, etc.

Here, we create a basic display panel showing a user-selected quantity of MAs values. We display their period in the first column, then their value with a green/red/gray background that varies with price's position with regards to each MA. When price is above/below the MA, the cell's background is colored with the bull/bear color. When the MA falls between the current bar's open and close, the cell's background is of the neutral color:

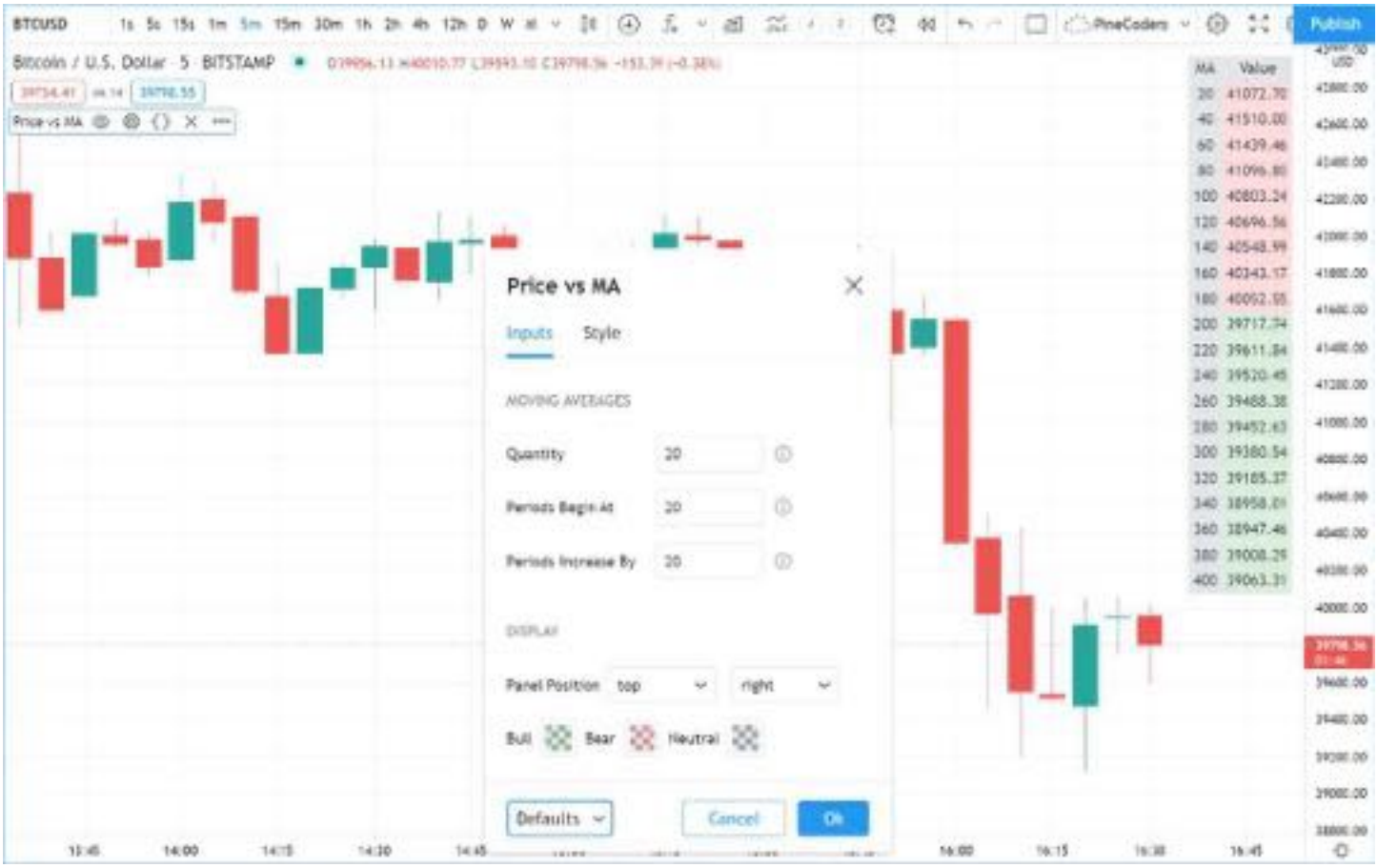


Figure 280: image

```
//@version=6
indicator("Price vs MA", "", true)

var string GP1 = "Moving averages"
int     masQtyInput    = input.int(20, "Quantity", minval = 1, maxval = 40, group = GP1, tooltip = "1-40")
int     masStartInput  = input.int(20, "Periods begin at", minval = 2, maxval = 200, group = GP1, tooltip = "2
int     masStepInput   = input.int(20, "Periods increase by", minval = 1, maxval = 100, group = GP1, tooltip =

var string GP2 = "Display"
string  tableYposInput = input.string("top", "Panel position", inline = "11", options = ["top", "middle", "bot
```

```
string  tableXposInput = input.string("right", "", inline = "11", options = ["left", "center", "right"], group
color   bullColorInput = input.color(color.new(color.green, 30), "Bull", inline = "12", group = GP2)
color   bearColorInput = input.color(color.new(color.red, 30), "Bear", inline = "12", group = GP2)
color   neutColorInput = input.color(color.new(color.gray, 30), "Neutral", inline = "12", group = GP2)

var table panel = table.new(tableYposInput + "_" + tableXposInput, 2, masQtyInput + 1)
if barstate.islast
    // Table header.
    table.cell(panel, 0, 0, "MA", bgcolor = neutColorInput)
    table.cell(panel, 1, 0, "Value", bgcolor = neutColorInput)

int period = masStartInput
for i = 1 to masQtyInput
    // ----- Call MAs on each bar.
    float ma = ta.sma(close, period)
    // ----- Only execute table code on last bar.
    if barstate.islast
        // Period in left column.
        table.cell(panel, 0, i, str.tostring(period), bgcolor = neutColorInput)
        // If MA is between the open and close, use neutral color. If close is lower/higher than MA, use bull/
        bgColor = close > ma ? open < ma ? neutColorInput : bullColorInput : open > ma ? neutColorInput : bear
        // MA value in right column.
        table.cell(panel, 1, i, str.tostring(ma, format.mintick), text_color = color.black, bgcolor = bgColor)
    period += masStepInput
```

Note that:

- Users can select the table's position from the inputs, as well as the bull/bear/neutral colors to be used for the background of the right column's cells.
- The table's quantity of rows is determined using the number of MAs the user chooses to display. We add one row for the column headers.
- Even though we populate the table cells on the last bar only, we need to execute the calls to ta.sma() on every bar so they produce the correct results. The compiler warning that appears when you compile the code can be safely ignored.
- We separate our inputs in two sections using `group`, and join the relevant ones on the same line using `inline`. We supply tooltips to document the limits of certain fields using `tooltip`.

Displaying a heatmap

Our next project is a heatmap, which will indicate the bull/bear relationship of the current price relative to its past values. To do so, we will use a table positioned at the bottom of the chart. We will display colors only, so our table will contain no text; we will simply color the background of its cells to produce our heatmap. The heatmap uses a user-selectable lookback period. It loops across that period to determine if price is above/below each bar in that past, and displays a progressively lighter intensity of the bull/bear color as we go further in the past:

Figure 281: image

```
//@version=6
indicator("Price vs Past", "", true)

var int MAX_LOOKBACK = 300

int     lookBackInput  = input.int(150, minval = 1, maxval = MAX_LOOKBACK, step = 10)
color   bullColorInput = input.color(#00FF00ff, "Bull", inline = "11")
color   bearColorInput = input.color(#FF0080ff, "Bear", inline = "11")

// ----- Function draws a heatmap showing the position of the current `_src` relative to its past `_lookBack`
drawHeatmap(src, lookBack) =>
    // float src     : evaluated price series.
    // int   lookBack: number of past bars evaluated.
    // Dependency: MAX_LOOKBACK

    // Force historical buffer to a sufficient size.
    max_bars_back(src, MAX_LOOKBACK)
    // Only run table code on last bar.
    if barstate.islast
        var heatmap = table.new(position.bottom_center, lookBack, 1)
        for i = 1 to lookBackInput
            float transp = 100. * i / lookBack
            if src > src[i]
                table.cell(heatmap, lookBack - i, 0, bgcolor = color.new(bullColorInput, transp))
            else
                table.cell(heatmap, lookBack - i, 0, bgcolor = color.new(bearColorInput, transp))

drawHeatmap(high, lookBackInput)
```

Note that:

- We define a maximum lookback period as a `MAX_LOOKBACK` constant. This is an important value and we use it for two purposes: to specify the number of columns we will create in our one-row table, and to specify the lookback period required for the `_src` argument in our function, so that we force Pine Script™ to create a historical buffer size that will allow us to refer to the required quantity of past values of `_src` in our for loop.
- We offer users the possibility of configuring the bull/bear colors in the inputs and we use `inline` to place the color selections on the same line.
- Inside our function, we enclose our table-creation code in an ifbarstate.islast construct so that it only runs on the last bar of the chart.
- The initialization of the table is done inside the if statement. Because of that, and the fact that it uses the var keyword, initialization only occurs the first time the script executes on a last bar. Note that this behavior is different from the usual var declarations in the script's global scope, where initialization occurs on the first bar of the dataset, at bar_index zero.
- We do not specify an argument to the `text` parameter in our table.cell() calls, so an empty string is used.
- We calculate our transparency in such a way that the intensity of the colors decreases as we go further in history.
- We use dynamic color generation to create different transparencies of our base colors as needed.
- Contrary to other objects displayed in Pine scripts, this heatmap's cells are not linked to chart bars. The configured lookback period determines how many table cells the heatmap contains, and the heatmap will not change as the chart is panned horizontally, or scaled.
- The maximum number of cells that can be displayed in the scritp's visual space will depend on your viewing device's resolution and the portion of the display used by your chart. Higher resolution screens and wider windows will allow more table cells to be displayed.

Tips

- When creating tables in strategy scripts, keep in mind that unless the strategy uses `calc_on_every_tick = true`, table code enclosed in ifbarstate.islast blocks will not execute on each realtime update, so the table will not display as you expect.
- Keep in mind that successive calls to table.cell() overwrite the cell's properties specified by previous table.cell() calls. Use the setter functions to modify a cell's properties.
- Remember to control the execution of your table code wisely by restricting it to the necessary bars only. This saves

server resources and your charts will display faster, so everybody wins.

[Previous

Strategies](#strategies)[Next

Text and shapes](#text-and-shapes) User Manual/Concepts/Text and shapes

Text and shapes

Introduction

You may display text or shapes using five different ways with Pine Script™:

- plotchar()
- plotshape()
- plotarrow()
- Labels created with label.new()
- Tables created with table.new() (see Tables)

Which one to use depends on your needs:

- Tables can display text in various relative positions on charts that will not move as users scroll of zoom the chart horizontally. Their content is not tethered to bars. In contrast, text displayed with plotchar(), plotshape() or label.new() is always tethered to a specific bar, so it will move with the bar's position on the chart. See the page on Tables for more information on them.
- Three function include are able to display pre-defined shapes: plotshape(), plotarrow() and Labels created with label.new().
- plotarrow() cannot display text, only up or down arrows.
- plotchar() and plotshape() can display non-dynamic text on any bar or all bars of the chart.
- plotchar() can only display one character while plotshape() can display strings, including line breaks.
- label.new() can display a maximum of 500 labels on the chart. Its text **can** contain dynamic text, or "series strings". Line breaks are also supported in label text.
- While plotchar() and plotshape() can display text at a fixed offset in the past or the future, which cannot change during the script's execution, each label.new() call can use a "series" offset that can be calculated on the fly.

These are a few things to keep in mind concerning Pine Script™ strings:

- Since the `text` parameter in both plotchar() and plotshape() require a "const string" argument, it cannot contain values such as prices that can only be known on the bar ("series string").
- To include "series" values in text displayed using label.new(), they will first need to be converted to strings using str.tostring().
- The concatenation operator for strings in Pine is `+`. It is used to join string components into one string, e.g., `msg = "Chart symbol: " + syminfo.tickerid` (where syminfo.tickerid is a built-in variable that returns the chart's exchange and symbol information in string format).
- Characters displayed by all these functions can be Unicode characters, which may include Unicode symbols. See this Exploring Unicode script to get an idea of what can be done with Unicode characters.
- Some functions have parameters that can specify the color, size, font family, and formatting of displayed text. For example, drawing objects like labels, tables, and boxes support text formatting such as bold, italics, and monospace.
- Pine scripts display strings using the system default font. The exact font may vary based on the user's operating system.

This script displays text using the four methods available in Pine Script™:

```
//@version=6
indicator("Four displays of text", overlay = true)
plotchar(ta.rising(close, 5), "`plotchar()`", " ", location.belowbar, color.lime, size = size.small)
plotshape(ta.falling(close, 5), "`plotchar()`", location = location.abovebar, color = na, text = "•`plotshape(

if bar_index % 25 == 0
    label.new(bar_index, na, "•LABEL•\nHigh = " + str.tostring(high, format.mintick) + "\n ", yloc = yloc.above

printTable(txt) => var table t = table.new(position.middle_right, 1, 1), table.cell(t, 0, 0, txt, bgcolor = co
```

```
printTable("•TABLE•\n" + str.tostring(bar_index + 1) + " bars\nin the dataset")
```

Figure 282: image

Note that:

- The method used to display each text string is shown with the text, except for the lime up arrows displayed using plotchar(), as it can only display one character.
- Label and table calls can be inserted in conditional structures to control when their are executed, whereas plotchar() and plotshape() cannot. Their conditional plotting must be controlled using their first argument, which is a "series bool" whose `true` or `false` value determines when the text is displayed.
- Numeric values displayed in the table and labels is first converted to a string using str.tostring().
- We use the `+` operator to concatenate string components.
- plotshape() is designed to display a shape with accompanying text. Its `size` parameter controls the size of the shape, not of the text. We use na for its `color` argument so that the shape is not visible.
- Contrary to other texts, the table text will not move as you scroll or scale the chart.
- Some text strings contain the Unicode arrow (U+1F807).
- Some text strings contain the `\n` sequence that represents a new line.

plotchar()

This function is useful to display a single character on bars. It has the following syntax:

```
plotchar(series, title, char, location, color, offset, text, textcolor, editable, size, show_last, display, fo
```

See the Reference Manual entry for plotchar() for details on its parameters.

As explained in the Without affecting the scale section of our page on Debugging, the function can be used to display and inspect values in the Data Window or in the indicator values displayed to the right of the script's name on the chart:

```
//@version=6
indicator("", "", true)
plotchar(bar_index, "Bar index", "", location.top)
```

Note that:

- The cursor is on the chart's last bar.
- The value of bar_index on **that** bar is displayed in indicator values (1) and in the Data Window (2).
- We use location.top because the default location.abovebar will put the price into play in the script's scale, which will often interfere with other plots.

plotchar() also works well to identify specific points on the chart or to validate that conditions are `true` when we expect them to be. This example displays an up arrow under bars where close, high and volume have all been rising for two bars:

Figure 283: image

```
//@version=6
indicator("", "", true)
bool longSignal = ta.rising(close, 2) and ta.rising(high, 2) and (na(volume) or ta.rising(volume, 2))
plotchar(longSignal, "Long", " ", location.belowbar, color = na(volume) ? color.gray : color.blue, size = size.
```

Figure 284: image

Note that:

- We use `(na(volume) or ta.rising(volume, 2))` so our script will work on symbols without volume data. If we did not make provisions for when there is no volume data, which is what `na(volume)` does by being `true` when there is no volume, the `longSignal` variable's value would never be `true` because `ta.rising(volume, 2)` yields `false` in those cases.
- We display the arrow in gray when there is no volume, to remind us that all three base conditions are not being met.
- Because plotchar() is now displaying a character on the chart, we use `size = size.tiny` to control its size.
- We have adapted the `location` argument to display the character under bars.

If you don't mind plotting only circles, you could also use plot() to achieve a similar effect:

```
//@version=6
indicator("", "", true)
longSignal = ta.rising(close, 2) and ta.rising(high, 2) and (na(volume) or ta.rising(volume, 2))
plot(longSignal ? low - ta.tr : na, "Long", color.blue, 2, plot.style_circles)
```

This method has the inconvenience that, since there is no relative positioning mechanism with plot() one must shift the circles down using something like ta.tr (the bar's "True Range"):

`plotshape()`

This function is useful to display pre-defined shapes and/or text on bars. It has the following syntax:

```
plotshape(series, title, style, location, color, offset, text, textcolor, editable, size, show_last, display,
```

Figure 285: image

See the Reference Manual entry for plotshape() for details on its parameters.

Let's use the function to achieve more or less the same result as with our second example of the previous section:

```
//@version=6
indicator("", "", true)
longSignal = ta.rising(close, 2) and ta.rising(high, 2) and (na(volume) or ta.rising(volume, 2))
plotshape(longSignal, "Long", shape.arrowup, location.belowbar)
```

Note that here, rather than using an arrow character, we are using the `shape.arrowup` argument for the `style` parameter.

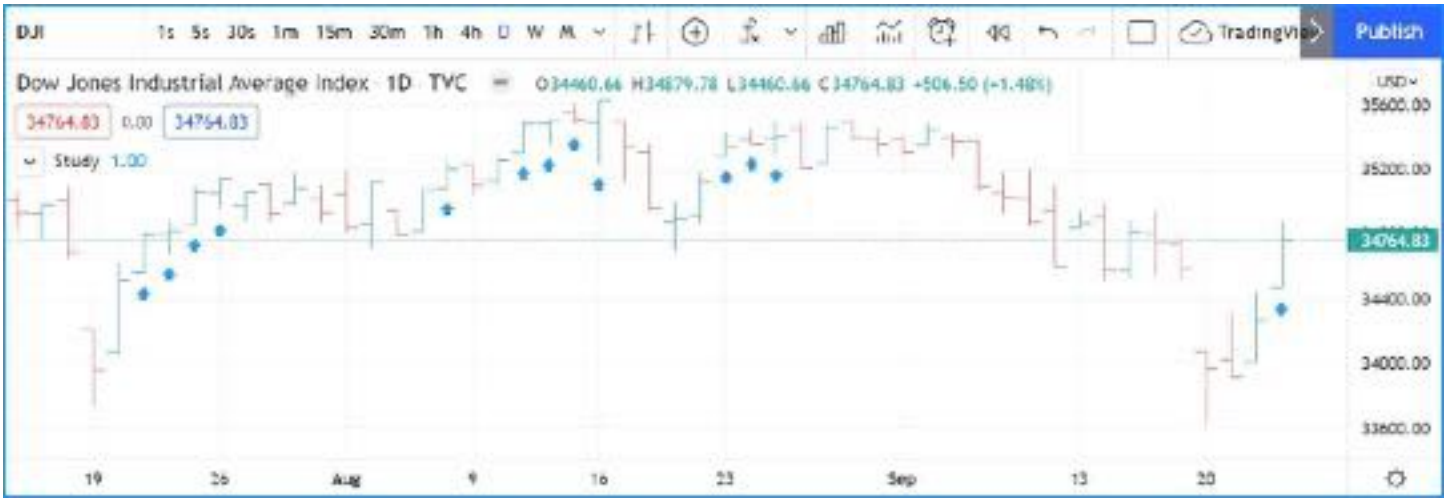


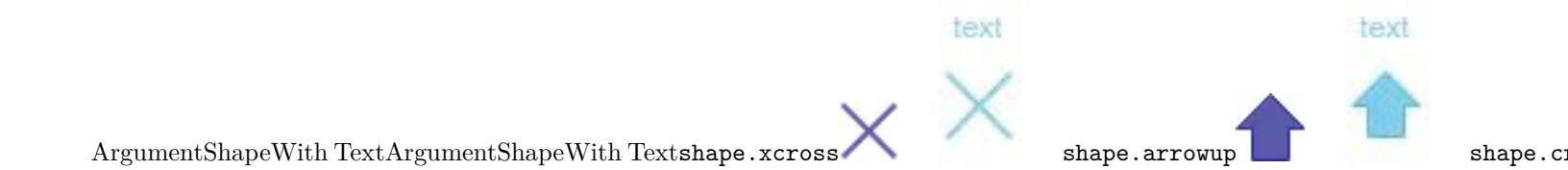
Figure 286: image

It is possible to use different plotshape() calls to superimpose text on bars. You will need to use `\n` followed by a special non-printing character that doesn't get stripped out to preserve the newline's functionality. Here we're using a Unicode Zero-width space (U+200E). While you don't see it in the following code's strings, it is there and can be copy/pasted. The special Unicode character needs to be the **last** one in the string for text going up, and the **first** one when you are plotting under the bar and text is going down:

```
//@version=6
indicator("Lift text", "", true)
plotshape(true, "", shape.arrowup,   location.abovebar, color.green,  text = "A")
plotshape(true, "", shape.arrowup,   location.abovebar, color.lime,   text = "B\n")
plotshape(true, "", shape.arrowdown, location.belowbar, color.red,    text = "C")
plotshape(true, "", shape.arrowdown, location.belowbar, color.maroon, text = "\nD")
```

Figure 287: image

The available shapes you can use with the `style` parameter are:



ArgumentShapeWith TextArgumentShapeWith Text`shape.xcross` `shape.arrowup` `shape.cr`
`plotarrow()`

The plotarrow function displays up or down arrows of variable length, based on the relative value of the series used in the function's first argument. It has the following syntax:

```
plotarrow(series, title, colorup, colordown, offset, minheight, maxheight, editable, show_last, display, force
```

See the Reference Manual entry for plotarrow() for details on its parameters.

The `series` parameter in plotarrow() is not a "series bool" as in plotchar() and plotshape(); it is a "series int/float" and there's more to it than a simple `true` or `false` value determining when the arrows are plotted. This is the logic governing how the argument supplied to `series` affects the behavior of plotarrow():

- `series > 0`: An up arrow is displayed, the length of which will be proportional to the relative value of the series on that bar in relation to other series values.
- `series < 0`: A down arrow is displayed, proportionally-sized using the same rules.
- `series == 0 or na(series)`: No arrow is displayed.

The maximum and minimum possible sizes for the arrows (in pixels) can be controlled using the `minheight` and `maxheight` parameters.

Here is a simple script illustrating how plotarrow() works:

```
//@version=6
indicator("", "", true)
body = close - open
plotarrow(body, colorup = color.teal, colordown = color.orange)
```

Note how the heigth of arrows is proportional to the relative size of the bar bodies.

You can use any series to plot the arrows. Here we use the value of the "Chaikin Oscillator" to control the location and size of the arrows:

```
//@version=6
indicator("Chaikin Oscillator Arrows", overlay = true)
```

Figure 288: image

```
fastLengthInput = input.int(3,  minval = 1)
slowLengthInput = input.int(10, minval = 1)
osc = ta.ema(ta.accdist, fastLengthInput) - ta.ema(ta.accdist, slowLengthInput)
plotarrow(osc)
```

Figure 289: image

Note that we display the actual "Chaikin Oscillator" in a pane below the chart, so you can see what values are used to determine the position and size of the arrows.

Labels

Labels are only available in v4 and higher versions of Pine Script™. They work very differently than plotchar() and plotshape().

Labels are objects, like lines and boxes, or tables. Like them, they are referred to using an ID, which acts like a pointer. Label IDs are of "label" type. As with other objects, labels IDs are "time series" and all the functions used to manage them accept "series" arguments, which makes them very flexible.

Labels are advantageous because:

- They allow "series" values to be converted to text and placed on charts. This means they are ideal to display values that cannot be known before time, such as price values, support and resistance levels, of any other values that your script calculates.
- Their positioning options are more flexible that those of the `plot*()` functions.
- They offer more display modes.
- Contrary to `plot*()` functions, label-handling functions can be inserted in conditional or loop structures, making it easier to control their behavior.
- You can add tooltips to labels.

One drawback to using labels versus plotchar() and plotshape() is that you can only draw a limited quantity of them on the chart. The default is ~50, but you can use the `max_labels_count` parameter in your indicator() or strategy() declaration statement to specify up to 500. Labels, like lines and boxes, are managed using a garbage collection mechanism which deletes the oldest ones on the chart, such that only the most recently drawn labels are visible.

Your toolbox of built-ins to manage labels are all in the `label` namespace. They include:

- label.new() to create labels.
- `label.set_*()` functions to modify the properties of an existing label.
- `label.get_*()` functions to read the properties of an existing label.
- label.delete() to delete labels
- The label.all array which always contains the IDs of all the visible labels on the chart. The array's size will depend on the maximum label count for your script and how many of those you have drawn. `aray.size(label.all)` will return the array's size.

Creating and modifying labels

The label.new() function creates a new label. It has the following signature:

```
label.new(x, y, text, xloc, yloc, color, style, textcolor, size, textalign, tooltip, force_overylay) → series
```

The *setter* functions allowing you to change a label's properties are:

- label.set_x()
- label.set_y()
- label.set_xy()
- label.set_text()
- label.set_xloc()
- label.set_yloc()
- label.set_color()
- label.set_style()
- label.set_textcolor()
- label.set_size()
- label.set_textalign()
- label.set_tooltip()

They all have a similar signature. The one for label.set_color() is:

```
label.set_color(id, color) → void
```

where:

- `id` is the ID of the label whose property is to be modified.
- The next parameter is the property of the label to modify. It depends on the setter function used. label.set_xy() changes two properties, so it has two such parameters.

This is how you can create labels in their simplest form:

```
//@version=6
indicator("", "", true)
label.new(bar_index, high)
```

Note that:

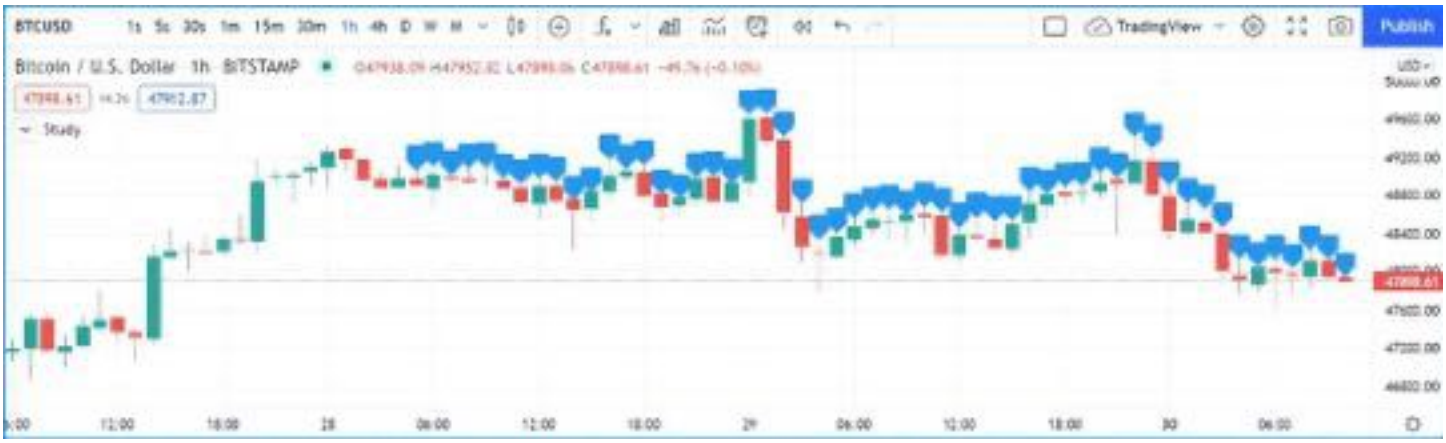


Figure 290: image

- The label is created with the parameters `x = bar_index` (the index of the current bar, bar_index) and `y = high` (the bar's high value).
- We do not supply an argument for the function's `text` parameter. Its default value being an empty string, no text is displayed.
- No logic controls our label.new() call, so labels are created on every bar.
- Only the last 54 labels are displayed because our indicator() call does not use the `max_labels_count` parameter to specify a value other than the ~50 default.
- Labels persist on bars until your script deletes them using label.delete(), or garbage collection removes them.

In the next example we display a label on the bar with the highest high value in the last 50 bars:

```
//@version=6
indicator("", "", true)

// Find the highest `high` in last 50 bars and its offset. Change it's sign so it is positive.
LOOKBACK = 50
hi = ta.highest(LOOKBACK)
highestBarOffset = - ta.highestbars(LOOKBACK)

// Create label on bar zero only.
var lbl = label.new(na, na, "", color = color.orange, style = label.style_label_lower_left)
// When a new high is found, move the label there and update its text and tooltip.
if ta.change(hi) != 0
    // Build label and tooltip strings.
    labelText = "High: " + str.tostring(hi, format.mintick)
    tooltipText = "Offest in bars: " + str.tostring(highestBarOffset) + "\nLow: " + str.tostring(low[highestBa
    // Update the label's position, text and tooltip.
    label.set_xy(lbl, bar_index[highestBarOffset], hi)
    label.set_text(lbl, labelText)
    label.set_tooltip(lbl, tooltipText)
```

Note that:

- We create the label on the first bar only by using the var keyword to declare the `lbl` variable that contains the label's ID. The `x`, `y` and `text` arguments in that label.new() call are irrelevant, as the label will be updated on further bars. We do, however, take care to use the `color` and `style` we want for the labels, so they don't need updating later.
- On every bar, we detect if a new high was found by testing for changes in the value of `hi`
- When a change in the high value occurs, we update our label with new information. To do this, we use three `label.set*()` calls to change the label's relevant information. We refer to our label using the `lbl` variable, which contains our label's ID. The script is thus maintaining the same label throughout all bars, but moving it and updating its information when a new high is detected.

Here we create a label on each bar, but we set its properties conditionally, depending on the bar's polarity:

```
//@version=6
indicator("", "", true)
```

Figure 291: image

```
lbl = label.new(bar_index, na)
if close >= open
    label.set_text( lbl, "green")
    label.set_color(lbl, color.green)
    label.set_yloc( lbl, yloc.belowbar)
    label.set_style(lbl, label.style_label_up)
else
    label.set_text( lbl, "red")
    label.set_color(lbl, color.red)
    label.set_yloc( lbl, yloc.abovebar)
    label.set_style(lbl, label.style_label_down)
```

Figure 292: image

Positioning labels

Labels are positioned on the chart according to x (bars) and y (price) coordinates. Five parameters affect this behavior: `x`, `y`, `xloc`, `yloc` and `style`:

`x`

Is either a bar index or a time value. When a bar index is used, the value can be offset in the past or in the future (maximum of 500 bars in the future). Past or future offsets can also be calculated when using time values. The `x` value of an existing label can be modified using label.set_x() or label.set_xy().

`xloc`

Is either xloc.bar_index (the default) or xloc.bar_time. It determines which type of argument must be used with `x`. With xloc.bar_index, `x` must be an absolute bar index. With xloc.bar_time, `x` must be a UNIX time in milliseconds corresponding to the time value of a bar's open. The `xloc` value of an existing label can be modified using label.set_xloc().

`y`

Is the price level where the label is positioned. It is only taken into account with the default `yloc` value of `yloc.price`. If `yloc` is yloc.abovebar or yloc.belowbar then the `y` argument is ignored. The `y` value of an existing label can be modified using label.set_y() or label.set_xy().

`yloc`

Can be yloc.price (the default), yloc.abovebar or yloc.belowbar. The argument used for `y` is only taken into account with yloc.price. The `yloc` value of an existing label can be modified using label.set_yloc().

`style`

The argument used has an impact on the visual appearance of the label and on its position relative to the reference point determined by either the `y` value or the top/bottom of the bar when yloc.abovebar or yloc.belowbar are used. The `style` of an existing label can be modified using label.set_style().

These are the available `style` arguments:

ArgumentLabelLabel with textArgumentLabelLabel with text`label.style_xcross` `label.style_label_up` using xloc.bar_time, the `x` value must be a UNIX timestamp in milliseconds. See the page on Time for more information. The start time of the current bar can be obtained from the time built-in variable. The bar time of previous bars is `time[1]`, `time[2]` and so on. Time can also be set to an absolute value with the timestamp function. You may add or subtract periods of time to achieve relative time offset.

Let's position a label one day ago from the date on the last bar:

```
//@version=6
indicator("")
daysAgoInput = input.int(1, tooltip = "Use negative values to offset in the future")
if barstate.islast
    MS_IN_ONE_DAY = 24 * 60 * 60 * 1000
    oneDayAgo = time - (daysAgoInput * MS_IN_ONE_DAY)
    label.new(oneDayAgo, high, xloc = xloc.bar_time, style = label.style_label_right)
```

Note that because of varying time gaps and missing bars when markets are closed, the positioning of the label may not always be exact. Time offsets of the sort tend to be more reliable on 24x7 markets.

You can also offset using a bar index for the `x` value, e.g.:

```
label.new(bar_index + 10, high)
label.new(bar_index - 10, high[10])
label.new(bar_index[10], high[10])
```

Reading label properties

The following *getter* functions are available for labels:

- label.get_x()
- label.get_y()
- label.get_text()

They all have a similar signature. The one for label.get_text() is:

```
label.get_text(id) → series string
```

where `id` is the label whose text is to be retrieved.

Cloning labels

The label.copy() function is used to clone labels. Its syntax is:

```
label.copy(id) → void
```

Deleting labels

The label.delete() function is used to delete labels. Its syntax is:

```
label.delete(id) → void
```

To keep only a user-defined quantity of labels on the chart, one could use code like this:

```
//@version=6
MAX_LABELS = 500
indicator("", max_labels_count = MAX_LABELS)
qtyLabelsInput = input.int(5, "Labels to keep", minval = 0, maxval = MAX_LABELS)
myRSI = ta.rsi(close, 20)
if myRSI > ta.highest(myRSI, 20)[1]
    label.new(bar_index, myRSI, str.tostring(myRSI, "#.00"), style = label.style_none)
    if array.size(label.all) > qtyLabelsInput
        label.delete(array.get(label.all, 0))
plot(myRSI)
```

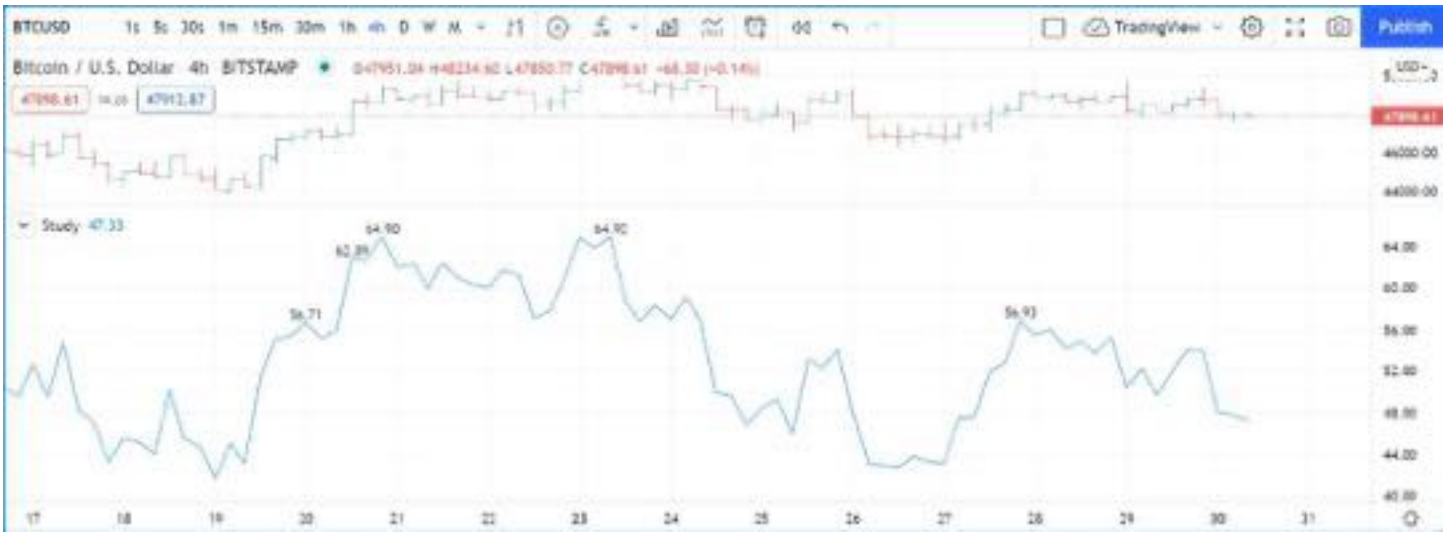


Figure 293: image

Note that:

- We define a `MAX_LABELS` constant to hold the maximum quantity of labels a script can accommodate. We use that value to set the `max_labels_count` parameter's value in our indicator() call, and also as the `maxval` value in our input.int() call to cap the user value.
- We create a new label when our RSI breaches its highest value of the last 20 bars. Note the offset of `[1]` we use in `if myRSI > ta.highest(myRSI, 20)[1]`. This is necessary. Without it, the value returned by ta.highest() would always include the current value of `myRSI`, so `myRSI` would never be higher than the function's return value.
- After that, we delete the oldest label in the label.all array that is automatically maintained by the Pine Script™ runtime and contains the ID of all the visible labels drawn by our script. We use the array.get() function to retrieve the array element at index zero (the oldest visible label ID). We then use label.delete() to delete the label linked with that ID.

Note that if one wants to position a label on the last bar only, it is unnecessary and inefficent to create and delete the label as the script executes on all bars, so that only the last label remains:

```
// INEFFICENT!
//@version=6
indicator("", "", true)
lbl = label.new(bar_index, high, str.tostring(high, format.mintick))
label.delete(lbl[1])
```

This is the efficient way to realize the same task:

```
//@version=6
indicator("", "", true)
if barstate.islast
    // Create the label once, the first time the block executes on the last bar.
    var lbl = label.new(na, na)
    // On all iterations of the script on the last bar, update the label's information.
    label.set_xy(lbl, bar_index, high)
    label.set_text(lbl, str.tostring(high, format.mintick))
```

Realtime behavior

Labels are subject to both *commit* and *rollback* actions, which affect the behavior of a script when it executes in the realtime bar. See the page on Pine Script™'s Execution model.

This script demonstrates the effect of rollback when running in the realtime bar:

```
//@version=6
indicator("", "", true)
label.new(bar_index, high)
```

On realtime bars, label.new() creates a new label on every script update, but because of the rollback process, the label created on the previous update on the same bar is deleted. Only the last label created before the realtime bar's close will be committed, and thus persist.

Text formatting

Drawing objects like labels, tables, and boxes have text-related properties that allow users to customize how an object's text appears on the chart. Some common properties include the text color, size, font family, and typographic emphasis.

Programmers can set an object's text properties when initializing it using the label.new(), box.new(), or table.cell() parameters. Alternatively, they can use the corresponding setter functions, e.g., label.set_text_font_family(), table.cell_set_text_color(), box.set_text_halign(), etc.

All three drawing objects have a `text_formatting` parameter, which sets the typographic emphasis to display **bold**, *italicized*, or unformatted text. It accepts the constants text.format_bold, text.format_italic, or text.format_none (no special formatting; default value). It also accepts `text.format_bold + text.format_italic` to display text that is both ***bold and italicized***.

The `size` parameter in label.new() and the `text_size` parameter in box.new() and table.cell() specify the size of the text displayed in the drawn objects. The parameters accept both "string" `size.*` constants and "int" typographic sizes. A "string" `size.*` constant represents one of six fixed sizing options. An "int" size value can be any positive integer, allowing scripts to replicate the `size.*` values or use other customized sizing.

This table lists the `size.*` constants and their equivalent "int" sizes for tables, boxes, and labels:

"string" constant"int" `text_size` in tables and boxes"int" `size` in labels`size.auto`00`size.tiny`8$_{7}$`size.small`10`size.normal`11412`size`
example below creates a label on the last bar to display its close price and creates a single-cell table to display the calculated bar move (the difference between the bar's open and close). The `closeLabel` text size is a user-selected "string" `size.*` constant, while the `barMoveTable` text size is a user-selected "int" value. The script also draws a box of the highest-lowest price range for the last 20 bars to assess the current price's position. Two "bool" inputs for "Bold" and "Italic" emphasis set the text formatting of the label, box, and table cell collectively. Enabling both inputs displays text that is both bold and italicized, while disabling both inputs displays text with no special formatting:

```
//@version=6
indicator("Text formatting demo", overlay = true)

//@variable The size of the `closeLabel` text, set using "string" `size.*` constants.
string closeLabelSize = input.string(size.large, "Label text size",
     [size.auto, size.tiny, size.small, size.normal, size.large, size.huge], group = "Text size")
//@variable The size of the `barMoveTable` text, set using "int" sizes.
int tableTextSize = input.int(25, "Table text size", minval = 0, group = "Text size")

// Toggles for the text formatting of all the drawing objects (`label`, `table` cell, and `box` texts).
```

Figure 294: image

```
bool formatBold   = input.bool(false, "Bold emphasis",   group = "Text formatting (all objects)")
bool formatItalic = input.bool(true,  "Italic emphasis", group = "Text formatting (all objects)")

// Track the highest and lowest prices in 20 bars. Used to draw a `box` of the high-low range.
float recentHighest = ta.highest(20)
float recentLowest  = ta.lowest(20)

if barstate.islast
    //@variable Label displaying `close` price on last bar. Text size is set using "string" constants.
    label closeLabel = label.new(bar_index, close, "Close price: " + str.tostring(close, "$0.00"),
         color = #EB9514D8, style = label.style_label_left, size = closeLabelSize)

    // Create a `table` cell to display the bar move (difference between `open` and `close` price).
    float barMove = close - open
    //@variable Single-cell table displaying the `barMove`. Cell text size is set using "int" values.
    var table barMoveTable = table.new(position.bottom_right, 1, 1, bgcolor = barMove > 0 ? #31E23FCC : #EE404
    barMoveTable.cell(0, 0, "Bar move = " + str.tostring(barMove, "$0.00") + "\n Percent = "
         + str.tostring(barMove / open, "0.00%"), text_halign = text.align_right, text_size = tableTextSize)

    // Draw a box to show where current price falls in the range of `recentHighest` to `recentLowest`.
    //@variable Box drawing the range from `recentHighest` to `recentLowest` in last 20 bars. Text size is set
    box rangeBox = box.new(bar_index - 20, recentHighest, bar_index + 1, recentLowest, text_size = 19,
         bgcolor = #A4B0F826, text_valign = text.align_top, text_color = #4A07E7D8)
    // Set box text to display how far current price is from the high or low of the range, depending on which
    rangeBox.set_text("Current price is " +
         (close >= (recentHighest + recentLowest) / 2 ? str.tostring(recentHighest - close, "$0.00") + " from
         : str.tostring(close - recentLowest, "$0.00") + " from box low"))

    // Set the text formatting of the `closeLabel`, `barMoveTable` cell, and `rangeBox` objects.
    // `formatBold` and `formatItalic` can both be `true` to combine formats, or both `false` for no special f
    switch
        formatBold and formatItalic =>
            closeLabel.set_text_formatting(text.format_bold + text.format_italic)
            barMoveTable.cell_set_text_formatting(0, 0, text.format_bold + text.format_italic)
            rangeBox.set_text_formatting(text.format_bold + text.format_italic)
        formatBold =>
            closeLabel.set_text_formatting(text.format_bold)
            barMoveTable.cell_set_text_formatting(0, 0, text.format_bold)
            rangeBox.set_text_formatting(text.format_bold)
        formatItalic =>
            closeLabel.set_text_formatting(text.format_italic)
```

```
            barMoveTable.cell_set_text_formatting(0, 0, text.format_italic)
            rangeBox.set_text_formatting(text.format_italic)
        =>
            closeLabel.set_text_formatting(text.format_none)
            barMoveTable.cell_set_text_formatting(0, 0, text.format_none)
            rangeBox.set_text_formatting(text.format_none)
```

[Previous

Tables](#tables)[Next

Time](#time) User Manual/Concepts/Time

Time

Introduction

In Pine Script™, the following key aspects apply when working with date and time values:

- **UNIX timestamp**: The native format for time values in Pine, representing the absolute number of *milliseconds* elapsed since midnight UTC on 1970-01-01. Several built-ins return UNIX timestamps directly, which users can format into readable dates and times. See the UNIX timestamps section below for more information.
- **Exchange time zone**: The time zone of the instrument's exchange. All calendar-based variables hold values expressed in the exchange time zone, and all built-in function overloads that have a `timezone` parameter use this time zone by default.
- **Chart time zone**: The time zone the chart and Pine Logs message prefixes use to express time values. Users can set the chart time zone using the "Timezone" input in the "Symbol" tab of the chart's settings. This setting only changes the *display* of dates and times on the chart and the times that prefix logged messages. It does **not** affect the behavior of Pine scripts because they cannot access a chart's time zone information.
- **`timezone` parameter**: A "string" parameter of time-related functions that specifies the time zone used in their calculations. For calendar-based functions, such as dayofweek(), the `timezone` parameter determines the time zone of the returned value. For functions that return UNIX timestamps, such as time(), the specified `timezone` defines the time zone of other applicable parameters, e.g., `session`. See the Time zone strings section to learn more.

UNIX timestamps

UNIX time is a standardized date and time representation that measures the number of *non-leap seconds* elapsed since January 1, 1970 at 00:00:00 UTC (the *UNIX Epoch*), typically expressed in seconds or smaller time units. A UNIX time value in Pine Script™ is an "int" *timestamp* representing the number of *milliseconds* from the UNIX Epoch to a specific point in time.

Because a UNIX timestamp represents the number of consistent time units elapsed from a fixed historical point (epoch), its value is **time zone-agnostic**. A UNIX timestamp in Pine always corresponds to the same distinct point in time, accurate to the millisecond, regardless of a user's location.

For example, the UNIX timestamp `1723472500000` always represents the time 1,723,472,500,000 milliseconds (1,723,472,500 seconds) after the UNIX Epoch. This timestamp's meaning does **not** change relative to any time zone.

To *format* an "int" UNIX timestamp into a readable date/time "string" expressed in a specific time zone, use the str.format_time() function. The function does not *modify* UNIX timestamps. It simply *represents* timestamps in a desired human-readable format.

For instance, the function can represent the UNIX timestamp `1723472500000` as a "string" in several ways, depending on its `format` and `timezone` arguments, without changing the *absolute* point in time that it refers to. The simple script below calculates three valid representations of this timestamp and displays them in the Pine Logs pane:

```
//@version=6
indicator("UNIX timestamps demo")

//@variable A UNIX time value representing the specific point 1,723,472,500,000 ms after the UNIX Epoch.
int unixTimestamp = 1723472500000

// These are a few different ways to express the `unixTimestamp` in a relative, human-readable format.
```

Figure 295: image

```
// Despite their format and time zone differences, all the calculated strings represent the SAME distinct poin
string isoExchange  = str.format_time(unixTimestamp)
string utcDateTime  = str.format_time(unixTimestamp, "MM/dd/yyyy HH:mm:ss.S", "UTC+0")
string utc4TimeDate = str.format_time(unixTimestamp, "hh:mm:ss a, MMMM dd, yyyy z", "UTC+4")

// Log the `unixTimestamp` and the custom "string" representations on the first bar.
if barstate.isfirst
    log.info(
         "\nUNIX time (ms): {0, number, #}\n
         ISO 8601 representation (Exchange time zone): {1}\n
         Custom date and time representation (UTC+0 time zone): {2}\n
         Custom time and date representation (UTC+4 time zone): {3}",
         unixTimestamp, isoExchange, utcDateTime, utc4TimeDate
     )
```

Note that:

- The value enclosed within square brackets in the logged message is an *automatic* prefix representing the historical time of the log.info() call in ISO 8601 format, expressed in the chart time zone.

See the Formatting dates and times section to learn more about representing UNIX timestamps with formatted strings.

Time zones

A time zone is a geographic region with an assigned *local time*. The specific time within a time zone is consistent throughout the region. Time zone boundaries typically relate to a location's longitude. However, in practice, they tend to align with administrative boundaries rather than strictly following longitudinal lines.

The local time within a time zone depends on its defined *offset* from Coordinated Universal Time (UTC), which can range from UTC-12:00 (12 hours *behind* UTC) to UTC+14:00 (14 hours *ahead* of UTC). Some regions maintain a consistent offset from UTC, and others have an offset that changes over time due to daylight saving time (DST) and other factors.

Two primary time zones apply to data feeds and TradingView charts: the *exchange time zone* and the *chart time zone*.

The exchange time zone represents the time zone of the current symbol's *exchange*, which Pine scripts can access with the syminfo.timezone variable. Calendar-based variables, such as month, dayofweek, and hour, always hold values expressed in the exchange time zone, and all time function overloads that have a `timezone` parameter use this time zone by default.

The chart time zone is a *visual preference* that defines how the chart and the time prefixes of Pine Logs represent time values. To set the chart time zone, use the "Timezone" input in the "Symbol" tab of the chart's settings or click on the current time shown below the chart. The specified time zone does **not** affect time calculations in Pine scripts because they cannot access this chart information. Although scripts cannot access a chart's time zone, programmers can provide inputs that users can adjust to match the time zone.

For example, the script below uses str.format_time() to represent the UNIX timestamps of the last historical bar's opening time and closing time as date-time strings, expressed in the function's default time zone, the exchange time zone, UTC-0,

and a user-specified time zone. It displays all four representations for comparison within a table in the bottom-right corner of the chart:

Figure 296: image

```
//@version=6
indicator("Time zone comparison demo", overlay = true)

//@variable The time zone of the time values in the last table row.
//          The "string" can contain either UTC offset notation or an IANA time zone identifier.
string timezoneInput = input.string("UTC+4:00", "Time zone")

//@variable A `table` showing strings representing bar times in three preset time zones and a custom time zone
var table displayTable = table.new(
     position.bottom_right, columns = 3, rows = 5, border_color = chart.fg_color, border_width = 2
 )

//@function Initializes three `displayTable` cells on the `row` that show the `title`, `text1`, and `text2` st
tableRow(int row, string title, string text1, string text2, color titleColor = #9b27b066, color infoColor = na
    displayTable.cell(0, row, title, bgcolor = titleColor, text_color = chart.fg_color)
    displayTable.cell(1, row, text1, bgcolor = infoColor,  text_color = chart.fg_color)
    displayTable.cell(2, row, text2, bgcolor = infoColor,  text_color = chart.fg_color)

if barstate.islastconfirmedhistory
    // Draw an empty label to signify the bar that the displayed time strings represent.
    label.new(bar_index, high, color = #9b27b066, size = size.huge)

    //@variable The formatting string for all `str.format_time()` calls. Sets the format of the date-time stri
    var string formatString = "yyyy-MM-dd  HH:mm:ss"
    // Initialize a header row at the top of the `displayTable`.
    tableRow(0, "", "OPEN time", "CLOSE time", na, #9b27b066)
    // Initialize a row showing the bar's times in the default time zone (no specified `timezone` arguments).
    tableRow(1, "Default", str.format_time(time, formatString), str.format_time(time_close, formatString))
    // Initialize a row showing the bar's times in the exchange time zone (`syminfo.timezone`).
    tableRow(2, "Exchange: " + syminfo.timezone,
         str.format_time(time, formatString, syminfo.timezone),
         str.format_time(time_close, formatString, syminfo.timezone)
     )
    // Initialize a row showing the bar's times in the UTC-0 time zone (using "UTC" as the `timezone` argument
    tableRow(3, "UTC-0", str.format_time(time, formatString, "UTC"), str.format_time(time_close, formatString,

    // Initialize a row showing the bar's times in the custom time zone (`timezoneInput`).
```

```
    tableRow(
         4, "Custom: " + timezoneInput,
         str.format_time(time, formatString, timezoneInput),
         str.format_time(time_close, formatString, timezoneInput)
     )
```

Note that:

- The label on the chart signifies which bar's times the displayed strings represent.
- The "Default" and "Exchange" rows in the table show identical results because syminfo.timezone is the str.format_time() function's default `timezone` argument.
- The exchange time zone on our example chart appears as `"America/New_York"`, the IANA identifier for the NASDAQ exchange's time zone. It represents UTC-4 *or* UTC-5, depending on the time of year. See the next section to learn more about time zone strings.

Time zone strings

All built-in functions with a `timezone` parameter accept a "string" argument specifying the time zone they use in their calculations. These functions can accept time zone strings in either of the following formats:

- **UTC** (or *GMT*) offset notation, e.g., `"UTC-5"`, `"UTC+05:30"`, `"GMT+0100"`
- **IANA database** notation, e.g., `"America/New_York"`, `"Asia/Calcutta"`, `"Europe/Paris"`

The IANA time zone database reference page lists possible time zone identifiers and their respective UTC offsets. The listed identifiers are valid as `timezone` arguments.

Note that various time zone strings expressed in UTC or IANA notation can represent the *same* offset from Coordinated Universal Time. For instance, these strings all represent a local time three hours ahead of UTC:

- `"UTC+3"`
- `"GMT+03:00"`
- `"Asia/Kuwait"`
- `"Europe/Moscow"`
- `"Africa/Nairobi"`

For the str.format_time() function and the functions that calculate calendar-based values from a UNIX timestamp, including month(), dayofweek(), and hour(), the "string" passed to the `timezone` parameter changes the returned value's calculation to express the result in the specified time zone. See the Formatting dates and times and Calendar-based functions sections for more information.

The example below shows how time zone strings affect the returned values of calendar-based functions. This script uses three hour() function calls to calculate "int" values representing the opening hour of each bar in the exchange time zone, UTC-0, and a user-specified UTC offset. It plots all three calculated hours in a separate pane for comparison:

```
//@version=6
indicator("Time zone strings in calendar functions demo")

//@variable An "int" representing the user-specified hourly offset from UTC.
int utcOffsetInput = input.int(defval = 4, title ="Timezone offset UTC (+/-)", minval = -12, maxval = 14)

//@variable A valid time zone string based on the `utcOffsetInput`, in UTC offset notation (e.g., "UTC-4").
string customOffset = "UTC" + (utcOffsetInput > 0 ? "+" : "") + str.tostring(utcOffsetInput)

//@variable The bar's opening hour in the exchange time zone (default). Equivalent to the `hour` variable.
int exchangeHour = hour(time)
//@variable The bar's opening hour in the "UTC-0" time zone.
int utcHour = hour(time, "UTC-0")
//@variable The bar's opening hour in the `customOffset` time zone.
int customOffsetHour = hour(time, customOffset)

// Plot the `exchangeHour`, `utcHour`, and `customOffsetHour` for comparison.
plot(exchangeHour,     "Exchange hour",      #E100FF5B,    8)
plot(utcHour,          "UTC-0 hour",         color.blue,   3)
plot(customOffsetHour, "Custom offset hour", color.orange, 3)
```

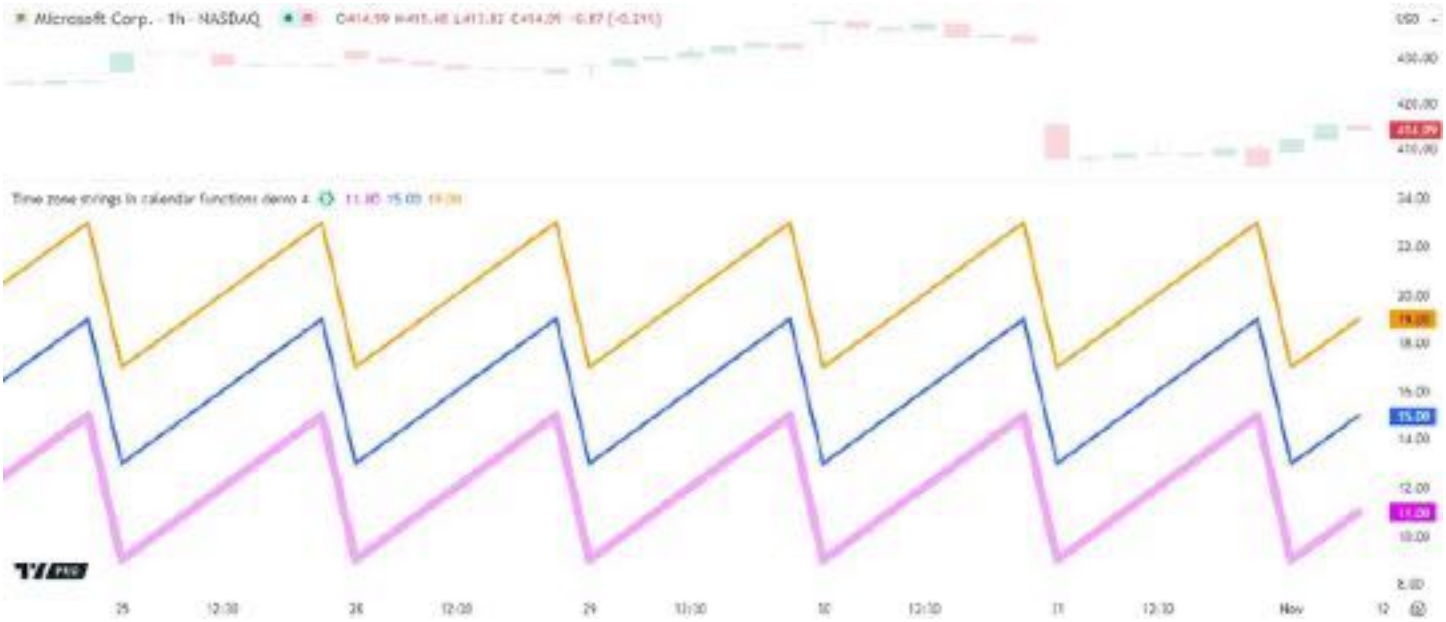


Figure 297: image

Note that:

- The `exchangeHour` value is four *or* five hours behind the `utcHour` because the NASDAQ exchange is in the "America/New_York" time zone. This time zone has a UTC offset that *changes* during the year due to daylight saving time (DST). The script's default `customOffsetHour` is consistently four hours ahead of the `utcHour` because its time zone is UTC+4.
- The call to the hour() function without a specified `timezone` argument returns the same value that the hour*variable* holds because both represent the bar's opening hour in the exchange time zone (syminfo.timezone).

For functions that return UNIX timestamps directly, such as time() and timestamp(), the `timezone` parameter defines the time zone of the function's calendar-based *parameters*, including `session`, `year`, `month`, `day`, `hour`, `minute`, and `second`. The parameter does *not* determine the time zone of the returned value, as UNIX timestamps are *time zone-agnostic*. See the Testing for sessions and `timestamp()` sections to learn more.

The following script calls the timestamp() function to calculate the UNIX timestamp of a specific date and time, and it draws a label at the timestamp's corresponding bar location. The user-selected `timezone` argument (`timezoneInput`) determines the time zone of the call's calendar-based arguments. Consequently, the calculated timestamp varies with the `timezoneInput` value because identical local times in various time zones correspond to *different* amounts of time elapsed since the UNIX Epoch:

```
//@version=6
indicator("Time zone strings in UNIX timestamp functions demo", overlay = true)

//@variable The `timezone` argument of the `timestamp()` call, which sets the time zone of all date and time p
string timezoneInput = input.string("Etc/UTC", "Time zone")

//@variable The UNIX timestamp corresponding to a specific calendar date and time.
//          The specified `year`, `month`, `day`, `hour`, `minute`, and `second` represent calendar values in
//          `timezoneInput` time zone.
//          Different `timezone` arguments produce different UNIX timestamps because an identical date in anot
//          time zone does NOT represent the same absolute point in time.
int unixTimestamp = timestamp(
     timezone = timezoneInput, year = 2024, month = 10, day = 31, hour = 0, minute = 0, second = 0
 )

//@variable The `close` value when the bar's opening time crosses the `unixTimestamp`.
float labelPrice = ta.valuewhen(ta.cross(time, unixTimestamp), close, 0)

// On the last historical bar, draw a label showing the `unixTimestamp` value at the corresponding bar locatio
```

Figure 298: image

```
if barstate.islastconfirmedhistory
    label.new(
         unixTimestamp, nz(labelPrice, close), "UNIX timestamp: " + str.tostring(unixTimestamp),
         xloc.bar_time, yloc.price, chart.fg_color, label.style_label_down, chart.bg_color, size.large
     )
```

Note that:

- `"Etc/UTC"` is the *IANA identifier* for the UTC+0 time zone.
- The label.new() call uses xloc.bar_time as its `xloc` argument, which is required to anchor the drawing to an absolute time value. Without this argument, the function treats the `unixTimestamp` as a relative bar index, leading to an incorrect location.
- The label's `y` value is the close of the bar where the opening time crosses the `unixTimestamp`. If the timestamp represents a future time, the label uses the last historical bar's price.

Although time zone strings can use either UTC or IANA notation, we recommend using *IANA notation* for `timezone` arguments in most cases, especially if a script's time calculations must align with the observed time offset in a specific country or subdivision. When a time function call uses an IANA time zone identifier for its `timezone` argument, its calculations adjust automatically for historical and future changes to the specified region's observed time, such as daylight saving time (DST) and updates to time zone boundaries, instead of using a fixed offset from UTC.

The following script demonstrates how UTC and IANA time zone strings can affect time calculations differently. It uses two calls to the hour() function to calculate the hour from the current bar's opening timestamp using `"UTC-4"` and `"America/New_York"` as `timezone` arguments. The script plots the results of both calls for comparison and colors the main pane's background when the returned values do not match. Although these two hour() calls may seem similar because UTC-4 is an observed UTC offset in New York, they *do not* always return the same results, as shown below:

```
//@version=6
indicator("UTC vs IANA time zone strings demo")

//@variable The hour of the current `time` in the "UTC-4" time zone.
//          This variable's value represents the hour in New York only during DST. It is one hour ahead otherw
int hourUTC = hour(time, "UTC-4")
//@variable The hour of the current `time` in the "America/New_York" time zone.
//          This form adjusts to UTC offset changes automatically, so the value always represents the hour in
int hourIANA = hour(time, "America/New_York")

//@variable Is translucent blue when `hourUTC` does not equal `hourIANA`, `na` otherwise.
color bgColor = hourUTC != hourIANA ? color.rgb(33, 149, 243, 80) : na

// Plot the values of `hourUTC` and `hourIANA` for comparison.
plot(hourUTC,  "UTC-4",            color.blue,   linewidth = 6)
```

Figure 299: image

```
plot(hourIANA, "America/New_York", color.orange, linewidth = 3)
// Highlight the main chart pane with the `bgColor`.
bgcolor(bgColor, title = "Unequal result highlight", force_overlay = true)
```

The plots in the chart above diverge periodically because New York observes daylight saving time, meaning its UTC offset *changes* at specific points in a year. During DST, New York's local time follows UTC-4. Otherwise, it follows UTC-5. Because the script's first hour() call uses `"UTC-4"` as its `timezone` argument, it returns the correct hour in New York *only* during DST. In contrast, the call that uses the `"America/New_York"` time zone string adjusts its UTC offset automatically to return the correct hour in New York at *any* time of the year.

Time variables

Pine Script™ has several built-in variables that provide scripts access to different forms of time information:

- The time and time_close variables hold UNIX timestamps representing the current bar's opening and closing times, respectively.
- The time_tradingday variable holds a UNIX timestamp representing the starting time of the last UTC calendar day in a session.
- The timenow variable holds a UNIX timestamp representing the current time when the script executes.
- The year, month, weekofyear, dayofmonth, dayofweek, hour, minute, and second variables reference calendar values based on the current bar's opening time, expressed in the exchange time zone.
- The last_bar_time variable holds a UNIX timestamp representing the last available bar's opening time.
- The chart.left_visible_bar_time and chart.right_visible_bar_time variables hold UNIX timestamps representing the opening times of the leftmost and rightmost visible chart bars.
- The syminfo.timezone variable holds a "string" value representing the time zone of the current symbol's exchange in IANA database notation. All time-related function overloads with a `timezone` parameter use this variable as the default argument.

`time` and `time_close` variables

The time variable holds the UNIX timestamp of the current bar's *opening time*, and the time_close variable holds the UNIX timestamp of the bar's *closing time*.

These timestamps are unique, time zone-agnostic "int" values, which programmers can use to anchor drawing objects to specific bar times, calculate and inspect bar time differences, construct readable date/time strings with the str.format_time() function, and more.

The script below displays bar opening and closing times in different ways. On each bar, it formats the time and time_close timestamps into strings containing the hour, minute, and second in the exchange time zone, and it draws labels displaying

the formatted strings at the open and close prices. Additionally, the script displays strings containing the unformatted UNIX timestamps of the last chart bar within a table in the bottom-right corner:

Figure 300: image

```
//@version=6
indicator("`time` and `time_close` demo", overlay = true, max_labels_count = 500)

//@variable A "string" representing the hour, minute, and second of the bar's opening time in the exchange tim
string openTimeString = str.format_time(time, "HH:mm:ss")
//@variable A "string" representing the hour, minute, and second of the bar's closing time in the exchange tim
string closeTimeString = str.format_time(time_close, "HH:mm:ss")

//@variable Is `label.style_label_down` when the `open` is higher than `close`, `label.style_label_up` otherwi
string openLabelStyle  = open > close ? label.style_label_down : label.style_label_up
//@variable Is `label.style_label_down` when the `close` is higher than `open`, `label.style_label_up` otherwi
string closeLabelStyle = close > open ? label.style_label_down : label.style_label_up

// Draw labels anchored to the bar's `time` to display the `openTimeString` and `closeTimeString`.
label.new(time, open,  openTimeString,  xloc.bar_time, yloc.price, color.orange, openLabelStyle,  color.white)
label.new(time, close, closeTimeString, xloc.bar_time, yloc.price, color.blue,   closeLabelStyle, color.white)

if barstate.islast
    //@variable A `table` displaying the last bar's *unformatted* UNIX timestamps.
    var table t = table.new(position.bottom_right, 2, 2, bgcolor = #ffe70d)
    // Populate the `t` table with "string" representations of the the "int" `time` and `time_close` values.
    t.cell(0, 0, "`time`")
    t.cell(1, 0, str.tostring(time))
    t.cell(0, 1, "`time_close`")
    t.cell(1, 1, str.tostring(time_close))
```

Note that:

- This script's label.new() calls include xloc.bar_time as the `xloc` argument and time as the `x` argument to anchor the drawings to bar opening times.
- The formatted strings express time in the exchange time zone because we did not specify `timezone` arguments in the str.format_time() calls. NYSE, our chart symbol's exchange, is in the "America/New_York" time zone (UTC-4/-5).
- Although our example chart uses an *hourly* timeframe, the table and the labels at the end of the chart show that the last bar closes only *30 minutes* (1,800,000 milliseconds) after opening. This behavior occurs because the chart aligns bars with session opening and closing times. A session's final bar closes when the session ends, and a new bar opens

when a new session starts. A typical session on our 60-minute chart with regular trading hours (RTH) spans from 09:30 to 16:00 (6.5 hours). The chart divides this interval into as many 60-minute bars as possible, starting from the session's opening time, which leaves only 30 minutes for the final bar to cover.

It's crucial to note that unlike the time variable, which has consistent behavior across chart types, time_close behaves differently on *time-based* and *non-time-based* charts.

Time-based charts have bars that typically open and close at regular, *predictable* times within a session. Thanks to this predictability, time_close can accurately represent the *expected* closing time of an open bar on a time-based chart, as shown on the last bar in the example above.

In contrast, the bars on tick charts and *price-based* charts (all non-standard charts excluding Heikin Ashi) cover *irregular* time intervals. Tick charts construct bars based on successive ticks in the data feed, and price-based charts construct bars based on significant price movements. The time it takes for new ticks or price changes to occur is *unpredictable*. As such, the time_close value is na on the *realtime bars* of these charts.

The following script uses the time and time_close variables with str.tostring() and str.format_time() to create strings containing bar opening and closing UNIX timestamps and formatted date-time representations, which it displays in labels at each bar's high and low prices.

When applied to a Renko chart, which forms new bars based on *price movements*, the labels show correct results on all historical bars. However, the last bar has a time_close value of na because the future closing time is unpredictable. Consequently, the bar's closing time label shows a timestamp of `"NaN"` and an *incorrect* date and time:

Figure 301: image

```
//@version=6
indicator("`time_close` on non-time-based chart demo", overlay = true)

//@variable A formatted "string" containing the date and time that `time_close` represents in the exchange tim
string formattedCloseTime = str.format_time(time_close, format = "'Date and time:' yyyy-MM-dd HH:mm:ss")
//@variable A formatted "string" containing the date and time that `time` represents in the exchange time zone
string formattedOpenTime = str.format_time(time, format = "'Date and time:' yyyy-MM-dd HH:mm:ss")

//@variable A "string" containing the `time_close` UNIX timestamp and the `formattedCloseTime`.
string closeTimeText = str.format("Close timestamp: {0,number,#}\n{1}", time_close, formattedCloseTime)
//@variable A "string" containing the `time` UNIX timestamp and the `formattedOpenTime`.
string openTimeText = str.format("Open timestamp: {0,number,#}\n{1}", time, formattedOpenTime)

// Define label colors for historical and realtime bars.
color closeLabelColor = barstate.islast ? color.purple : color.aqua
```

```
color openLabelColor  = barstate.islast ? color.green  : color.orange

// Draw a label at the `high` to display the `closeTimeText` and a label at the `low` to display the `openTime`
// both anchored to the bar's `time`.
label.new(
      time, high, closeTimeText, xloc.bar_time, color = closeLabelColor, textcolor = color.white,
      size = size.large, textalign = text.align_left
 )
label.new(
      time, low, openTimeText, xloc.bar_time, color = openLabelColor, style = label.style_label_up,
      size = size.large, textcolor = color.white, textalign = text.align_left
 )

// Highlight the background yellow on the latest bar.
bgcolor(barstate.islast ? #f3de22cb : na, title = "Latest bar highlight")
```

Note that:

- The script draws up to 50 labels because we did not specify a `max_labels_count` argument in the indicator() declaration statement.
- The str.format_time() function replaces na values with 0 in its calculations, which is why it returns an incorrect date-time "string" on the last bar. A timestamp of 0 corresponds to the *UNIX Epoch* (00:00:00 UTC on January 1, 1970). However, the str.format_time() call does not specify a `timezone` argument, so it expresses the epoch's date and time in the *exchange time zone*, which was five hours behind UTC at that point in time.
- The time_close() function, which returns the closing timestamp of a bar on a specified timeframe within a given session, also returns na on the realtime bars of tick-based and price-based charts.

time_tradingday

The time_tradingday variable holds a UNIX timestamp representing the starting time (00:00 UTC) of the last trading day in the current bar's final session. It is helpful primarily for date and time calculations on *time-based* charts for symbols with overnight sessions that start and end on *different* calendar days.

On "1D" and lower timeframes, the time_tradingday timestamp corresponds to the beginning of the day when the current session *ends*, even for bars that open and close on the previous day. For example, the "Monday" session for "EURUSD" starts on Sunday at 17:00 and ends on Monday at 17:00 in the exchange time zone. The time_tradingday values of *all* intraday bars within the session represent Monday at 00:00 UTC.

On timeframes higher than "1D", which can cover *multiple* sessions, time_tradingday holds the timestamp representing the beginning of the last calendar day of the bar's *final* trading session. For example, on a "EURUSD, 1W" chart, the timestamp represents the start of the last trading day in the week, which is typically Friday at 00:00 UTC.

The script below demonstrates how the time_tradingday and time variables differ on Forex symbols. On each bar, it draws labels to display strings containing the variables' UNIX timestamps and formatted dates and times. It also uses the dayofmonth() function to calculate the UTC calendar day from both timestamps, highlighting the background when the calculated days do not match.

When applied to the "FXCM:EURUSD" chart with the "3h" ("180") timeframe, the script highlights the background of the *first bar* in each session, as each session opens on the *previous* calendar day. The dayofmonth() call that uses time calculates the opening day on the session's first bar, whereas the call that uses time_tradingday calculates the day when the session *ends*:

```
//@version=6
indicator("`time_tradingday` demo", overlay = true)

//@variable A concatenated "string" containing the `time_tradingday` timestamp and a formatted representation
string tradingDayText = "`time_tradingday`: " + str.tostring(time_tradingday) + "\n"
     + "Date and time: " + str.format_time(time_tradingday, "dd MMM yyyy, HH:mm (z)", "UTC+0")
//@variable A concatenated "string" containing the `time` timestamp and a formatted representation in UTC.
string barOpenText = "`time`: " + str.tostring(time) + "\n"
     + "Date and time: " + str.format_time(time, "dd MMM yyyy, HH:mm (z)", "UTC+0")

//@variable Is `true` on every even bar, `false` otherwise. This condition determines the appearance of the la
bool isEven = bar_index % 2 == 0
```

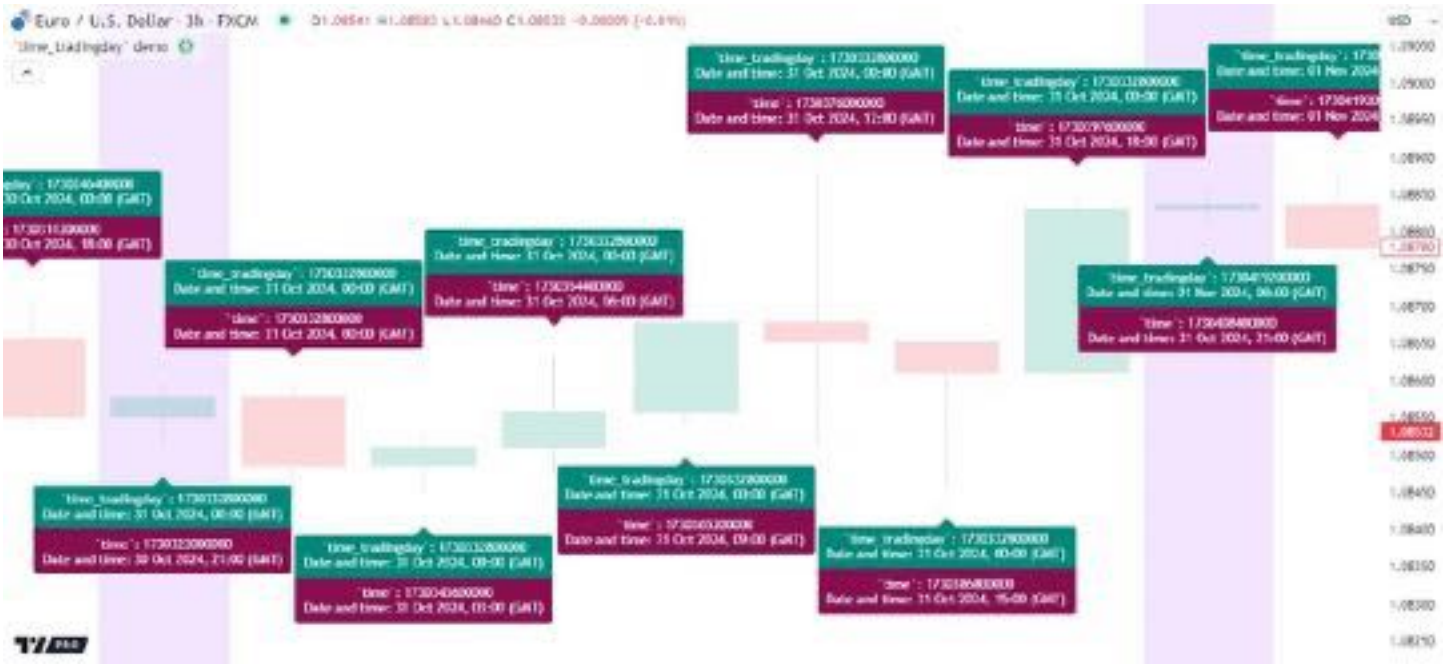


Figure 302: image

```
// The `yloc` and `style` properties of the labels. They alternate on every other bar for visibility.
labelYloc  = isEven ? yloc.abovebar : yloc.belowbar
labelStyle = isEven ? label.style_label_down : label.style_label_up
// Draw alternating labels anchored to the bar's `time` to display the `tradingDayText` and `barOpenText`.
if isEven
    label.new(time, 0, tradingDayText + "\n\n\n", xloc.bar_time, labelYloc, color.teal, labelStyle, color.whit
    label.new(time, 0, barOpenText, xloc.bar_time, labelYloc, color.maroon, labelStyle, color.white)
else
    label.new(time, 0, "\n\n\n" + barOpenText, xloc.bar_time, labelYloc, color.maroon, labelStyle, color.white
    label.new(time, 0, tradingDayText, xloc.bar_time, labelYloc, color.teal, labelStyle, color.white)

//@variable The day of the month, in UTC, that the `time_tradingday` timestamp corresponds to.
int tradingDayOfMonth = dayofmonth(time_tradingday, "UTC+0")
//@variable The day of the month, in UTC, that the `time` timestamp corresponds to.
int openingDayOfMonth = dayofmonth(time, "UTC+0")

// Highlight the background when the `tradingDayOfMonth` does not equal the `openingDayOfMonth`.
bgcolor(tradingDayOfMonth != openingDayOfMonth ? color.rgb(174, 89, 243, 85) : na, title = "Different day high
```

Note that:

- The str.format_time() and dayofmonth() calls use `"UTC+0"` as the `timezone` argument, meaning the results represent calendar time values with no offset from UTC. In the screenshot, the first bar opens at 21:00 UTC, 17:00 in the exchange time zone ("America/New_York").
- The formatted strings show `"GMT"` as the acronym of the time zone, which is equivalent to `"UTC+0"` in this context.
- The time_tradingday value is the same for *all* three-hour bars within each session, even for the initial bar that opens on the previous UTC calendar day. The assigned timestamp changes only when a new session starts.

`timenow`

The timenow variable holds a UNIX timestamp representing the script's *current time*. Unlike the values of other variables that hold UNIX timestamps, the values in the timenow series correspond to times when the script *executes*, not the times of specific bars or trading days.

A Pine script executes only *once* per historical bar, and all historical executions occur when the script first *loads* on the chart. As such, the timenow value is relatively consistent on historical bars, with only occasional millisecond changes across the

series. In contrast, on realtime bars, a script executes once for *each new update* in the data feed, which can happen several times per bar. With each new execution, the timenow value updates on the latest bar to represent the current time.

This variable is most useful on realtime bars, where programmers can apply it to track the times of the latest script executions, count the time elapsed within open bars, control drawings based on bar updates, and more.

The script below inspects the value of timenow on the latest chart bars and uses it to analyze realtime bar updates. When the script first reaches the last chart bar, it declares three variables with the varip keyword to hold the latest timenow value, the total time elapsed between the bar's updates, and the total number of updates. It uses these values to calculate the average number of milliseconds between updates, which it displays in a label along with the current execution's timestamp, a formatted time and date in the exchange time zone, and the current number of bar updates:

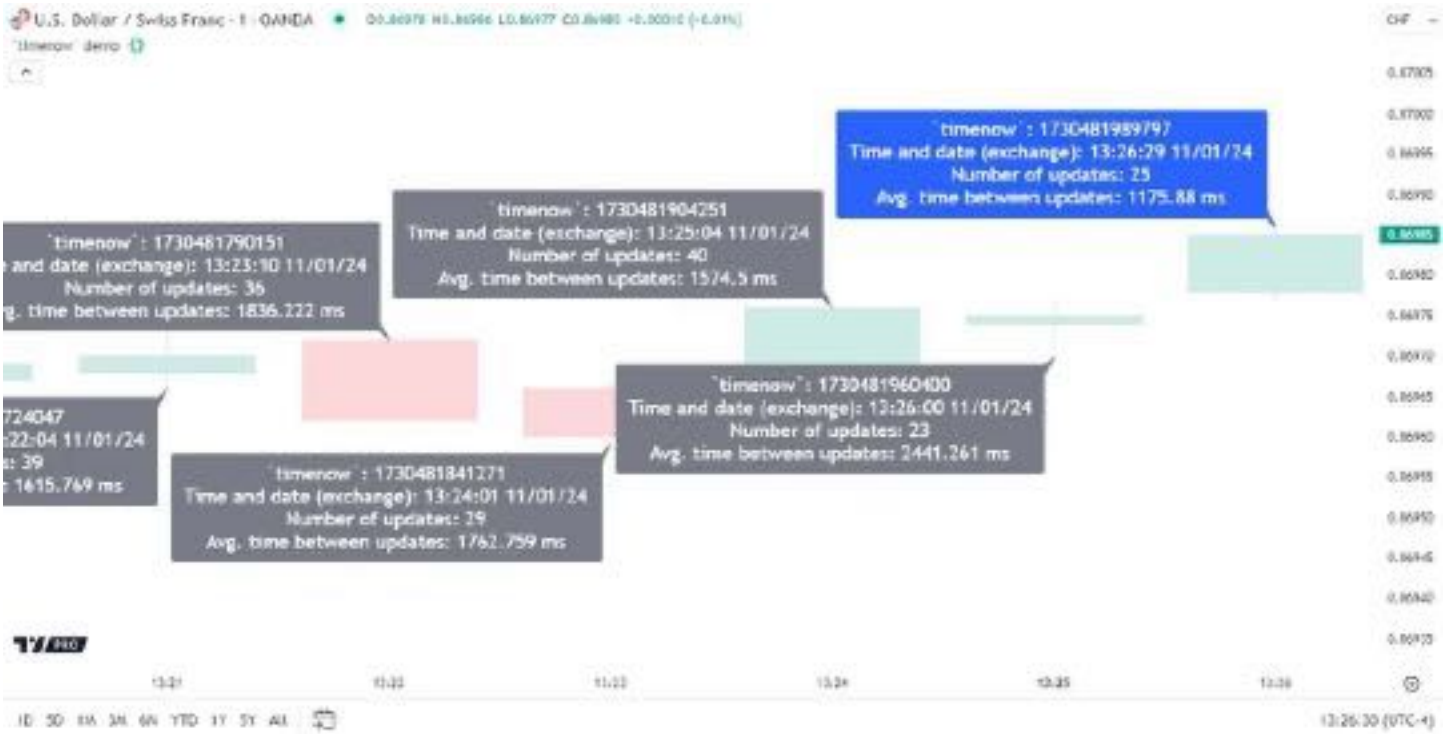


Figure 303: image

```
//@version=6
indicator("`timenow` demo", overlay = true, max_labels_count = 500)

if barstate.islast
    //@variable Holds the UNIX timestamp of the latest script execution for timing updates in the data feed.
    varip int lastUpdateTimestamp = timenow
    //@variable The total number of milliseconds elapsed across the bar's data updates.
    varip int totalUpdateTime = 0
    //@variable The number of updates that have occurred on the current bar.
    varip int numUpdates = 0

    // Add the time elapsed from the `lastUpdateTimestamp` to `totalUpdateTime`, increase the `numUpdates` cou
    // update the `lastUpdateTimestamp` when the `timenow` value changes.
    if timenow != lastUpdateTimestamp
        totalUpdateTime     += timenow - lastUpdateTimestamp
        numUpdates          += 1
        lastUpdateTimestamp := timenow

    //@variable The average number of milliseconds elapsed between the bar's updates.
    float avgUpdateTime = nz(totalUpdateTime / numUpdates)

    //@variable Contains the `timenow` value, a custom representation, and the `numUpdates` and `avgUpdateTime
    string displayText = "`timenow`: " + str.tostring(timenow)
```

```
                        + "\nTime and date (exchange): " + str.format_time(timenow, "HH:mm:ss MM/dd/yy")
                        + "\nNumber of updates: " + str.tostring(numUpdates)
                        + "\nAvg. time between updates: " + str.tostring(avgUpdateTime, "#.###") + " ms"

    //@variable The color of the label. Is blue when the bar is open, and gray after it closes.
    color labelColor = barstate.isconfirmed ? color.gray : color.blue
    //@variable The label's y-coordinate. Alternates between `high` and `low` on every other bar.
    float labelPrice = bar_index % 2 == 0 ? high : low
    //@variable The label's style. Alternates between "lower-right" and "upper-right" styles.
    labelStyle = bar_index % 2 == 0 ? label.style_label_lower_right : label.style_label_upper_right
    // Draw a `labelColor` label anchored to the bar's `time` to show the `displayText`.
    label.new(
         time, labelPrice, displayText, xloc.bar_time, color = labelColor, style = labelStyle,
         textcolor = color.white, size = size.large
     )

    // Reset the `totalUpdateTime` and `numUpdates` counters when the bar is confirmed (closes).
    if barstate.isconfirmed
        totalUpdateTime := 0
        numUpdates      := 0
```

Note that:

- When a bar is unconfirmed (open), the label is blue to signify that additional updates can occur. After the bar is confirmed (closed), the label turns gray.
- Although we've set the chart time zone to match the exchange time zone, the formatted time in the open bar's label and the time shown below the chart *do not* always align. The script records a new timestamp *only* when a new execution occurs, whereas the time below the chart updates *continuously*.
- The varip keyword specifies that a variable does not revert to the last committed value in its series when new updates occur. This behavior allows the script to use variables to track changes in timenow on an open bar.
- Updates to timenow on open realtime bars do not affect the recorded timestamps on confirmed bars as the script executes. However, the historical series changes (*repaints*) after reloading the chart because timenow references the script's *current time*, not the times of specific bars.

Calendar-based variables

The year, month, weekofyear, dayofmonth, dayofweek, hour, minute, and second variables hold *calendar-based* "int" values calculated from the current bar's *opening time*, expressed in the exchange time zone. These variables reference the same values that calendar-based functions return when they use the default `timezone` argument and time as the `time` argument. For instance, the year variable holds the same value that a year(time) call returns.

Programmers can use these calendar-based variables for several purposes, such as:

- Identifying a bar's opening date and time.
- Passing the variables to the timestamp() function to calculate UNIX timestamps.
- Testing when date/time values or ranges occur in a data feed.

One of the most common use cases for these variables is checking for date or time ranges to control when a script displays visuals or executes calculations. This simple example inspects the year variable to determine when to plot a visible value. If the year is 2022 or higher, the script plots the bar's close. Otherwise, it plots na:

```
//@version=6
indicator("`year` demo", overlay = true)

// Plot the `close` price on bars that open in the `year` 2022 onward. Otherwise, plot `na` to display nothing
plot(year >= 2022 ? close : na, "`close` price (year 2022 and later)", linewidth = 3)
```

When using these variables in conditions that isolate specific dates or times rather than ranges, it's crucial to consider that certain conditions might not detect some occurrences of the values due to a chart's timeframe, the opening times of chart bars, or the symbol's active session.

For instance, suppose we want to detect when the first calendar day of each month occurs on the chart. Intuitively, one might consider simply checking when the dayofmonth value equals 1. However, this condition only identifies when a bar *opens* on a month's first day. The bars on some charts can open and close in *different* months. Additionally, a chart bar might not

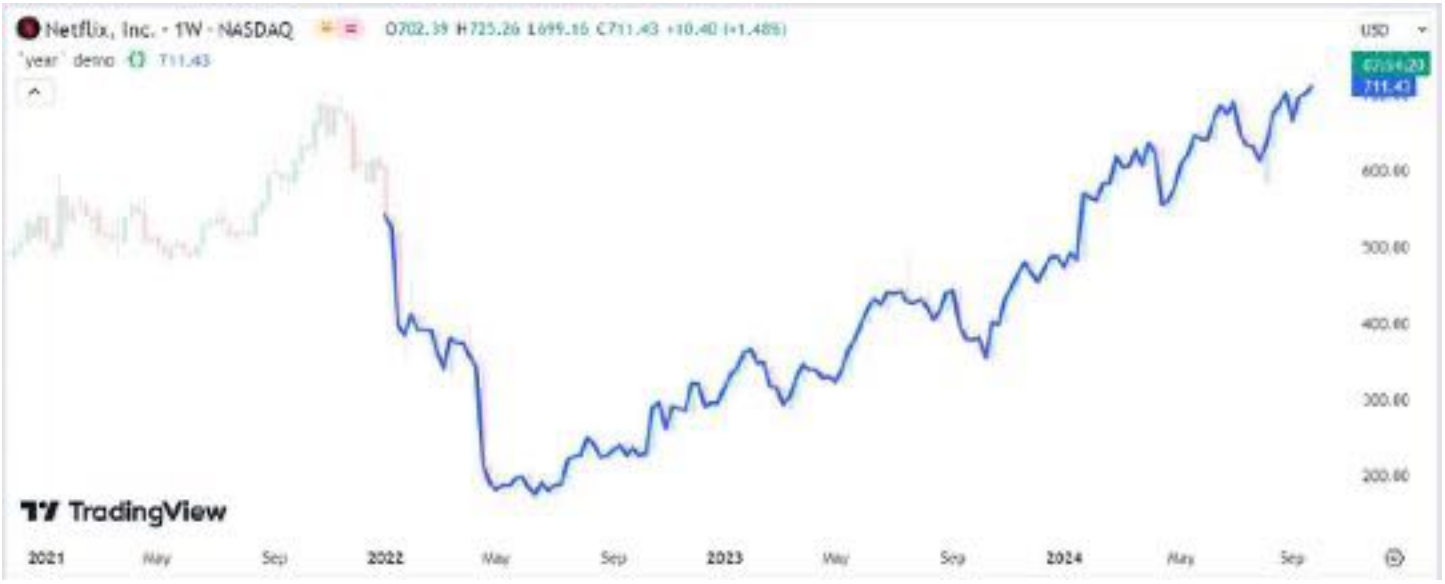


Figure 304: image

contain the first day of a month if the market is *closed* on that day. Therefore, we must create extra conditions that work in these scenarios to identify the first day in *any* month on the chart.

The script below uses the dayofmonth and month variables, and the month() function, to create three conditions that detect the first day of the month in different ways. The first condition detects if the bar opens on the first day, the second checks if the bar opens in one month and closes in another, and the third checks if the chart skips the date entirely. The script draws labels showing bar opening dates and highlights the background with different colors to visualize when each condition occurs:

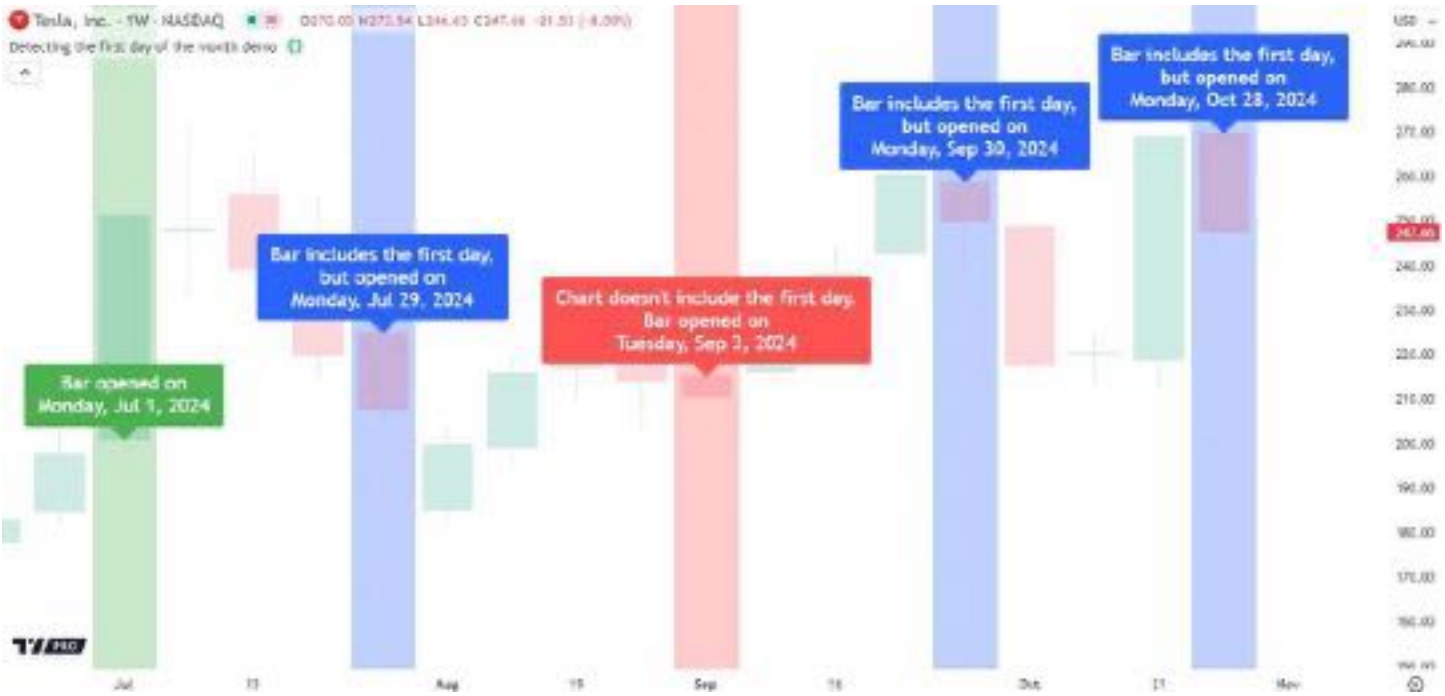


Figure 305: image

```
//@version=6
indicator("Detecting the first day of the month demo", overlay = true, max_labels_count = 500)

//@variable Is `true` only if the current bar opens on the first day of the month in exchange time, `false` ot
bool opensOnFirst = dayofmonth == 1
//@variable Is `true` if the bar opens in one month and closes in another, meaning its time span includes the
```

```
bool containsFirst = month != month(time_close)
//@variable Is `true` only if the bar opens in a new month and the current or previous bar does not cover the
bool skipsFirst = month != month[1] and not (opensOnFirst or containsFirst[1])

//@variable The name of the current bar's opening weekday.
string weekdayName = switch dayofweek
    dayofweek.sunday    => "Sunday"
    dayofweek.monday    => "Monday"
    dayofweek.tuesday   => "Tuesday"
    dayofweek.wednesday => "Wednesday"
    dayofweek.thursday  => "Thursday"
    dayofweek.friday    => "Friday"
    dayofweek.saturday  => "Saturday"

//@variable A custom "string" representing the bar's opening date, including the weekday name.
string openDateText = weekdayName + str.format_time(time, ", MMM d, yyyy")

// Draw a green label when the bar opens on the first day of the month.
if opensOnFirst
    string labelText = "Bar opened on\n" + openDateText
    label.new(time, open, labelText, xloc.bar_time, color = color.green, textcolor = color.white, size = size.
// Draw a blue label when the bar opens and closes in different months.
if containsFirst
    string labelText = "Bar includes the first day,\nbut opened on\n" + openDateText
    label.new(time, open, labelText, xloc.bar_time, color = color.blue, textcolor = color.white, size = size.l
// Draw a red label when the chart skips the first day of the month.
if skipsFirst
    string labelText = "Chart doesn't include the first day.\nBar opened on\n" + openDateText
    label.new(time, open, labelText, xloc.bar_time, color = color.red, textcolor = color.white, size = size.la

// Highlight the background when the conditions occur.
bgcolor(opensOnFirst ? color.new(color.green, 70) : na, title = "`opensOnFirst` condition highlight")
bgcolor(containsFirst ? color.new(color.blue, 70) : na, title = "`containsFirst` condition highlight")
bgcolor(skipsFirst ? color.new(color.red, 70) : na, title = "`skipsFirst` condition highlight")
```

Note that:

- The script calls the month() function with time_close as the `time` argument to calculate each bar's *closing* month for the `containsFirst` condition.
- The `dayofweek.*` namespace contains variables that hold each possible dayofweek value, e.g., dayofweek.sunday has a constant value of 1 and dayofweek.saturday has a constant value of 7. The script compares dayofweek to these variables in a switch structure to determine the weekday name shown inside each label.
- To detect the *first opening time* in a monthly timeframe, not strictly the first day in a calendar month, use `ta.change(time("1M")) > 0` or `timeframe.change("1M")` instead of conditions based on these variables. See the Testing for changes in higher timeframes section to learn more.

last_bar_time

The last_bar_time variable holds a UNIX timestamp representing the *last* available bar's opening time. It is similar to last_bar_index, which references the latest bar index. On historical bars, last_bar_time consistently references the time value of the last bar available when the script first *loads* on the chart. The only time the variable's value updates across script executions is when a new realtime bar opens.

The following script uses the last_bar_time variable to get the opening timestamp of the last chart bar during its execution on the first bar. It displays the UNIX timestamp and a formatted date and time in a single-cell table created only on that bar. When the script executes on the last available bar, it draws a label showing the bar's time value and its formatted representation for visual comparison.

As the chart below shows, both drawings display *identical* times, verifying that last_bar_time correctly references the last bar's time value on previous historical bars:

```
//@version=6
indicator("`last_bar_time` demo", overlay = true)
```

Figure 306: image

```
if barstate.isfirst
    //@variable A single-cell `table`, created only on the *first* bar, showing the *last* available bar's ope
    table displayTable = table.new(position.bottom_right, 1, 1, color.aqua)
    //@variable A "string" containing the `last_bar_time` UNIX timestamp and a custom date and time representa
    string lastBarTimeText = "`last_bar_time`: "              + str.tostring(last_bar_time)
                           + "\nDate and time (exchange): " + str.format_time(last_bar_time, "dd/MM/yy HH:mm
    // Initialize the `displayTable` cell with the `lastBarTimeText`.
    displayTable.cell(
         0, 0, lastBarTimeText,
         text_color = color.white, text_size = size.large, text_halign = text.align_left
     )

//@variable Is `true` only on the first occurrence of `barstate.islast`, `false` otherwise.
//          This condition occurs on the bar whose `time` the `last_bar_time` variable refers to on historical
bool isInitialLastBar = barstate.islast and not barstate.islast[1]

if isInitialLastBar
    //@variable A "string" containing the last available bar's `time` value and a custom date and time represe
    //          Matches the `lastBarTimeText` from the first bar because `last_bar_time` equals this bar's `ti
    string openTimeText = "`time`: "                       + str.tostring(time)
                        + "\nDate and time (exchange): " + str.format_time(time, "dd/MM/yy HH:mm:ss")
    // Draw a label anchored to the bar's `time` to display the `openTimeText`.
    label.new(
         time, high, openTimeText, xloc.bar_time,
         color = color.purple, textcolor = color.white, size = size.large, textalign = text.align_left
     )

// Highlight the background when `isInitialLastBar` is `true` for visual reference.
bgcolor(barstate.islast ? color.rgb(155, 39, 176, 80) : na, title = "Initial last bar highlight")
```

Note that:

- The script creates the label only on the *first* bar with the barstate.islast state because that bar's time value is what last_bar_time equals on all historical bars. On subsequent bars, the last_bar_time value *updates* to represent the latest realtime bar's opening time.
- Updates to last_bar_time on realtime bars do not affect the values on historical bars as the script executes. However, the variable's series *repaints* when the script restarts because last_bar_time always references the latest available bar's opening time.

- This script expresses dates using the `"dd/MM/yy"` format, meaning the two-digit day appears before the two-digit month, and the month appears before the two-digit representation of the year. See this section below for more information.

Visible bar times

The chart.left_visible_bar_time and chart.right_visible_bar_time variables reference the opening UNIX timestamps of the chart's leftmost (first) and rightmost (last) *visible bars* on every script execution. When a script uses these variables, it responds dynamically to visible chart changes, such as users scrolling across bars or zooming in/out. Each time the visible window changes, the script *re-executes* automatically to update the variables' values on all available bars.

The example below demonstrates how the chart.left_visible_bar_time and chart.right_visible_bar_time variables work across script executions. The script draws labels anchored to the visible bars' times to display the UNIX timestamps. In addition, it draws two single-cell tables showing corresponding dates and times in the standard ISO 8601 format. The script creates these drawings only when it executes on the first bar. As the script continues to execute on subsequent bars, it identifies each bar whose time value equals either visible bars' timestamp and colors it on the chart:

Figure 307: image

```
//@version=6
indicator("Visible bar times demo", overlay = true)

// Create strings on the first chart bar that contain the first and last visible bars' UNIX timestamps.
var string leftTimestampText  = str.format("UNIX timestamp: {0,number,#}", chart.left_visible_bar_time)
var string rightTimestampText = str.format("UNIX timestamp: {0,number,#}", chart.right_visible_bar_time)
// Create strings on the first bar that contain the exchange date and time of the visible bars in ISO 8601 for
var string leftDateTimeText  = "Date and time: " + str.format_time(chart.left_visible_bar_time)
var string rightDateTimeText = "Date and time: " + str.format_time(chart.right_visible_bar_time)

//@variable A `label` object anchored to the first visible bar's time. Shows the `leftTimestampText`.
var label leftTimeLabel = label.new(
     chart.left_visible_bar_time, 0, leftTimestampText, xloc.bar_time, yloc.abovebar, color.purple,
     label.style_label_lower_left, color.white, size.large
 )
//@variable A `label` object anchored to the last visible bar's time. Shows the `rightTimestampText`.
var label rightTimeLabel = label.new(
     chart.right_visible_bar_time, 0, rightTimestampText, xloc.bar_time, yloc.abovebar, color.teal,
     label.style_label_lower_right, color.white, size.large
 )
```

```
//@variable A single-cell `table` object showing the `leftDateTimeText`.
var table leftTimeTable = table.new(position.middle_left, 1, 1, color.purple)
//@variable A single-cell `table` object showing the `rightDateTimeText`.
var table rightTimeTable = table.new(position.middle_right, 1, 1, color.teal)
// On the first bar, initialize the `leftTimeTable` and `rightTimeTable` with the corresponding date-time text
if barstate.isfirst
    leftTimeTable.cell(0, 0, leftDateTimeText, text_color = color.white, text_size = size.large)
    rightTimeTable.cell(0, 0, rightDateTimeText, text_color = color.white, text_size = size.large)

//@variable Is purple at the left visible bar's opening time, teal at the right bar's opening time, `na` other
color barColor = switch time
    chart.left_visible_bar_time  => color.purple
    chart.right_visible_bar_time => color.teal

// Color the leftmost and rightmost visible bars using the `barColor`.
barcolor(barColor, title = "Leftmost and rightmost visible bar color")
```

Note that:

- The chart.left_visible_bar_time and chart.right_visible_bar_time values are consistent across all executions, which allows the script to identify the visible bars' timestamps on the *first* bar and check when the time value equals them. The script restarts on any chart window changes, updating the variables' series to reference the new timestamps on every bar.
- The str.format_time() function uses ISO 8601 format by default when the call does not include a `format` argument because it is the *international standard* for expressing dates and times. See the Formatting dates and times section to learn more about time string formats.

`syminfo.timezone`

The syminfo.timezone variable holds a time zone string representing the current symbol's *exchange* time zone. The "string" value expresses the time zone as an *IANA identifier* (e.g., `"America/New_York"`). The overloads of time functions that include a `timezone` parameter use syminfo.timezone as the default argument.

Because this variable is the default `timezone` argument for all applicable time function overloads, it is unnecessary to use as an explicit argument, except for stylistic purposes. However, programmers can use the variable in other ways, such as:

- Displaying the "string" in Pine Logs or drawings to inspect the exchange time zone's IANA identifier.
- Comparing the value to other time zone strings to create time zone-based conditional logic.
- Requesting the exchange time zones of other symbols with `request.*()` function calls.

The following script uses the timenow variable to retrieve the UNIX timestamp of its latest execution. It formats the timestamp into date-time strings expressed in the main symbol's exchange time zone and a requested symbol's exchange time zone, which it displays along with the IANA identifiers in a table on the last chart bar:

```
//@version=6
indicator("`syminfo.timezone` demo", overlay = true)

//@variable The symbol to request exchange time zone information for.
string symbolInput = input.symbol("NSE:BANKNIFTY", "Requested symbol")

//@variable A `table` object displaying the exchange time zone and the time of the script's execution.
var table t = table.new(position.bottom_right, 2, 3, color.yellow, border_color = color.black, border_width =

//@variable The IANA identifier of the exchange time zone requested for the `symbolInput` symbol.
var string requestedTimezone = na

if barstate.islastconfirmedhistory
    // Retrieve the time zone of the user-specified symbol's exchange.
    requestedTimezone := request.security(symbolInput, "", syminfo.timezone, calc_bars_count = 2)
    // Initialize the `t` table's header cells.
    t.cell(0, 0, "Exchange prefix and time zone string", text_size = size.large)
    t.cell(1, 0, "Last execution date and time", text_size = size.large)
```

Figure 308: image

```
    t.cell(0, 1, syminfo.prefix(syminfo.tickerid) + " (" + syminfo.timezone + ")", text_size = size.large)
    t.cell(0, 2, syminfo.prefix(symbolInput) + " (" + requestedTimezone + ")", text_size = size.large)

if barstate.islast
    //@variable The formatting string for all `str.format_time()` calls.
    var string formatString = "HH:mm:ss 'on' MMM dd, YYYY"
    // Initialize table cells to display the formatted text.
    t.cell(1, 1, str.format_time(timenow, formatString), text_size = size.large)
    t.cell(1, 2, str.format_time(timenow, formatString, requestedTimezone), text_size = size.large)
```

Note that:

- Pine scripts execute on realtime bars only when new updates occur in the data feed, and timenow updates only on script executions. As such, when no realtime updates are available, the timenow timestamp does not change. See this section above for more information.
- The default `symbolInput` value is `"NSE:BANKNIFTY"`. NSE is in the "Asia/Kolkata" time zone, which is 9.5 hours ahead of the main symbol's exchange time zone ("America/New_York") at the time of the screenshot. Although the *local* time representations differ, both refer to the same *absolute* time that the timenow timestamp represents.
- Pine v6 scripts use dynamic `request.*()` calls by default, which allows the script to call request.security() dynamically inside the if structure's *local scope*. See the Dynamic requests section of the Other timeframes and data page to learn more.

Time functions

Pine Script™ features several built-in functions that scripts can use to retrieve, calculate, and express time values:

- The time() and time_close() functions allow scripts to retrieve UNIX timestamps for the opening and closing times of bars within a session on a specified timeframe, without requiring `request.*()` function calls.
- The year(), month(), weekofyear(), dayofmonth(), dayofweek(), hour(), minute(), and second() functions calculate calendar-based values, expressed in a specified time zone, from a UNIX timestamp.
- The timestamp() function calculates a UNIX timestamp from a specified calendar date and time.
- The str.format_time() function formats a UNIX timestamp into a human-readable date/time "string", expressed in a specified time zone. The Formatting dates and times section below provides detailed information about formatting timestamps with this function.
- The input.time() function returns a UNIX timestamp corresponding to the user-specified date and time, and the input.session() function returns a valid session string corresponding to the user-specified start and end times. See the Time input and Session input sections of the Inputs page to learn more about these functions.

`time()` and `time_close()` functions

The time() and time_close() functions return UNIX timestamps representing the opening and closing times of bars on a specified timeframe. Both functions can *filter* their returned values based on a given session in a specific time zone. They each have the following signatures:

```
functionName(timeframe, bars_back) → series intfunctionName(timeframe, session, bars_back) → series intfunctio
```

Where:

- `functionName` is the function's identifier.
- The `timeframe` parameter accepts a timeframe string. The function uses the script's main timeframe if the argument is timeframe.period or an empty "string".
- The `session` parameter accepts a session string defining the session's start and end times (e.g., `"0930-1600"`) and the days for which it applies (e.g., `":23456"` means Monday - Friday). If the value does not specify the days, the session applies to *all* weekdays automatically. The function returns UNIX timestamps only for the bars *within* the session. It returns na if a bar's time is *outside* the session. If the `session` argument is an empty "string" or not specified, The function uses the symbol's session information.
- The `timezone` parameter accepts a valid time zone string that defines the time zone of the specified `session`. It does **not** change the meaning of returned UNIX timestamps, as they are *time zone-agnostic*. If the `timezone` argument is not specified, the function uses the exchange time zone (syminfo.timezone).
- The `bars_back` parameter accepts an "int" value specifying which bar's time the returned timestamp represents. If the value is positive, the function returns the timestamp from that number of bars back relative to the current bar. If the value is negative and greater than or equal to -500, the function returns the *expected* timestamp of a future bar. The default value is 0.

Similar to the time and time_close variables, these functions behave differently on *time-based* and *non-time-based* charts.

Time-based charts have bars that open and close at *predictable* times, whereas the bars on tick charts and all non-standard charts, excluding Heikin Ashi, open and close at irregular, *unpredictable* times. Consequently, time_close() cannot calculate the expected closing times of realtime bars on non-time-based charts, so it returns na on those bars. Similarly, the time() function with a negative `bars_back` value cannot accurately calculate the expected opening time of a future realtime bar on these charts. See the *second example* in this section above. That example script exhibits the same behavior on a price-based chart if it uses a time_close("") call instead of the time_close variable.

Typical use cases for the time() and time_close() functions include:

- Testing for bars that open or close in specific sessions defined by the `session` and `timezone` parameters.
- Testing for changes or measuring time differences on specified higher timeframes.

Testing for sessions The time() and time_close() functions' `session` and `timezone` parameters define the sessions for which they can return *non-na* values. If a call to either function references a bar that opens/closes within the defined session in a given time zone, it returns a UNIX timestamp for that bar. Otherwise, it returns na. Programmers can pass the returned values to the na() function to identify which bars open or close within specified intervals, which is helpful for session-based calculations and logic.

This simple script identifies when a bar on the chart's timeframe opens at or after 11:00 and before 13:00 in the exchange time zone on any trading day. It calls time() with timeframe.period as the `timeframe` argument and the `"1100-1300"`session string as the `session` argument, and then verifies whether the returned value is na with the na() function. When the value is **not**na, the script highlights the chart's background to indicate that the bar opened in the session:

```
//@version=6
indicator("Testing for session bars demo", overlay = true)

//@variable Checks if the bar opens between 11:00 and 13:00. Is `true` if the `time()` function does not retur
bool inSession = not na(time(timeframe.period, "1100-1300"))

// Highlight the background of the bars that are in the session "11:00-13:00".
bgcolor(inSession ? color.rgb(155, 39, 176, 80) : na)
```

Note that:

- The `session` argument in the time() call represents an interval in the *exchange* time zone because syminfo.timezone is the default `timezone` argument.
- The session string expresses the start and end times in the `"HHmm-HHmm"` format, where `"HH"` is the two-digit *hour* and `"mm"` is the two-digit *minute*. Session strings can also specify the *weekdays* a session applies to. However, the time()

Figure 309: image

call's `session` argument (`"1100-1300"`) does not include this information, which is why it considers the session valid for *every* day. See the Sessions page to learn more.

When using session strings in time() and time_close() calls, it's crucial to understand that such strings define start and end times in a *specific*time zone. The local hour and minute in one region may not correspond to the same point in UNIX time as that same hour and minute in another region. Therefore, calls to these functions with different `timezone` arguments can return non-na timestamps at *different* times, as the specified time zone string changes the meaning of the local times represented in the `session` argument.

This example demonstrates how the `timezone` parameter affects the `session` parameter in a time() function call. The script calculates an `opensInSession` condition that uses a time() call with arguments based on inputs. The session input for the `session` argument includes four preset options: `"0000-0400"`, `"0930-1400"`, `"1300-1700"`, and `"1700-2100"`. The string input that defines the `timezone` argument includes four IANA time zone options representing different offsets from UTC: `"America/Vancouver"` (UTC-7/-8), `"America/New_York"` (UTC-4/-5), `"Asia/Dubai"` (UTC+4), and `"Austrailia/Sydney"` (UTC+10/+11).

For any chosen `sessionInput` value, changing the `timezoneInput` value changes the specified session's time zone. The script highlights *different bars* with each time zone choice because, unlike UNIX timestamps, the *absolute* times that local hour and minute values correspond to *varies* across time zones:

Figure 310: image

```
//@version=6
indicator("Testing session time zones demo", overlay = true)
```

```
//@variable The timeframe of the analyzed bars.
string timeframeInput = input.timeframe("60", "Timeframe")
//@variable The session to check. Features four interval options.
string sessionInput = input.session("1300-1700", "Session", ["0000-0400", "0930-1400", "1300-1700", "1700-2100
//@variable An IANA identifier representing the time zone of the `sessionInput`. Fetures four preset options.
string timezoneInput = input.string("America/New_York", "Time zone",
     ["America/Vancouver", "America/New_York", "Asia/Dubai", "Australia/Sydney"]
 )

//@variable Is `true` if a `timeframeInput` bar opens within the `sessionInput` session in the `timezoneInput`
//          The condition detects the session on different bars, depending on the chosesn time zone, because i
//          local times in different time zones refer to different absolute points in UNIX time.
bool opensInSession = not na(time(timeframeInput, sessionInput, timezoneInput))

// Highlight the background when `opensInSession` is `true`.
bgcolor(opensInSession ? color.rgb(33, 149, 243, 80) : na, title = "Open in session highlight")
```

Note that:

- This script uses *IANA notation* for all time zone strings because it is the recommended format. Using an IANA identifier allows the time() call to automatically adjust the session's UTC offset based on a region's local time policies, such as daylight saving time.

Testing for changes in higher timeframes The `timeframe` parameter of the time() and time_close() functions specifies the timeframe of the bars in the calculation, allowing scripts to retrieve opening/closing UNIX timestamps from *higher timeframes* than the current chart's timeframe without requiring `request.*()` function calls.

Programmers can use the opening/closing timestamps from higher-timeframe (HTF) bars to detect timeframe changes. One common approach is to call time() or time_close() with a consistent `timeframe` argument across all executions on a time-based chart and measure the one-bar change in the returned value with the ta.change() function. The result is a *nonzero* value only when an HTF bar opens. One can also check whether the data has a time gap at that point by comparing the time() value to the previous bar's time_close() value. A gap is present when the opening timestamp on the current bar is greater than the closing timestamp on the previous bar.

The script below calls time("1M") to get the opening UNIX timestamp of the current bar on the "1M" timeframe. It detects when bars on that timeframe open by checking when the ta.change() of the timestamp returns a value greater than 0. On each occurrence of the condition, the script detects whether the HTF bar opened after a gap by checking if the opening time is greater than the previous bar's time_close("1M") value.

The script draws labels containing formatted "1M" opening times to indicate the chart bars that mark the start of monthly bars. If a monthly bar opens without a gap from the previous closing time, the script draws a blue label. If a monthly bar starts after a gap, it draws a red label. Additionally, if the "1M" opening time does not match the opening time of the chart bar, the script displays that bar's formatted time in the label for comparison:

```
//@version=6
indicator("Detecting changes in higher timeframes demo", overlay = true)

//@variable The opening UNIX timestamp of the current bar on the "1M" timeframe.
int currMonthlyOpenTime = time("1M")
//@variable The closing timestamp on the "1M" timeframe as of the previous chart bar.
int prevMonthlyCloseTime = time_close("1M")[1]

//@variable Is `true` when the opening time on the "1M" timeframe changes, indicating a new monthly bar.
bool isNewTf = ta.change(currMonthlyOpenTime) > 0

if isNewTf
    // Initialize variables for the `text` and `color` properties of the label drawing.
    string lblText  = "New '1M' opening time:\n" + str.format_time(currMonthlyOpenTime)
    color  lblColor = color.blue

    //@variable Is `true` when the `currMonthlyOpenTime` exceeds the `prevMonthlyCloseTime`, indicating a time
    bool hasGap = currMonthlyOpenTime > prevMonthlyCloseTime
```

Figure 311: image

```
// Modify the `lblText` and `lblColor` based on the `hasGap` value.
if hasGap
    lblText := "Gap from previous '1M' close.\n\n" + lblText
    lblColor := color.red
// Include the formatted `time` value if the `currMonthlyOpenTime` is before the first available chart bar
if time > currMonthlyOpenTime
    lblText += "\nFirst chart bar has a later time:\n" + str.format_time(time)

// Draw a `lblColor` label anchored to the `time` to display the `lblText`.
label.new(
     time, high, lblText, xloc.bar_time, color = lblColor, style = label.style_label_lower_right,
     textcolor = color.white, size = size.large
 )
```

Note that:

- Using ta.change() on a time() or time_close() call's result is not the *only* way to detect changes in a higher timeframe. The timeframe.change() function is an equivalent, more convenient option for scripts that do not need to *use* the UNIX timestamps from HTF bars in other calculations, as it returns a "bool" value directly without extra code.
- The detected monthly opening times do not always correspond to the first calendar day of the month. Instead, they correspond to the first time assigned to a "1M" bar, which can be *after* the first calendar day. For symbols with overnight sessions, such as "USDJPY" in our example chart, a "1M" bar can also open *before* the first calendar day.
- Sometimes, the opening time assigned to an HTF bar might *not* equal the opening time of any chart bar, which is why other conditions such as `time == time("1M")` cannot detect new monthly bars consistently. For example, on our "USDJPY" chart, the "1M" opening time `2023-12-31T17:00:00-0500` does not match an opening time on the "1D" timeframe. The first available "1D" bar after that point opened at `2024-01-01T17:00:00-0500`.

Calendar-based functions

The year(), month(), weekofyear(), dayofmonth(), dayofweek(), hour(), minute(), and second() functions calculate *calendar-based* "int" values from a UNIX timestamp. Unlike the calendar-based variables, which always hold exchange calendar values based on the current bar's opening timestamp, these functions can return calendar values for any valid timestamp and express them in a chosen time zone.

Each of these calendar-based functions has the following two signatures:

`functionName(time) → series intfunctionName(time, timezone) → series int`

Where:

- `functionName` is the function's identifier.
- The `time` parameter accepts an "int" UNIX timestamp for which the function calculates a corresponding calendar value.
- The `timezone` parameter accepts a time zone string specifying the returned value's time zone. If the `timezone` argument is not specified, the function uses the exchange time zone (syminfo.timezone).

In contrast to the functions that return UNIX timestamps, a calendar-based function returns different "int" results for various time zones, as calendar values represent parts of a *local time* in a *specific region*.

For instance, the simple script below uses two calls to dayofmonth() to calculate each bar's opening day in the exchange time zone and the "Australia/Sydney" time zone. It plots the results of the two calls in a separate pane for comparison:

Figure 312: image

```
//@version=6
indicator("`dayofmonth()` demo", overlay = false)

//@variable An "int" representing the current bar's opening calendar day in the exchange time zone.
//          Equivalent to the `dayofmonth` variable.
int openingDay = dayofmonth(time)

//@variable An "int" representing the current bar's opening calendar day in the "Australia/Sydney" time zone.
int openingDaySydney = dayofmonth(time, "Australia/Sydney")

// Plot the calendar day values.
plot(openingDay,       "Day of Month (Exchange)", linewidth = 6, color = color.blue)
plot(openingDaySydney, "Day of Month (Sydney)",   linewidth = 3, color = color.orange)
```

Note that:

- The first dayofmonth() call calculates the bar's opening day in the exchange time zone because it does not include a `timezone` argument. This call returns the same value that the dayofmonth variable references.
- Our example symbol's exchange time zone is "America/New_York", which follows UTC-5 during standard time and UTC-4 during daylight saving time (DST). The "Australia/Sydney" time zone follows UTC+10 during standard time and UTC+11 during DST. However, Sydney observes DST at *different* times of the year than New York. As such, its time zone is 14, 15, or 16 hours ahead of the exchange time zone, depending on the time of year. The plots on our "1D" chart diverge when the difference is at least 15 hours because the bars open at 09:30 in exchange time, and 15 hours ahead is 00:30 on the *next* calendar day.

It's important to understand that although the `time` argument in a calendar-based function call represents a single, absolute point in time, each function returns only *part* of the date and time information available from the timestamp. Consequently,

a calendar-based function's returned value does **not** directly correspond to a *unique* time point, and conditions based on individual calendar values can apply to *multiple* bars.

For example, this script uses the timestamp() function to calculate a UNIX timestamp from a date "string", and it calculates the calendar day from that timestamp, in the exchange time zone, with the dayofmonth() function. The script compares each bar's opening day to the calculated day and highlights the background when the two are equal:

Figure 313: image

```
//@version=6
indicator("`dayofmonth()` demo", overlay = false)

//@variable The UNIX timestamp corresponding to August 29, 2024 at 00:00:00 UTC.
const int fixedTimestamp = timestamp("29 Aug 2024")
//@variable The day of the month calculated from the `fixedTimestamp`, expressed in the exchange time zone.
//          If the exchange time zone has a negative UTC offset, this variable's value is 28 instead of 29.
int dayFromTimestamp = dayofmonth(fixedTimestamp)

//@variable An "int" representing the current bar's opening calendar day in the exchange time zone.
//          Equivalent to the `dayofmonth` variable.
int openingDay = dayofmonth(time)

// Plot the `openingDay`.
plot(openingDay, "Opening day of month", linewidth = 3)
// Highlight the background when the `openingDay` equals the `dayFromTimestamp`.
bgcolor(openingDay == dayFromTimestamp ? color.orange : na, title = "Day detected highlight")
```

Note that:

- The timestamp() call treats its argument as a *UTC* calendar date because its `dateString` argument does not specify time zone information. However, the dayofmonth() call calculates the day in the *exchange time zone*. Our example symbol's exchange time zone is "America/New_York" (UTC-4/-5). Therefore, the returned value on this chart is 28 instead of 29.
- The script highlights *any* bar on our chart that opens on the 28th day of *any* month instead of only a specific bar because the dayofmonth() function's returned value does **not** represent a specific point in time on its own.
- This script highlights the bars that *open* on the day of the month calculated from the timestamp. However, some months on our chart have no trading activity on that day. For example, the script does not highlight when the July 28, 2024 occurs on our chart because NASDAQ is closed on Sundays.

Similar to calendar-based variables, these functions are also helpful when testing for dates/times and detecting calendar changes on the chart. The example below uses the year(), month(), weekofyear(), and dayofweek() functions on the time_close timestamp to create conditions that test if the current bar is the first bar that closes in a new year, quarter, month, week, and day. The script uses plotted shapes, labels, and background colors to visualize the conditions on the chart:

```
//@version=6
indicator("Calendar changes demo", overlay = true, max_labels_count = 500)
```

Figure 314: image

```
// Calculate the year, month, week of year, and day of week corresponding to the `time_close` UNIX timestamp.
// All values are expressed in the exchange time zone.
int closeYear       = year(time_close)
int closeMonth      = month(time_close)
int closeWeekOfYear = weekofyear(time_close)
int closeDayOfWeek  = dayofweek(time_close)

//@variable Is `true` when the change in `closeYear` exceeds 0, marking the first bar that closes in a new yea
bool closeInNewYear = ta.change(closeYear) > 0
//@variable Is `true` when the difference in `closeMonth` is not 0, marking the first bar that closes in a new
bool closeInNewMonth = closeMonth - closeMonth[1] != 0
//@variable Is `true` when `closeMonth - 1` becomes divisible by 3, marking the first bar that closes in a new
bool closeInNewQuarter = (closeMonth[1] - 1) % 3 != 0 and (closeMonth - 1) % 3 == 0
//@variable Is `true` when the change in `closeWeekOfYear` is not 0, marking the first bar that closes in a ne
bool closeInNewWeek = ta.change(closeWeekOfYear) != 0
//@variable Is `true` when the `closeDayOfWeek` changes, marking the first bar that closes in the new day.
bool closeInNewDay = closeDayOfWeek != closeDayOfWeek[1]

//@variable Switches between `true` and `false` after every `closeInNewDay` occurrence for background color ca
var bool alternateDay = true
if closeInNewDay
    alternateDay := not alternateDay

// Draw a label above the bar to display the `closeWeekOfYear` when `closeInNewWeek` is `true`.
if closeInNewWeek
    label.new(
         time, 0, "W" + str.tostring(closeWeekOfYear), xloc.bar_time, yloc.abovebar, color.purple,
         textcolor = color.white, size = size.normal
     )
// Plot label shapes at the bottom and top of the chart for the `closeInNewYear` and `closeInNewMonth` conditi
plotshape(
     closeInNewYear, "Close in new year", shape.labelup, location.bottom, color.teal, text = "New year",
     textcolor = color.white, size = size.huge
 )
```

```
plotshape(
     closeInNewMonth, "Close in new month", shape.labeldown, location.top, text = "New month",
     textcolor = color.white, size = size.large
 )
// Plot a triangle below the chart bar when `closeInNewQuarter` occurs.
plotshape(
     closeInNewQuarter, "Close in new quarter", shape.triangleup, location.belowbar, color.maroon,
     text = "New quarter", textcolor = color.maroon, size = size.large
 )
// Highlight the background in alternating colors based on occurrences of `closeInNewDay`.
bgcolor(alternateDay ? color.new(color.aqua, 80) : color.new(color.fuchsia, 80), title = "Closing day change")
```

Note that:

- This script's conditions check for the first bar that closes after each calendar unit changes its value. The bar where each condition is `true` varies with the data available on the chart. For example, the `closeInNewMonth` condition can be `true`*after* the first calendar day of the month if a chart bar did not close on that day.
- To detect when new bars start on a specific *timeframe* rather than strictly calendar changes, check when the ta.change() of a time() or time_close() call's returned value is nonzero, or use the timeframe.change() function. See this section above for more information.

`timestamp()`

The timestamp() function calculates a UNIX timestamp from a specified calendar date and time. It has the following three signatures:

```
timestamp(year, month, day, hour, minute, second) → simple/series inttimestamp(timezone, year, month, day, hou
```

The first two signatures listed include `year`, `month`, `day`, `hour`, `minute`, and `second` parameters that accept "int" values defining the calendar date and time. A timestamp() call with either signature must include `year`, `month`, and `day` arguments. The other parameters are optional, each with a default value of 0. Both signatures can return either *"simple"* or *"series"* values, depending on the qualified types of the specified arguments.

The primary difference between the first two signatures is the `timezone` parameter, which accepts a time zone string that determines the time zone of the *date and time* specified by the other parameters. If a timestamp() call with "int" calendar arguments does not include a `timezone` argument, it uses the exchange time zone (syminfo.timezone) by default.

The third signature listed has only *one* parameter, `dateString`, which accepts a "string" representing a valid calendar date (e.g., `"20 Aug 2024"`). The value can also include the time of day and time zone (e.g., `"20 Aug 2024 00:00:00 UTC+0"`). If the `dateString` argument does not specify the time of day, the timestamp() call considers the time 00:00 (midnight).

Unlike the other two signatures, the default time zone for the third signature is **GMT+0**. It does **not** use the exchange time zone by default because it interprets time zone information from the `dateString` directly. Additionally, the third signature is the only one that returns a *"const int"* value. As shown in the Time input section of the Inputs page, programmers can use this overload's returned value as the `defval` argument in an input.time() function call.

When using the timestamp() function, it's crucial to understand how time zone information affects its calculations. The *absolute* point in time represented by a specific calendar date *depends* on its time zone, as an identical date and time in various time zones can refer to **different** amounts of time elapsed since the *UNIX Epoch*. Therefore, changing the time zone of the calendar date and time in a timestamp() call *can change* its returned UNIX timestamp.

The following script compares the results of four different timestamp() calls that evaluate the date 2021-01-01 in different time zones. The first timestamp() call does not specify time zone information in its `dateString` argument, so it treats the value as a *UTC* calendar date. The fourth call also evaluates the calendar date in UTC because it includes `"UTC0"` as the `timezone` argument. The second timestamp() call uses the first signature listed above, meaning it uses the exchange time zone, and the third call uses the second signature with `"America/New_York"` as the `timezone` argument.

The script draws a table with rows displaying each timestamp() call, its assigned variable, the calculated UNIX timestamp, and a formatted representation of the time. As we see on the "NASDAQ:MSFT" chart below, the first and fourth table rows show *different* timestamps than the second and third, leading to different formatted strings in the last column:

```
//@version=6
indicator("`timestamp()` demo", overlay = false)

//@variable A `table` that displays the different `timestamp()` calls, their returned timestamps, and formatte
var table displayTable = table.new(
```

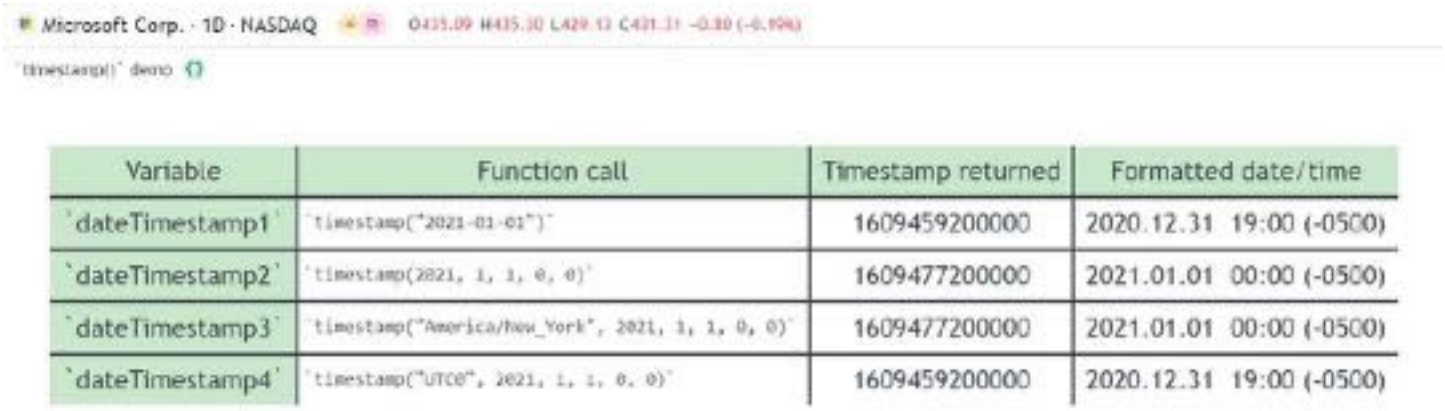


Figure 315: image

```
    position.middle_center, 4, 5, color.white, border_color = color.black, border_width = 2
)

//@function Initializes a `displayTable` cell showing the `displayText` with an optional `specialFormat`.
printCell(int colID, int rowID, string displayText, string specialFormat = "") =>
    displayTable.cell(colID, rowID, displayText, text_size = size.large)
    switch specialFormat
        "header"  => displayTable.cell_set_bgcolor(colID, rowID, color.rgb(76, 175, 79, 70))
        "code"    =>
            displayTable.cell_set_text_font_family(colID, rowID, font.family_monospace)
            displayTable.cell_set_text_size(colID, rowID, size.normal)
            displayTable.cell_set_text_halign(colID, rowID, text.align_left)

if barstate.islastconfirmedhistory
    //@variable The UNIX timestamp corresponding to January 1, 2021 in the UTC+0 time zone.
    int dateTimestamp1 = timestamp("2021-01-01")
    //@variable The UNIX timestamp corresponding to January 1, 2021 in the exchange time zone.
    int dateTimestamp2 = timestamp(2021, 1, 1, 0, 0)
    //@variable The UNIX timestamp corresponding to January 1, 2021 in the "America/New_York" time zone.
    int dateTimestamp3 = timestamp("America/New_York", 2021, 1, 1, 0, 0)
    //@variable The UNIX timestamp corresponding to January 1, 2021 in the "UTC0" (UTC+0) time zone.
    int dateTimestamp4 = timestamp("UTC0", 2021, 1, 1, 0, 0)

    // Initialize the top header cells in the `displayTable`.
    printCell(0, 0, "Variable",            "header")
    printCell(1, 0, "Function call",       "header")
    printCell(2, 0, "Timestamp returned",  "header")
    printCell(3, 0, "Formatted date/time", "header")
    // Initialize a table row for `dateTimestamp1` results.
    printCell(0, 1, "`dateTimestamp1`", "header")
    printCell(1, 1, "`timestamp(\"2021-01-01\")`", "code")
    printCell(2, 1, str.tostring(dateTimestamp1))
    printCell(3, 1, str.format_time(dateTimestamp1, "yyyy.MM.dd  HH:mm (Z)"))
    // Initialize a table row for `dateTimestamp2` results.
    printCell(0, 2, "`dateTimestamp2`", "header")
    printCell(1, 2, "`timestamp(2021, 1, 1, 0, 0)`", "code")
    printCell(2, 2, str.tostring(dateTimestamp2))
    printCell(3, 2, str.format_time(dateTimestamp2, "yyyy.MM.dd  HH:mm (Z)"))
    // Initialize a table row for `dateTimestamp3` results.
    printCell(0, 3, "`dateTimestamp3`", "header")
    printCell(1, 3, "`timestamp(\"America/New_York\", 2021, 1, 1, 0, 0)`", "code")
```

```
printCell(2, 3, str.tostring(dateTimestamp3))
printCell(3, 3, str.format_time(dateTimestamp3, "yyyy.MM.dd  HH:mm (Z)"))
// Initialize a table row for `dateTimestamp4` results.
printCell(0, 4, "`dateTimestamp4`", "header")
printCell(1, 4, "`timestamp(\"UTC0\", 2021, 1, 1, 0, 0)`", "code")
printCell(2, 4, str.tostring(dateTimestamp4))
printCell(3, 4, str.format_time(dateTimestamp4, "yyyy.MM.dd  HH:mm (Z)"))
```

Note that:

- The formatted date-time strings express results in the exchange time zone because the str.format_time() function uses syminfo.timezone as the default `timezone` argument. The formatted values on our example chart show the offset string `"-0500"` because NASDAQ's time zone ("America/New_York") follows UTC-5 during *standard time*.
- The formatted strings on the first and fourth rows show the date and time five hours *before* January 1, 2021, because the timestamp() calls evaluated the date in *UTC* and the str.format_time() calls used a time zone five hours *behind* UTC.
- On our chart, the second and third rows have matching timestamps because both corresponding timestamp() calls evaluated the date in the "America/New_York" time zone. The two rows would show different results if we applied the script to a symbol with a different exchange time zone.

Formatting dates and times

Programmers can format UNIX timestamps into human-readable dates and times, expressed in specific time zones, using the str.format_time() function. The function has the following signature:

`str.format_time(time, format, timezone) → series string`

Where:

- The `time` parameter specifies the "int" UNIX timestamp to express as a readable time.
- The `format` parameter accepts a "string" consisting of *formatting tokens* that determine the returned information. If the function call does not include a `format` argument, it uses the ISO 8601 standard format: `"yyyy-MM-dd'T'HH:mm:ssZ"`. See the table below for a list of valid tokens and the information they represent.
- The `timezone` parameter determines the time zone of the formatted result. It accepts a time zone string in UTC or IANA notation. If the call does not specify a `timezone`, it uses the exchange time zone (syminfo.timezone).

The general-purpose str.format() function can also format UNIX timestamps into readable dates and times. However, the function **cannot** express time information in *different* time zones. It always expresses dates and times in **UTC+0**. In turn, using this function to format timestamps often results in *erroneous* practices, such as mathematically modifying a timestamp to try and represent the time in another time zone. However, a UNIX timestamp is a unique, **time zone-agnostic** representation of a specific point in time. As such, modifying a UNIX timestamp changes the *absolute time* it represents rather than expressing the same time in a different time zone.

The str.format_time() function does not have this limitation, as it can calculate dates and times in *any* time zone correctly without changing the meaning of a UNIX timestamp. In addition, unlike str.format(), it is optimized specifically for processing time values. Therefore, we recommend that programmers use str.format_time() instead of str.format() to format UNIX timestamps into readable dates and times.

A str.format_time() call's `format` argument determines the time information its returned value contains. The function treats characters and sequences in the argument as *formatting tokens*, which act as *placeholders* for values in the returned date/time "string". The following table outlines the most commonly used formatting tokens and explains what each represents:

TokenRepresentsRemarks and examples`"y"`YearUse `"yy"` to include the last two digits of the year (e.g., `"00"`), or `"yyyy"` to include the complete year number (e.g., `"2000"`).`"M"`MonthUppercase `"M"` for the month, not to be confused with *lowercase*`"m"` for the minute.
Use `"MM"` to include the two-digit month number with a leading zero for single-digit values (e.g., `"01"`), `"MMM"` to include the three-letter abbreviation of month (e.g., `"Jan"`), or `"MMMM"` for the full month name (e.g., `"January"`).`"d"`Day of the **month**Lowercase `"d"`.
Includes the numeric day of the month (`"1"` to `"31"`).
Use `"dd"` for the two-digit day number with a leading zero for single-digit values.
It is *not* a placeholder for the day number of the *week* (1-7). Use dayofweek() to calculate that value.`"D"`Day of the **year**Uppercase `"D"`.
Includes the numeric day of the year (`"1"` to `"366"`).
Use `"DD"` or `"DDD"` for the two-digit or three-digit day number with leading zeros.`"E"`Day of the **week**Includes the abbreviation of the weekday *name* (e.g., `"Mon"`).

Use `"EEEE"` for the weekday's full name (e.g., `"Monday"`)`"w"`Week of the **year**Lowercase `"w"`.
Includes the week number of the year (`"1"` to `"53"`).
Use `"ww"` for the two-digit week number with a leading zero for single-digit values.`"W"`Week of the **month**Uppercase `"W"`.
Includes the week number of the month (`"1"` to `"5"`).`"a"`AM/PM postfixLowercase `"a"`.
Includes `"AM"` if the time of day is before noon, `"PM"` otherwise.`"h"`Hour in the **12-hour** formatLowercase `"h"`.
The included hour number from this token ranges from `"0"` to `"11"`.
Use `"hh"` for the two-digit hour with a leading zero for single-digit values.`"H"`Hour in the **24-hour** formatUppercase `"H"`.
The included hour number from this token ranges from `"0"` to `"23"`.
Use `"HH"` for the two-digit hour with a leading zero for single-digit values.`"m"`MinuteLowercase `"m"` for the minute, not to be confused with *uppercase*`"M"` for the month.
Use `"mm"` for the two-digit minute with a leading zero for single-digit values.`"s"`SecondLowercase `"s"` for the second, not to be confused with *uppercase*`"S"` for fractions of a second.
Use `"ss"` for the two-digit second with a leading zero for single-digit values.`"S"`Fractions of a secondUppercase `"S"`.
Includes the number of milliseconds in the fractional second (`"0"` to `"999"`).
Use `"SS"` or `"SSS"` for the two-digit or three-digit millisecond number with leading zeros.`"Z"`Time zone (**UTC offset**)Uppercase `"Z"`.
Includes the hour and minute UTC offset value in `"HHmm"` format, preceded by its sign (e.g., `"-0400"`).`"z"`Time zone (**abbreviation or name**)Lowercase `"z"`.
A single `"z"` includes the abbreviation of the time zone (e.g., `"EDT"`).
Use `"zzzz"` for the time zone's name (e.g., `"Eastern Daylight Time"`).
It is not a placeholder for the *IANA identifier*. Use syminfo.timezone to retrieve the exchange time zone's IANA representation.`":"`, `"/"`, `"-"`, `"."`, `","`, `"("`, `")"`, `" "`SeparatorsThese characters are separators for formatting tokens.
They appear as they are in the formatted text. (e.g., `"01/01/24"`, `"12:30:00"`, `"Jan 1, 2024"`).
Some other characters can also act as separators. However, the ones listed are the most common.`"'"`Escape characterCharacters enclosed within *two single quotes* appear as they are in the result, even if they otherwise act as formatting tokens. For example, `" 'Day' "` appears as-is in the resulting "string" instead of listing the day of the year, AM/PM postfix, and year.The following example demonstrates how various formatting tokens affect the str.format_time() function's result. The script calls the function with different `format` arguments to create date/time strings from time, timenow, and time_close timestamps. It displays each `format` value and the corresponding formatted result in a table on the last bar:

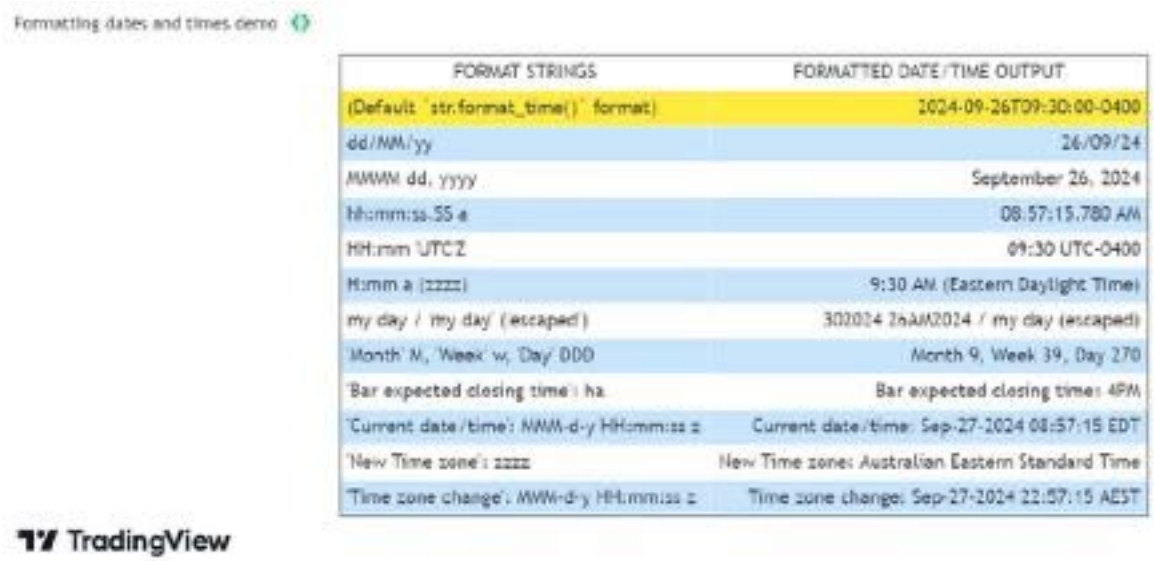


Figure 316: image

```
//@version=6
indicator("Formatting dates and times demo", overlay = false)

//@variable A `table` that displays different date/time `format` strings and their results.
var table displayTable = table.new(
     position.middle_center, 2, 15, bgcolor = color.white,
     frame_color = color.black, frame_width = 1, border_width = 1
 )
```

```
//@function Initializes a `displayTable` row showing a `formatString` and its formatted result for a specified
//          `timeValue` and `timezoneValue`.
displayText(rowID, formatString, timeValue = time, timezoneValue = syminfo.timezone) =>
    //@variable Is light blue if the `rowID` is even, white otherwise. Used to set alternating table row color
    color rowColor = rowID % 2 == 0 ? color.rgb(33, 149, 243, 75) : color.white
    // Display the `formatString` in the row's first cell.
    displayTable.cell(
         0, rowID, formatString,
         text_color = color.black, text_halign = text.align_left, bgcolor = rowColor
     )
    // Show the result of formatting the `timeValue` based on the `formatString` and `timezoneValue` in the se
    displayTable.cell(
         1, rowID, str.format_time(timeValue, formatString, timezoneValue),
         text_color = color.black, text_halign = text.align_right, bgcolor = rowColor
     )

if barstate.islast
    // Initialize the table's header cells.
    displayTable.cell(0, 0, "FORMAT STRINGS")
    displayTable.cell(1, 0, "FORMATTED DATE/TIME OUTPUT")
    // Initialize a row to show the default date-time "string" format and its result for `time`.
    displayTable.cell(
         0, 1, "(Default `str.format_time()` format)",
         text_color = color.black, text_halign = text.align_left, bgcolor = color.yellow)
    displayTable.cell(
         1, 1, str.format_time(time),
         text_color = color.black, text_halign = text.align_right, bgcolor = color.yellow)

    // Initialize rows to show different formatting strings and their results for `time`, `time_close`, and `t
    displayText(2, "dd/MM/yy")
    displayText(3, "MMMM dd, yyyy")
    displayText(4, "hh:mm:ss.SS a", timenow)
    displayText(5, "HH:mm 'UTC'Z")
    displayText(6, "H:mm a (zzzz)")
    displayText(7, "my day / 'my day' ('escaped')")
    displayText(8, "'Month' M, 'Week' w, 'Day' DDD")
    displayText(9, "'Bar expected closing time': ha", time_close)
    displayText(10, "'Current date/time': MMM-d-y HH:mm:ss z", timenow)
    displayText(11, "'New Time zone': zzzz", timezoneValue = "Australia/Sydney")
    displayText(12, "'Time zone change': MMM-d-y HH:mm:ss z", timenow, "Australia/Sydney")
```

Expressing time differences

Every UNIX timestamp represents a specific point in time as the absolute *time difference* from a fixed historical point (epoch). The specific epoch all UNIX timestamps reference is *midnight UTC on January 1, 1970*. Programmers can format UNIX timestamps into readable date-time strings with the str.format_time() function because it uses the time difference from the UNIX Epoch in its date and time calculations.

In contrast, the difference between two nonzero UNIX timestamps represents the number of milliseconds elapsed from one absolute point to another. The difference does not directly refer to a specific point in UNIX time if neither timestamp in the operation has a value of 0 (corresponding to the UNIX Epoch).

Programmers may want to express the millisecond difference between two UNIX timestamps in *other time units*, such as seconds, days, etc. Some might assume they can use the difference as the `time` argument in a str.format_time() call to achieve this result. However, the function always treats its `time` argument as the time elapsed from the *UNIX Epoch* to derive a *calendar date/time* representation in a specific time zone. It **does not** express time differences directly. Therefore, attempting to format timestamp *differences* rather than timestamps with str.format_time() leads to unintended results.

For example, the following script calculates the millisecond difference between the current execution time (timenow) and the "1M" bar's closing time (time_close("1M")) for a monthly countdown timer display. It attempts to express the time difference in another format using str.format_time(). It displays the function call's result in a table, along with the original millisecond difference (`timeLeft`) and formatted date-time representations of the timestamps.

As we see below, the table shows correct results for the formatted timestamps and the `timeLeft` value. However, the formatted time difference appears as `"1970-01-12T16:47:10-0500"`. Although the `timeLeft` value is supposed to represent a difference between timestamps rather than a specific point in time, the str.format_time() function still treats the value as a **UNIX timestamp**. Consequently, it creates a "string" expressing the value as a *date and time* in the UTC-5 time zone:

Figure 317: image

```
//@version=6
indicator("Incorrectly formatting time difference demo", overlay = true)

//@variable A table that displays monthly close countdown information.
var table displayTable = table.new(position.top_right, 1, 4, color.rgb(0, 188, 212, 60))

if barstate.islast
    //@variable A UNIX timestamp representing the current time as of the script's latest execution.
    int currentTime = timenow
    //@variable A UNIX timestamp representing the expected closing time of the current "1M" bar.
    int monthCloseTime = time_close("1M")

    //@variable The number of milliseconds between the `currentTime` and the `monthCloseTime`.
    //          This value is NOT intended as a UNIX timestamp.
    int timeLeft = monthCloseTime - currentTime
    //@variable A "string" representing the `timeLeft` as a date and time in the exchange time zone, in ISO 86
    //          This format is INCORRECT for the `timeLeft` value because it's supposed to represent the time
    //          two nonzero UNIX timestamps, NOT a specific point in time.
    string incorrectTimeFormat = str.format_time(timeLeft)

    // Initialize `displayTable` cells to initialize the `currentTime` and `monthCloseTime`.
    displayTable.cell(
         0, 0, "Current time: " + str.format_time(currentTime, "HH:mm:ss.S dd/MM/yy (z)"),
         text_size = size.large, text_halign = text.align_right
     )
    displayTable.cell(
         0, 1, "`1M` Bar closing time: " + str.format_time(monthCloseTime, "HH:mm:ss.SS dd/MM/yy (z)"),
         text_size = size.large, text_halign = text.align_right
     )
    // Initialize a cell to display the `timeLeft` millisecond difference.
    displayTable.cell(
         0, 2, "`timeLeft` value: " + str.tostring(timeLeft),
         text_size = size.large, bgcolor = color.yellow
     )
    // Initialize a cell to display the `incorrectTimeFormat` representation.
    displayTable.cell(
         0, 3, "Time left (incorrect format): " + incorrectTimeFormat,
```

```
        text_size = size.large, bgcolor = color.maroon, text_color = color.white
    )
```

To express the difference between timestamps in other time units correctly, programmers must write code that *calculates* the number of units elapsed instead of erroneously formatting the difference as a specific date or time.

The calculations required to express time differences depend on the chosen time units. The sections below explain how to express millisecond differences in weekly and smaller units, and monthly and larger units.

Weekly and smaller units

Weeks and smaller time units (days, hours, minutes, seconds, and milliseconds) cover *consistent* blocks of time. These units have the following relationship:

- One week equals seven days.
- One day equals 24 hours.
- One hour equals 60 minutes.
- One minute equals 60 seconds.
- One second equals 1000 milliseconds.

Using this relationship, programmers can define the span of these units by the number of *milliseconds* they contain. For example, since every hour has 60 minutes, every minute has 60 seconds, and every second has 1000 milliseconds, the number of milliseconds per hour is `60 * 60 * 1000`, which equals `3600000`.

Programmers can use *modular arithmetic* based on the milliseconds in each unit to calculate the total number of weeks, days, and smaller spans covered by the difference between two UNIX timestamps. The process is as follows, starting from the *largest* time unit in the calculation:

1. Calculate the number of milliseconds in the time unit.
2. Divide the remaining millisecond difference by the calculated value and round down to the nearest whole number. The result represents the number of *complete* time units within the interval.
3. Use the *remainder* from the division as the new remaining millisecond difference.
4. Repeat steps 1-3 for each time unit in the calculation, in *descending* order based on size.

The following script implements this process in a custom `formatTimeSpan()` function. The function accepts two UNIX timestamps defining a start and end point, and its "bool" parameters control whether it calculates the number of weeks or smaller units covered by the time range. The function calculates the millisecond distance between the two timestamps. It then calculates the numbers of complete units covered by that distance and formats the results into a "string".

The script calls `formatTimeSpan()` to express the difference between two separate time input values in selected time units. It then displays the resulting "string" in a table alongside formatted representations of the start and end times:

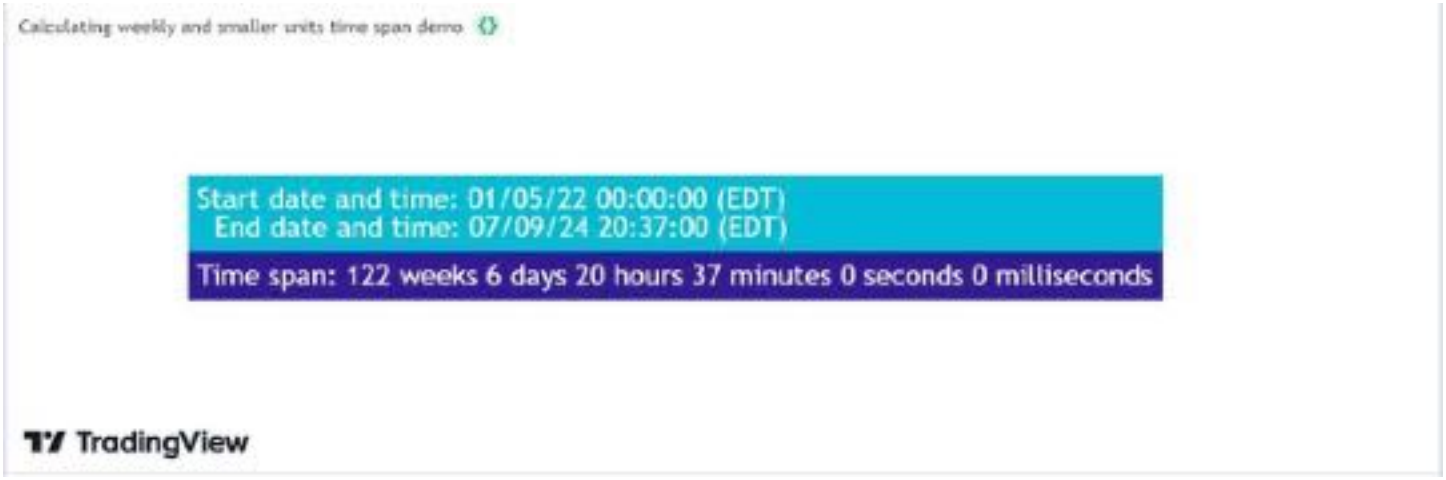


Figure 318: image

```
//@version=6
indicator("Calculating time span demo")

// Assign the number of milliseconds in weekly and smaller units to "const" variables for convenience.
const int ONE_WEEK   = 604800000
```

```
const int ONE_DAY    =  86400000
const int ONE_HOUR   =   3600000
const int ONE_MINUTE =     60000
const int ONE_SECOND =      1000

//@variable A UNIX timestamp calculated from the user-input start date and time.
int startTimeInput = input.time(timestamp("1 May 2022 00:00 -0400"), "Start date and time", group = "Time betw
//@variable A UNIX timestamp calculated from the user-input end date and time.
int endTimeInput =   input.time(timestamp("7 Sep 2024 20:37 -0400"), "End date and time", group = "Time betwee
// Create "bool" inputs to toggle weeks, days, hours, minutes, seconds, and milliseconds in the calculation.
bool weeksInput        = input.bool(true, "Weeks",        group = "Time units", inline = "A")
bool daysInput         = input.bool(true, "Days",         group = "Time units", inline = "A")
bool hoursInput        = input.bool(true, "Hours",        group = "Time units", inline = "B")
bool minutesInput      = input.bool(true, "Minutes",      group = "Time units", inline = "B")
bool secondsInput      = input.bool(true, "Seconds",      group = "Time units", inline = "B")
bool millisecondsInput = input.bool(true, "Milliseconds", group = "Time units", inline = "B")

//@function Calculates the difference between two UNIX timestamps as the number of complete time units in
//          descending order of size, formatting the results into a "string". The "int" parameters accept time
//          and the "bool" parameters determine which units the function uses in its calculations.
formatTimeSpan(
      int startTimestamp, int endTimestamp, bool calculateWeeks, bool calculateDays, bool calculateHours,
      bool calculateMinutes, bool calculateSeconds, bool calculateMilliseconds
 ) =>
    //@variable The milliseconds between the `startTimestamp` and `endTimestamp`.
    int timeDifference = math.abs(endTimestamp - startTimestamp)
    //@variable A "string" representation of the interval in mixed time units.
    string formattedString = na
    // Calculate complete units within the interval for each toggled unit, reducing the `timeDifference` by th
    if calculateWeeks
        int totalWeeks = math.floor(timeDifference / ONE_WEEK)
        timeDifference %= ONE_WEEK
        formattedString += str.tostring(totalWeeks) + (totalWeeks == 1 ? " week " : " weeks ")
    if calculateDays
        int totalDays = math.floor(timeDifference / ONE_DAY)
        timeDifference %= ONE_DAY
        formattedString += str.tostring(totalDays) + (totalDays == 1 ? " day " : " days ")
    if calculateHours
        int totalHours = math.floor(timeDifference / ONE_HOUR)
        timeDifference %= ONE_HOUR
        formattedString += str.tostring(totalHours) + (totalHours == 1 ? " hour " : " hours ")
    if calculateMinutes
        int totalMinutes = math.floor(timeDifference / ONE_MINUTE)
        timeDifference %= ONE_MINUTE
        formattedString += str.tostring(totalMinutes) + (totalMinutes == 1 ? " minute " : " minutes ")
    if calculateSeconds
        int totalSeconds = math.floor(timeDifference / ONE_SECOND)
        timeDifference %= ONE_SECOND
        formattedString += str.tostring(totalSeconds) + (totalSeconds == 1 ? " second " : " seconds ")
    if calculateMilliseconds
        // `timeDifference` is in milliseconds already, so add it to the `formattedString` directly.
        formattedString += str.tostring(timeDifference) + (timeDifference == 1 ? " millisecond" : " millisecon
    // Return the `formattedString`.
    formattedString

if barstate.islastconfirmedhistory
    //@variable A table that that displays formatted start and end times and their custom-formatted time diffe
    var table displayTable = table.new(position.middle_center, 1, 2, color.aqua)
    //@variable A "string" containing formatted `startTimeInput` and `endTimeInput` values.
    string timeText = "Start date and time: "    + str.format_time(startTimeInput, "dd/MM/yy HH:mm:ss (z)")
```

```
                + "\n  End date and time: " + str.format_time(endTimeInput, "dd/MM/yy HH:mm:ss (z)")
//@variable A "string" representing the span between `startTimeInput` and `endTimeInput` in mixed time uni
string userTimeSpan = formatTimeSpan(
     startTimeInput, endTimeInput, weeksInput, daysInput, hoursInput, minutesInput, secondsInput, millisec
  )
// Display the `timeText` in the table.
displayTable.cell(0, 0, timeText,
     text_color = color.white, text_size = size.large, text_halign = text.align_left)
// Display the `userTimeSpan` in the table.
displayTable.cell(0, 1, "Time span: " + userTimeSpan,
      text_color = color.white, text_size = size.large, text_halign = text.align_left, bgcolor = color.nav
```

Note that:

- The user-defined function uses math.floor() to round each divided result down to the nearest "int" value to get the number of *complete* units in the interval. After division, it uses the modulo assignment operator (%=) to get the *remainder* and assign that value to the `timeDifference` variable. This process repeats for each selected unit.

The image above shows the calculated time difference in mixed time units. By toggling the "bool" inputs, users can also isolate specific units in the calculation. For example, this image shows the result after enabling only the "Milliseconds" input:

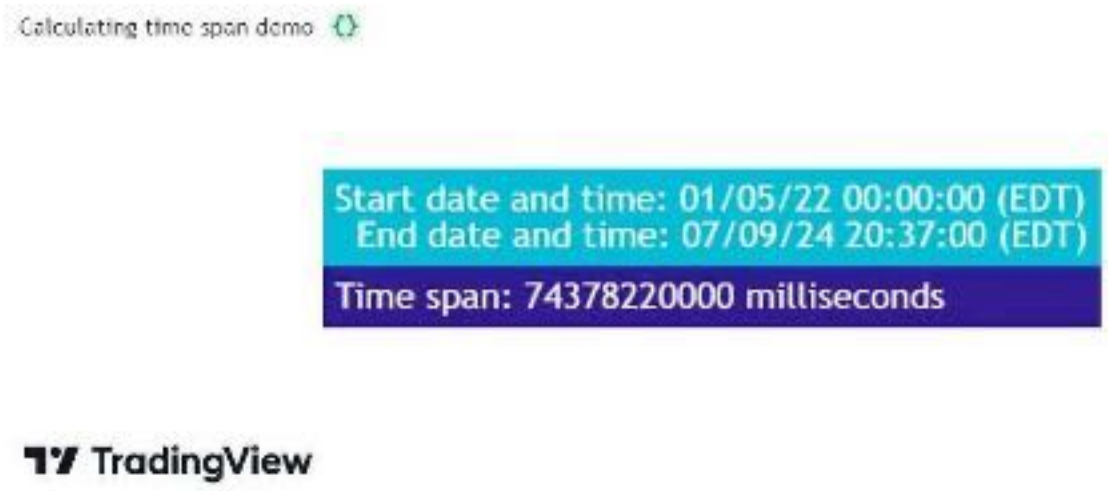


Figure 319: image

Monthly and larger units

Unlike weeks and smaller units, months and larger units *vary* in length based on calendar rules. For example, a month can contain 28, 29, 30, or 31 days, and a year can contain 365 or 366 days.

Some programmers prefer to use the modular arithmetic outlined in the previous section, with *approximate lengths* for these irregular units, to calculate large-unit durations between UNIX timestamps. With this process, programmers usually define the units in either of the following ways:

- Using *common* lengths, e.g., a common year equals 365 days, and a common month equals 30 days.
- Using the *average* lengths, e.g., an average year equals 365.25 days, and an average month equals 30.4375 days.

Calculations involving approximate units produce *rough estimates* of the elapsed time. Such estimates are often practical when expressing relatively short durations. However, their precision diminishes with the size of the difference, drifting further away from the actual time elapsed.

Therefore, expressing time differences in monthly and larger units with precision requires a different calculation than the process outlined above. For a more precise estimate of months, years, and larger units elapsed, the calculations should use the *actual* span of each individual unit rather than approximations, meaning it must account for *leap years* and *variations* in month sizes.

The advanced example below contains a custom `formatTimeDifference()` function that calculates the years and months, in addition to days and smaller units, elapsed between two UNIX timestamps.

The function uses the process outlined in the previous section to calculate the daily and smaller units within the interval. For the monthly and yearly units, which have *irregular* lengths, the function uses a while loop to iterate across calendar months. On each iteration, it increments monthly and yearly counters and subtracts the number of days in the added month from the day counter. After the loop ends, the function adjusts the year, month, and day counters to account for partial months elapsed between the timestamps. Finally, it uses the counters in a str.format() call to create a formatted "string" containing the calculated values.

The script calls this `formatTimeDifference()` function to calculate the years, months, days, hours, minutes, seconds, and milliseconds elapsed between two separate time input values and displays the result in a label:

Figure 320: image

```
//@version=6
indicator("Calculating time span for larger units demo")

//@variable The starting date and time of the time span, input by the user.
int startTimeInput = input.time(timestamp("3 Apr 2022 20:00 -0400"), "Start date", group = "Time between")
//@variable The ending date and time of the time span, input by the user.
int endTimeInput =   input.time(timestamp("3 Sep 2024 15:45 -0400"), "End date",   group = "Time between")

//@function Returns the number of days in the `monthNumber` month of the `yearNumber` year.
daysPerMonth(int yearNumber, int monthNumber) =>
    //@variable Is `true` if the `yearNumber` represents a leap year.
    bool leapYear = (yearNumber % 4 == 0 and yearNumber % 100 != 0) or (yearNumber % 400 == 0)
    //@variable The number of days calculated for the month.
    int result = switch
        monthNumber == 2 => leapYear ? 29 : 28
        =>                  31 - (monthNumber - 1) % 7 % 2

//@function Calculates the relative time difference between two timestamps, covering monthly and larger units.
formatTimeDifference(int timestamp1, int timestamp2) =>
    // The starting time and ending time.
    int startTime = math.min(timestamp1, timestamp2), int endTime = math.max(timestamp1, timestamp2)
    // The year, month, and day of the `startTime` and `endTime`.
    int startYear = year(startTime), int startMonth = month(startTime), int startDay = dayofmonth(startTime)
    int endYear   = year(endTime),   int endMonth   = month(endTime),   int endDay   = dayofmonth(endTime)
    // Calculate the total number of days, hours, minutes, seconds, and milliseconds in the interval.
    int milliseconds = endTime - startTime
    int days         = math.floor(milliseconds / 86400000), milliseconds %= 86400000
    int hours        = math.floor(milliseconds / 3600000),  milliseconds %= 3600000
    int minutes      = math.floor(milliseconds / 60000),    milliseconds %= 60000
```

```
    int seconds      = math.floor(milliseconds / 1000),     milliseconds %= 1000
    // Calculate the number of days in the `startMonth` and `endMonth`.
    int daysInStartMonth = daysPerMonth(startYear, startMonth), int daysInEndMonth = daysPerMonth(endYear, endM
    //@variable The number of days remaining in the `startMonth`.
    int remainingInMonth = daysInStartMonth - startDay + 1
    // Subtract `remainingInMonth` from the `days`, and offset the `startDay` and `startMonth`.
    days -= remainingInMonth, startDay := 1, startMonth += 1
    // Set `startMonth` to 1, and increase the `startYear` if the `startMonth` exceeds 12.
    if startMonth > 12
        startMonth := 1, startYear += 1
    // Initialize variables to count the total number of months and years in the interval.
    int months = 0, int years  = 0
    // Loop to increment `months` and `years` values based on the `days`.
    while days > 0
        //@variable The number of days in the current `startMonth`.
        int daysInMonth = daysPerMonth(startYear, startMonth)
        // Break the loop if the number of remaining days is less than the `daysInMonth`.
        if days < daysInMonth
            break
        // Reduce the `days` by the `daysInMonth` and increment the `months`.
        days -= daysInMonth, months += 1
        // Increase the `years` and reset the `months` to 0 when `months` is 12.
        if months == 12
            months := 0, years += 1
        // Increase the `startMonth` and adjust the `startMonth` and `startYear` if its value exceeds 12.
        startMonth += 1
        if startMonth > 12
            startMonth := 1, startYear += 1
    // Re-add the `remainingInMonth` value to the number of `days`. Adjust the `days`, `months`, and `years` if
    // new value exceeds the `daysInStartMonth` or `daysInEndMonth`, depending on the `startDay`.
    days += remainingInMonth
    if days >= (startDay < daysInStartMonth / 2 ? daysInStartMonth : daysInEndMonth)
        months += 1
        if months == 12
            months := 0, years += 1
        days -= remainingInMonth
    // Format the calculated values into a "string" and return the result.
    str.format(
         " Years:    {0}\n Months:  {1}\n Days:     {2}\n Hours:    {3}\n Minutes: {4}\n Seconds: {5}
         \n Milliseconds: {6}", years, months, days, hours, minutes, seconds, milliseconds
     )

if barstate.islastconfirmedhistory
    //@variable A "string" representing the time between the `startTimeInput` and `endTimeInput` in mixed unit
    string userTimeSpan = formatTimeDifference(startTimeInput, endTimeInput)
    //@variable Text shown in the label.
    string labelText = "Start time: " + str.format_time(startTimeInput, "dd/MM/yy HH:mm:ss (z)") + "\n  End ti
         +  str.format_time(endTimeInput, "dd/MM/yy HH:mm:ss (z)") + "\n---------\nTime difference:\n" + userT
    label.new(
         bar_index, high, labelText, color = #FF946E, size = size.large,
         textalign = text.align_left, style = label.style_label_center
     )
```

Note that:

- The script determines the number of days in each month with the user-defined `daysPerMonth()` function. The function identifies whether a month has 28, 29, 30, or 31 days based on its month number and the year it belongs to. Its calculation accounts for leap years. A leap year occurs when the year is divisible by 4 or 400 but not by 100.
- Before the while loop, the function subtracts the number of days in a partial starting month from the initial day count, aligning the counters with the beginning of a new month. It re-adds the subtracted days after the loop to adjust the counters for partial months. It adjusts the month and year counters based on the days in the `startMonth` if the

`startDay` is less than halfway through that month. Otherwise, it adjusts the values based on the days in the `endMonth`.

[Previous

Text and shapes](#text-and-shapes)[Next

Timeframes](#timeframes) User Manual/Concepts/Timeframes

Timeframes

Introduction

The *timeframe* of a chart is sometimes also referred to as its *interval* or *resolution*. It is the unit of time represented by one bar on the chart. All standard chart types use a timeframe: "Bars", "Candles", "Hollow Candles", "Line", "Area" and "Baseline". One non-standard chart type also uses timeframes: "Heikin Ashi".

Programmers interested in accessing data from multiple timeframes will need to become familiar with how timeframes are expressed in Pine Script™, and how to use them.

Timeframe strings come into play in different contexts:

- They must be used in request.security() when requesting data from another symbol and/or timeframe. See the page on Other timeframes and data to explore the use of request.security().
- They can be used as an argument to time() and time_close() functions, to return the time of a higher timeframe bar. This, in turn, can be used to detect changes in higher timeframes from the chart's timeframe without using request.security(). See the Testing for changes in higher timeframes section to see how to do this.
- The input.timeframe() function provides a way to allow script users to define a timeframe through a script's "Inputs" tab (see the Timeframe input section for more information).
- The indicator() declaration statement has an optional `timeframe` parameter that can be used to provide multi-timeframe capabilities to simple scripts without using request.security().
- Many built-in variables provide information on the timeframe used by the chart the script is running on. See the Chart timeframe section for more information on them, including timeframe.period which returns a string in Pine Script™'s timeframe specification format.

Timeframe string specifications

Timeframe strings follow these rules:

- They are composed of the multiplier and the timeframe unit, e.g., "1S", "30" (30 minutes), "1D" (one day), "3M" (three months).
- The unit is represented by a single letter, with no letter used for minutes: "T" for ticks, "S" for seconds, "D" for days, "W" for weeks, and "M" for months.
- When no multiplier is used, 1 is assumed: "S" is equivalent to "1S", "D" to "1D", etc. If only "1" is used, it is interpreted as 1 minute, since no unit letter identifier is used for minutes.
- There is no "hour" unit; "1H" is **not** valid. The correct format for one hour is "60" (remember no unit letter is specified for minutes).
- The valid multipliers vary for each timeframe unit:
  - For ticks, only the discrete 1, 10, 100, and 1000 multipliers are valid.
  - For seconds, only the discrete 1, 5, 10, 15, 30, and 45 multipliers are valid.
  - For minutes, 1 to 1440.
  - For days, 1 to 365.
  - For weeks, 1 to 52.
  - For months, 1 to 12.

Comparing timeframes

It can be useful to compare different timeframe strings to determine, for example, if the timeframe used on the chart is lower than the higher timeframes used in the script.

Converting timeframe strings to a representation in fractional minutes provides a way to compare them using a universal unit. This script uses the timeframe.in_seconds() function to convert a timeframe into float seconds and then converts the result into minutes:

```
//@version=6
indicator("Timeframe in minutes example", "", true)
string tfInput = input.timeframe(defval = "", title = "Input TF")

float chartTFInMinutes = timeframe.in_seconds() / 60
float inputTFInMinutes = timeframe.in_seconds(tfInput) / 60

var table t = table.new(position.top_right, 1, 1)
string txt = "Chart TF: "    + str.tostring(chartTFInMinutes, "#.##### minutes") +
  "\nInput TF: " + str.tostring(inputTFInMinutes, "#.##### minutes")
if barstate.isfirst
    table.cell(t, 0, 0, txt, bgcolor = color.yellow)
else if barstate.islast
    table.cell_set_text(t, 0, 0, txt)

if chartTFInMinutes > inputTFInMinutes
    runtime.error("The chart's timeframe must not be higher than the input's timeframe.")
```

Note that:

- We use the built-in timeframe.in_seconds() function to convert the chart and the input.timeframe() function into seconds, then divide by 60 to convert into minutes.
- We use two calls to the timeframe.in_seconds() function in the initialization of the `chartTFInMinutes` and `inputTFInMinutes` variables. In the first instance, we do not supply an argument for its `timeframe` parameter, so the function returns the chart's timeframe in seconds. In the second call, we supply the timeframe selected by the script's user through the call to input.timeframe().
- Next, we validate the timeframes to ensure that the input timeframe is equal to or higher than the chart's timeframe. If it is not, we generate a runtime error.
- We finally print the two timeframe values converted to minutes.

[Previous

Time](#time) User Manual/Writing scripts/Style guide

Style guide

Introduction

This style guide provides recommendations on how to name variables and organize your Pine scripts in a standard way that works well. Scripts that follow our best practices will be easier to read, understand and maintain.

You can see scripts using these guidelines published from the TradingView and PineCoders accounts on the platform.

Naming Conventions

We recommend the use of:

- `camelCase` for all identifiers, i.e., variable or function names: `ma`, `maFast`, `maLengthInput`, `maColor`, `roundedOHLC()`, `pivotHi()`.
- All caps `SNAKE_CASE` for constants: `BULL_COLOR`, `BEAR_COLOR`, `MAX_LOOKBACK`.
- The use of qualifying suffixes when it provides valuable clues about the type or provenance of a variable: `maShowInput`, `bearColor`, `bearColorInput`, `volumesArray`, `maPlotID`, `resultsTable`, `levelsColorArray`.

Script organization

The Pine Script™ compiler is quite forgiving of the positioning of specific statements or the version compiler annotation in the script. While other arrangements are syntactically correct, this is how we recommend organizing scripts:

```
<license><version><declaration_statement><import_statements><constant_declarations><inputs><function_declarati
```

If you publish your open-source scripts publicly on TradingView (scripts can also be published privately), your open-source code is by default protected by the Mozilla license. You may choose any other license you prefer.

The reuse of code from those scripts is governed by our House Rules on Script Publishing which preempt the author's license.

The standard license comments appearing at the beginning of scripts are:

```
// This source code is subject to the terms of the Mozilla Public License 2.0 at https://mozilla.org/MPL/2.0/
// © username
```

This is the compiler annotation defining the version of Pine Script™ the script will use. If none is present, v1 is used. For v6, use:

```
//@version=6
```

This is the mandatory declaration statement which defines the type of your script. It must be a call to either indicator(), strategy(), or library().

If your script uses one or more Pine Script™ libraries, your import statements belong here.

Scripts can declare variables qualified as "const", i.e., ones referencing a constant value.

We refer to variables as "constants" when they meet these criteria:

- Their declaration uses the optional `const` keyword (see our User Manual's section on type qualifiers for more information).
- They are initialized using a literal (e.g., `100` or `"AAPL"`) or a built-in qualified as "const" (e.g., `color.green`).
- Their value does not change during the script's execution.

We use `SNAKE_CASE` to name these variables and group their declaration near the top of the script. For example:

```
// ----- Constants
int     MS_IN_MIN   = 60 * 1000
int     MS_IN_HOUR  = MS_IN_MIN  * 60
int     MS_IN_DAY   = MS_IN_HOUR * 24

color   GRAY        = #808080ff
color   LIME        = #00FF00ff
color   MAROON      = #800000ff
color   ORANGE      = #FF8000ff
color   PINK        = #FF0080ff
color   TEAL        = #008080ff
color   BG_DIV      = color.new(ORANGE, 90)
color   BG_RESETS   = color.new(GRAY, 90)

string  RST1        = "No reset; cumulate since the beginning of the chart"
string  RST2        = "On a stepped higher timeframe (HTF)"
string  RST3        = "On a fixed HTF"
string  RST4        = "At a fixed time"
string  RST5        = "At the beginning of the regular session"
string  RST6        = "At the first visible chart bar"
string  RST7        = "Fixed rolling period"

string  LTF1        = "Least precise, covering many chart bars"
string  LTF2        = "Less precise, covering some chart bars"
string  LTF3        = "More precise, covering less chart bars"
```

```
string  LTF4        = "Most precise, 1min intrabars"

string  TT_TOTVOL     = "The 'Bodies' value is the transparency of the total volume candle bodies. Zero is opa
string  TT_RST_HTF    = "This value is used when '" + RST3 +"' is selected."
string  TT_RST_TIME   = "These values are used when '" + RST4 +"' is selected.
  A reset will occur when the time is greater or equal to the bar's open time, and less than its close time.\n
string  TT_RST_PERIOD = "This value is used when '" + RST7 +"' is selected."
```

In this example:

- The `RST*` and `LTF*` constants will be used as tuple elements in the `options` argument of `input.*()` calls.
- The `TT_*` constants will be used as `tooltip` arguments in `input.*()` calls. Note how we use a line continuation for long string literals.
- We do not use var to initialize constants. The Pine Script™ runtime is optimized to handle declarations on each bar, but using var to initialize a variable only the first time it is declared incurs a minor penalty on script performance because of the maintenance that var variables require on further bars.

Note that:

- Literals used in more than one place in a script should always be declared as a constant. Using the constant rather than the literal makes it more readable if it is given a meaningful name, and the practice makes code easier to maintain. Even though the quantity of milliseconds in a day is unlikely to change in the future, `MS_IN_DAY` is more meaningful than `1000 * 60 * 60 * 24`.
- Constants only used in the local block of a function or if, while, etc., statement for example, can be declared in that local block.

It is **much** easier to read scripts when all their inputs are in the same code section. Placing that section at the beginning of the script also reflects how they are processed at runtime, i.e., before the rest of the script is executed.

Suffixing input variable names with `input` makes them more readily identifiable when they are used later in the script: `maLengthInput`, `bearColorInput`, `showAvgInput`, etc.

```
// ----- Inputs
string  resetInput              = input.string(RST2,        "CVD Resets",                       inline = "00",
string  fixedTfInput            = input.timeframe("D",      "    Fixed HTF:    ",                   tooltip = T
int     hourInput               = input.int(9,              "    Fixed time hour: ",             inline = "01",
int     minuteInput             = input.int(30,             "minute",                           inline = "01",
int     fixedPeriodInput        = input.int(20,             "    Fixed period:    ",                inline = "0
string  ltfModeInput            = input.string(LTF3,        "Intrabar precision",               inline = "03",
```

All user-defined functions must be defined in the script's global scope; nested function definitions are not allowed in Pine Script™.

Optimal function design should minimize the use of global variables in the function's scope, as they undermine function portability. When it can't be avoided, those functions must follow the global variable declarations in the code, which entails they can't always be placed in the section. Such dependencies on global variables should ideally be documented in the function's comments.

It will also help readers if you document the function's objective, parameters and result. The same syntax used in libraries can be used to document your functions. This can make it easier to port your functions to a library should you ever decide to do so:

```
//@version=6
indicator("<function_declarations>", "", true)

string SIZE_LARGE  = "Large"
string SIZE_NORMAL = "Normal"
string SIZE_SMALL  = "Small"

string sizeInput = input.string(SIZE_NORMAL, "Size", options = [SIZE_LARGE, SIZE_NORMAL, SIZE_SMALL])
```

```
// @function        Used to produce an argument for the `size` parameter in built-in functions.
// @param userSize  (simple string) User-selected size.
// @returns         One of the `size.*` built-in constants.
// Dependencies     SIZE_LARGE, SIZE_NORMAL, SIZE_SMALL
getSize(simple string userSize) =>
    result =
      switch userSize
        SIZE_LARGE  => size.large
        SIZE_NORMAL => size.normal
        SIZE_SMALL  => size.small
        => size.auto

if ta.rising(close, 3)
    label.new(bar_index, na, yloc = yloc.abovebar, style = label.style_arrowup, size = getSize(sizeInput))
```

This is where the script's core calculations and logic should be placed. Code can be easier to read when variable declarations are placed near the code segment using the variables. Some programmers prefer to place all their non-constant variable declarations at the beginning of this section, which is not always possible for all variables, as some may require some calculations to have been executed before their declaration.

Strategies are easier to read when strategy calls are grouped in the same section of the script.

This section should ideally include all the statements producing the script's visuals, whether they be plots, drawings, background colors, candle-plotting, etc. See the Pine Script™ user manual's section on here for more information on how the relative depth of visuals is determined.

Alert code will usually require the script's calculations to have executed before it, so it makes sense to put it at the end of the script.

Spacing

A space should be used on both sides of all operators, except unary operators (`-1`). A space is also recommended after all commas and when using named function arguments, as in `plot(series = close)`:

```
int a = close > open ? 1 : -1
var int newLen = 2
newLen := min(20, newlen + 1)
float a = -b
float c = d > e ? d - e : d
int index = bar_index % 2 == 0 ? 1 : 2
plot(close, color = color.red)
```

Line wrapping

Line wrapping can make long lines easier to read. Line wraps are defined by using an indentation level that is not a multiple of four, as four spaces or a tab are used to define local blocks. Here we use two spaces:

```
plot(
   series = close,
   title = "Close",
   color = color.blue,
   show_last = 10
 )
```

Vertical alignment

Vertical alignment using tabs or spaces can be useful in code sections containing many similar lines such as constant declarations or inputs. They can make mass edits much easier using the Pine Editor's multi-cursor feature (`ctrl` + `alt` +):

```
// Colors used as defaults in inputs.
color COLOR_AQUA  = #0080FFff
color COLOR_BLACK = #000000ff
color COLOR_BLUE  = #013BCAff
color COLOR_CORAL = #FF8080ff
color COLOR_GOLD  = #CCCC00ff
```

Explicit typing

Including the type of variables when declaring them is not required. However, it helps make scripts easier to read, navigate, and understand. It can help clarify the expected types at each point in a script's execution and distinguish a variable's declaration (using `=`) from its reassignments (using `:=`). Using explicit typing can also make scripts easier to debug.

[Next

Debugging](#debugging) User Manual/Writing scripts/Debugging

Debugging

Introduction

TradingView's close integration between the Pine Editor and the chart interface facilitates efficient, interactive debugging of Pine Script™ code, as scripts can produce dynamic results in multiple locations, on and off the chart. Programmers can utilize such results to refine their script's behaviors and ensure everything works as expected.

When a programmer understands the appropriate techniques for inspecting the variety of behaviors one may encounter while writing a script, they can quickly and thoroughly identify and resolve potential problems in their code, which allows for a more seamless overall coding experience. This page demonstrates some of the handiest ways to debug code when working with Pine Script™.

The lay of the land

Pine scripts can output their results in multiple different ways, any of which programmers can utilize for debugging.

The `plot*()` functions can display results in a chart pane, the script's status line, the price (y-axis) scale, and the Data Window, providing simple, convenient ways to debug numeric and conditional values:

```
//@version=6
indicator("The lay of the land - Plots")

// Plot the `bar_index` in all available locations.
plot(bar_index, "bar_index", color.teal, 3)
```

Note that:

- A script's status line outputs will only show when enabling the "Values" checkbox within the "Indicators" section of the chart's "Status line" settings.
- Price scales will only show plot values or names when enabling the options from the "Indicators and financials" dropdown in the chart's "Scales and lines" settings.

The bgcolor() function displays colors in the script pane's background, and the barcolor() function changes the colors of the main chart's bars or candles. Both of these functions provide a simple way to visualize conditions:

```
//@version=6
indicator("The lay of the land - Background and bar colors")

//@variable Is `true` if the `close` is rising over 2 bars.
bool risingPrice = ta.rising(close, 2)
```

Figure 321: image

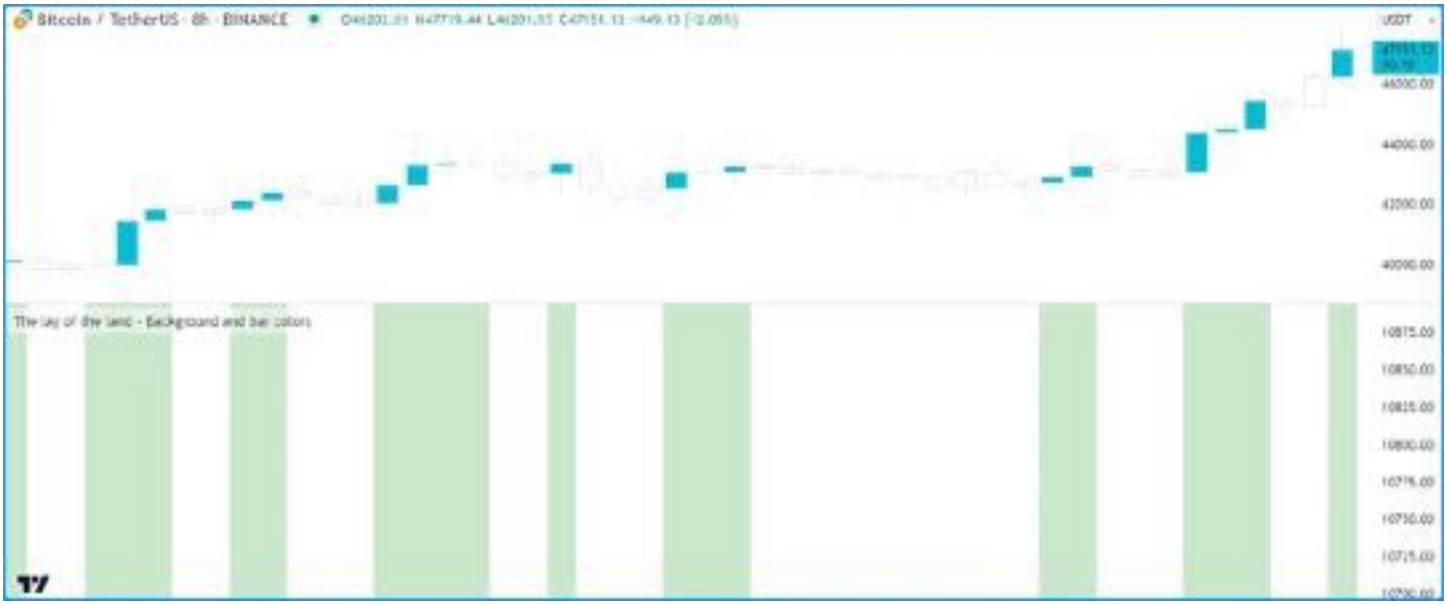


Figure 322: image

```
// Highlight the chart background and color the main chart bars based on `risingPrice`.
bgcolor(risingPrice ? color.new(color.green, 70) : na, title= "`risingPrice` highlight")
barcolor(risingPrice ? color.aqua : chart.bg_color, title = "`risingPrice` bar color")
```

Pine's drawing types (line, box, polyline, label) produce drawings in the script's pane. While they don't return results in other locations, such as the status line or Data Window, they provide alternative, flexible solutions for inspecting numeric values, conditions, and strings directly on the chart:

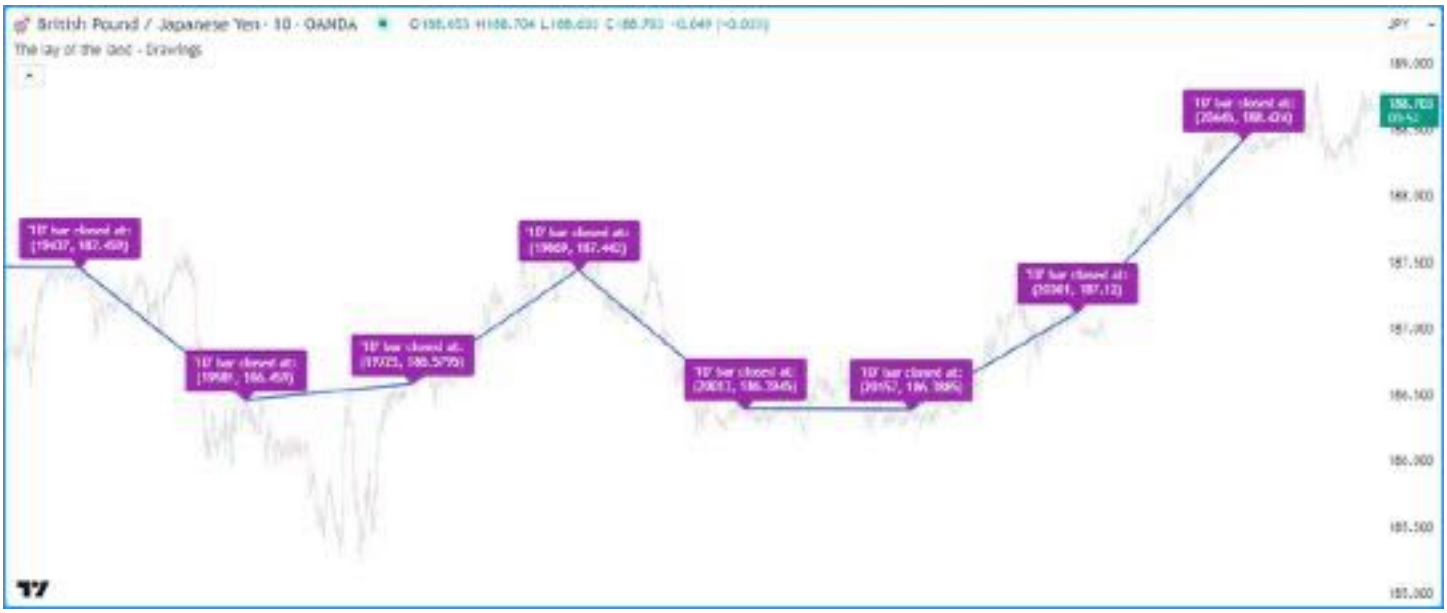


Figure 323: image

```
//@version=6
indicator("The lay of the land - Drawings", overlay = true)

//@variable Is `true` when the time changes on the "1D" timeframe.
bool newDailyBar = timeframe.change("1D")
//@variable The previous bar's `bar_index` from when `newDailyBar` last occurred.
int closedIndex = ta.valuewhen(newDailyBar, bar_index - 1, 0)
//@variable The previous bar's `close` from when `newDailyBar` last occurred.
float closedPrice = ta.valuewhen(newDailyBar, close[1], 0)

if newDailyBar
    //@variable Draws a line from the previous `closedIndex` and `closedPrice` to the current values.
    line debugLine = line.new(closedIndex[1], closedPrice[1], closedIndex, closedPrice, width = 2)
    //@variable Variable info to display in a label.
    string debugText = "'1D' bar closed at: \n(" + str.tostring(closedIndex) + ", " + str.tostring(closedPrice
    //@variable Draws a label at the current `closedIndex` and `closedPrice`.
    label.new(closedIndex, closedPrice, debugText, color = color.purple, textcolor = color.white)
```

The `log.*()` functions produce Pine Logs results. Every time a script calls any of these functions, the script logs a message in the Pine Logs pane, along with a timestamp and navigation options to identify the specific times, chart bars, and lines of code that triggered a log:

```
//@version=6
indicator("The lay of the land - Pine Logs")

//@variable The natural logarithm of the current `high - low` range.
float logRange = math.log(high - low)

// Plot the `logRange`.
plot(logRange, "logRange")

if barstate.isconfirmed
```

Figure 324: image

```
    // Generate an "error" or "info" message on the confirmed bar, depending on whether `logRange` is defined.
    switch
        na(logRange) => log.error("Undefined `logRange` value.")
        =>              log.info("`logRange` value: " + str.tostring(logRange))
else
    // Generate a "warning" message for unconfirmed values.
    log.warning("Unconfirmed `logRange` value: " + str.tostring(logRange))
```

One can apply any of the above, or a combination, to establish debugging routines to fit their needs and preferences, depending on the data types and structures they're working with. See the sections below for detailed explanations of various debugging techniques.

Numeric values

When creating code in Pine Script™, working with numbers is inevitable. Therefore, to ensure a script works as intended, it's crucial to understand how to inspect the numeric (int and float) values it receives and calculates.

Plotting numbers

One of the most straightforward ways to inspect a script's numeric values is to use `plot*()` functions, which can display results graphically on the chart and show formatted numbers in the script's status line, the price scale, and the Data Window. The locations where a `plot*()` function displays its results depend on the `display` parameter. By default, its value is display.all.

The following example uses the plot() function to display the 1-bar change in the value of the built-in time variable measured in chart timeframes (e.g., a plotted value of 1 on the "1D" chart means there is a one-day difference between the opening times of the current and previous bars). Inspecting this series can help to identify time gaps in a chart's data, which is helpful information when designing time-based indicators.

Since we have not specified a `display` argument, the function uses display.all, meaning it will show data in *all* possible locations, as we see below:

```
//@version=6
indicator("Plotting numbers demo", "Time changes")

//@variable The one-bar change in the chart symbol's `time` value, measured in units of the chart timeframe.
```

Figure 325: image

```
float timeChange = ta.change(time) / (1000.0 * timeframe.in_seconds())

// Display the `timeChange` in all possible locations.
plot(timeChange, "Time difference (in chart bar units)", color.purple, 3)
```

Note that:

- The numbers displayed in the script's status line and the Data Window reflect the plotted values at the location of the chart's cursor. These areas will show the latest bar's value when the mouse pointer isn't on the chart.
- The number in the price scale reflects the latest available value on the visible chart.

Without affecting the scale When debugging multiple numeric values in a script, programmers may wish to inspect them without interfering with the price scales or cluttering the visual outputs in the chart's pane, as distorted scales and overlapping plots may make it harder to evaluate the results.

A simple way to inspect numbers without adding more visuals to the chart's pane is to change the `display` values in the script's `plot*()` calls to other `display.*` variables or expressions using them.

Let's look at a practical example. Here, we've drafted the following script that calculates a custom-weighted moving average by dividing the sum of `weight * close` values by the sum of the `weight` series:

```
//@version=6
indicator("Plotting without affecting the scale demo", "Weighted Average", true)

//@variable The number of bars in the average.
int lengthInput = input.int(20, "Length", 1)

//@variable The weight applied to the price on each bar.
float weight = math.pow(close - open, 2)

//@variable The numerator of the average.
float numerator = math.sum(weight * close, lengthInput)
//@variable The denominator of the average.
float denominator = math.sum(weight, lengthInput)

//@variable The `lengthInput`-bar weighted average.
float average = numerator / denominator

// Plot the `average`.
```

Figure 326: image

```
plot(average, "Weighted Average", linewidth = 3)
```

Suppose we'd like to inspect the variables used in the `average` calculation to understand and fine-tune the result. If we were to use plot() to display the script's `weight`, `numerator`, and `denominator` in all locations, we can no longer easily identify our `average` line on the chart since each variable has a radically different scale:

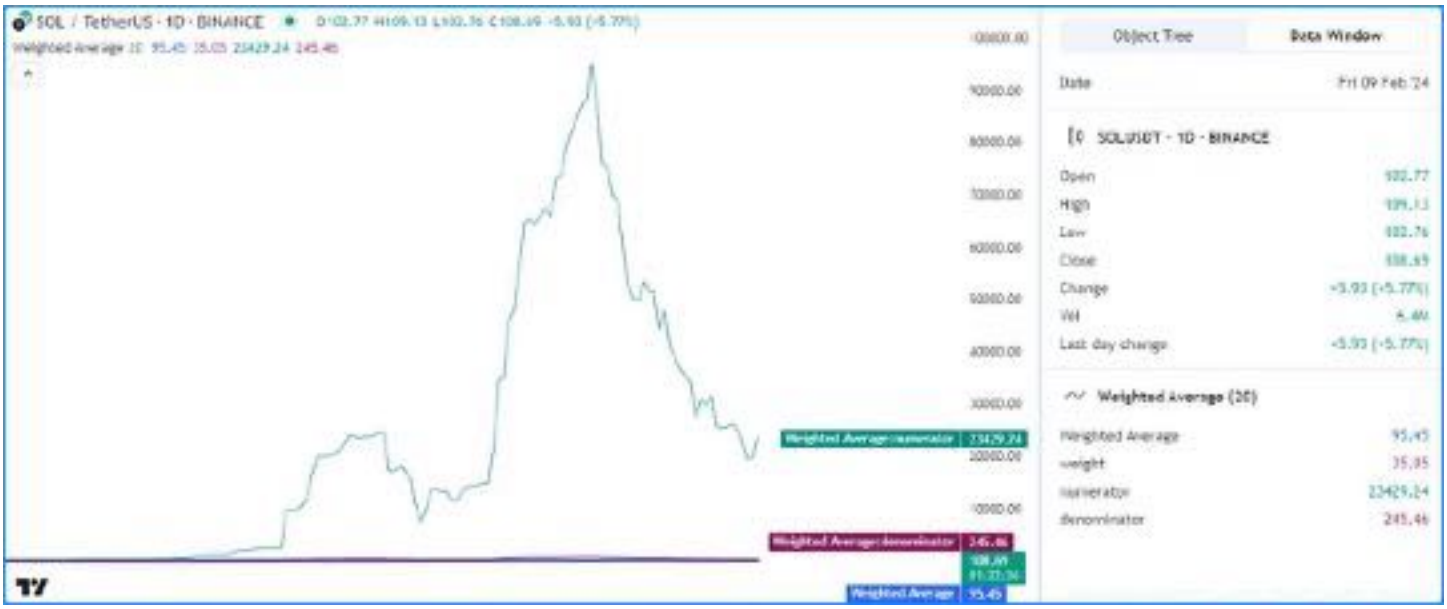


Figure 327: image

```
//@version=6
indicator("Plotting without affecting the scale demo", "Weighted Average", true)

//@variable The number of bars in the average.
int lengthInput = input.int(20, "Length", 1)

//@variable The weight applied to the price on each bar.
float weight = math.pow(close - open, 2)

//@variable The numerator of the average.
```

```
float numerator = math.sum(close * weight, lengthInput)
//@variable The denominator of the average.
float denominator = math.sum(weight, lengthInput)

//@variable The `lengthInput`-bar weighted average.
float average = numerator / denominator

// Plot the `average`.
plot(average, "Weighted Average", linewidth = 3)

// Create debug plots for the `weight`, `numerator`, and `denominator`.
plot(weight, "weight", color.purple)
plot(numerator, "numerator", color.teal)
plot(denominator, "denominator", color.maroon)
```

While we could hide individual plots from the "Style" tab of the script's settings, doing so also prevents us from inspecting the results in any other location. To simultaneously view the variables' values and preserve the scale of our chart, we can change the `display` values in our debug plots.

The version below includes a `debugLocations` variable in the debug plot() calls with a value of `display.all - display.pane` to specify that all locations *except* the chart pane will show the results. Now we can inspect the calculation's values without the extra clutter:

Figure 328: image

```
//@version=6
indicator("Plotting without affecting the scale demo", "Weighted Average", true)

//@variable The number of bars in the average.
int lengthInput = input.int(20, "Length", 1)

//@variable The weight applied to the price on each bar.
float weight = math.pow(close - open, 2)

//@variable The numerator of the average.
float numerator = math.sum(close * weight, lengthInput)
//@variable The denominator of the average.
float denominator = math.sum(weight, lengthInput)

//@variable The `lengthInput`-bar weighted average.
float average = numerator / denominator
```

```
// Plot the `average`.
plot(average, "Weighted Average", linewidth = 3)

//@variable The display locations of all debug plots.
debugLocations = display.all - display.pane
// Create debug plots for the `weight`, `numerator`, and `denominator`.
plot(weight, "weight", color.purple, display = debugLocations)
plot(numerator, "numerator", color.teal, display = debugLocations)
plot(denominator, "denominator", color.maroon, display = debugLocations)
```

From local scopes A script's *local scopes* are sections of indented code within conditional structures, functions, and methods. When working with variables declared within these scopes, using the `plot*()` functions to display their values directly *will not* work, as plots only work with literals and *global* variables.

To display a local variable's values using plots, one can assign its results to a global variable and pass that variable to the `plot*()` call.

For example, this script calculates the all-time maximum and minimum change in the close price over a `lengthInput` period. It uses an if structure to declare a local `change` variable and update the global `maxChange` and `minChange` once every `lengthInput` bars:

Figure 329: image

```
//@version=6
indicator("Plotting numbers from local scopes demo", "Periodic changes")

//@variable The number of chart bars in each period.
int lengthInput = input.int(20, "Period length", 1)

//@variable The maximum `close` change over each `lengthInput` period on the chart.
var float maxChange = na
//@variable The minimum `close` change over each `lengthInput` period on the chart.
var float minChange = na

//@variable Is `true` once every `lengthInput` bars.
bool periodClose = bar_index % lengthInput == 0

if periodClose
    //@variable The change in `close` prices over `lengthInput` bars.
    float change = close - close[lengthInput]
```

```
    // Update the global `maxChange` and `minChange`.
    maxChange := math.max(nz(maxChange, change), change)
    minChange := math.min(nz(minChange, change), change)

// Plot the `maxChange` and `minChange`.
plot(maxChange, "Max periodic change", color.teal, 3)
plot(minChange, "Min periodic change", color.maroon, 3)
hline(0.0, color = color.gray, linestyle = hline.style_solid)
```

Suppose we want to inspect the history of the `change` variable using a plot. While we cannot plot the variable directly since the script declares it in a local scope, we can assign its value to another *global* variable for use in a `plot*()` function.

Below, we've added a `debugChange` variable with an initial value of na to the global scope, and the script reassigns its value within the if structure using the local `change` variable. Now, we can use plot() with the `debugChange` variable to view the history of available `change` values:

Figure 330: image

```
//@version=6
indicator("Plotting numbers from local scopes demo", "Periodic changes")

//@variable The number of chart bars in each period.
int lengthInput = input.int(20, "Period length", 1)

//@variable The maximum `close` change over each `lengthInput` period on the chart.
var float maxChange = na
//@variable The minimum `close` change over each `lengthInput` period on the chart.
var float minChange = na

//@variable Is `true` once every `lengthInput` bars.
bool periodClose = bar_index % lengthInput == 0

//@variable Tracks the history of the local `change` variable.
float debugChange = na

if periodClose
    //@variable The change in `close` prices over `lengthInput` bars.
    float change = close - close[lengthInput]
    // Update the global `maxChange` and `minChange`.
    maxChange := math.max(nz(maxChange, change), change)
    minChange := math.min(nz(minChange, change), change)
    // Assign the `change` value to the `debugChange` variable.
```

```
    debugChange := change

// Plot the `maxChange` and `minChange`.
plot(maxChange, "Max periodic change", color.teal, 3)
plot(minChange, "Min periodic change", color.maroon, 3)
hline(0.0, color = color.gray, linestyle = hline.style_solid)

// Create a debug plot to visualize the `change` history.
plot(debugChange, "Extracted change", color.purple, 15, plot.style_areabr)
```

Note that:

- The script uses plot.style_areabr in the debug plot, which doesn't bridge over na values as the default style does.
- When the rightmost visible bar's plotted value is na the number in the price scale represents the latest *non-na* value before that bar, if one exists.

With drawings

An alternative approach to graphically inspecting the history of a script's numeric values is to use Pine's drawing types, including lines, boxes, polylines, and labels.

While Pine drawings don't display results anywhere other than the chart pane, scripts can create them from within *local scopes*, including the scopes of functions and methods (see the Debugging functions section to learn more). Additionally, scripts can position drawings at *any* available chart location, irrespective of the current bar_index.

For example, let's revisit the "Periodic changes" script from the previous section. Suppose we'd like to inspect the history of the local `change` variable *without* using a plot. In this case, we can avoid declaring a separate global variable and instead create drawing objects directly from the if structure's local scope.

The script below is a modification of the previous script that uses boxes to visualize the `change` variable's behavior. Inside the scope of the if structure, it calls box.new() to create a box that spans from the bar `lengthInput` bars ago to the current bar_index:

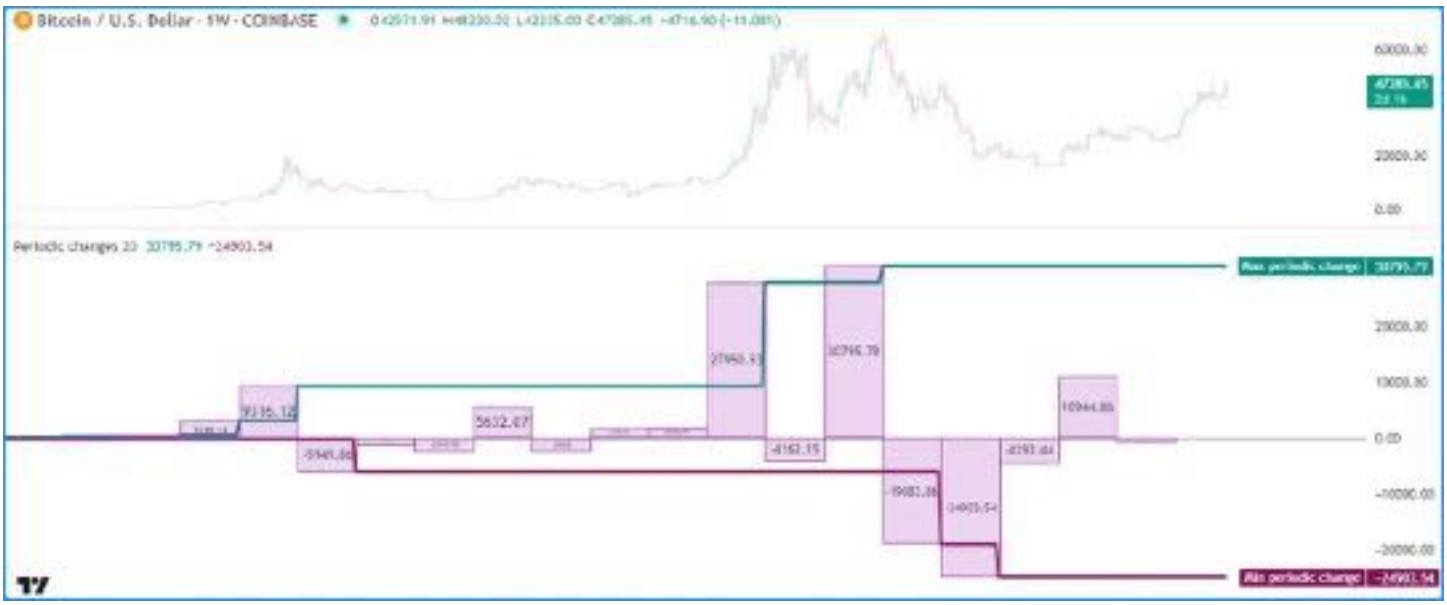


Figure 331: image

```
//@version=6
indicator("Drawing numbers from local scopes demo", "Periodic changes", max_boxes_count = 500)

//@variable The number of chart bars in each period.
int lengthInput = input.int(20, "Period length", 1)

//@variable The maximum `close` change over each `lengthInput` period on the chart.
var float maxChange = na
```

```
//@variable The minimum `close` change over each `lengthInput` period on the chart.
var float minChange = na

//@variable Is `true` once every `lengthInput` bars.
bool periodClose = bar_index % lengthInput == 0

if periodClose
    //@variable The change in `close` prices over `lengthInput` bars.
    float change = close - close[lengthInput]
    // Update the global `maxChange` and `minChange`.
    maxChange := math.max(nz(maxChange, change), change)
    minChange := math.min(nz(minChange, change), change)
    //@variable Draws a box on the chart to visualize the `change` value.
    box debugBox = box.new(
         bar_index - lengthInput, math.max(change, 0.0), bar_index, math.min(change, 0.0),
         color.purple, bgcolor = color.new(color.purple, 80), text = str.tostring(change)
     )

// Plot the `maxChange` and `minChange`.
plot(maxChange, "Max periodic change", color.teal, 3)
plot(minChange, "Min periodic change", color.maroon, 3)
hline(0.0, color = color.gray, linestyle = hline.style_solid)
```

Note that:

- The script includes `max_boxes_count = 500` in the indicator() function, which allows it to show up to 500 boxes on the chart.
- We used math.max(change, 0.0) and math.min(change, 0.0) in the box.new() function as the `top` and `bottom` values.
- The box.new() call includes str.tostring(change) as its `text` argument to display a *"string" representation* of the `change` variable's "float" value in each box drawing. See this portion of the Strings section below to learn more about representing data with strings.

For more information about using boxes and other related drawing types, see our User Manual's Lines and boxes page.

Conditions

Many scripts one will create in Pine involve declaring and evaluating *conditions* to dictate specific script actions, such as triggering different calculation patterns, visuals, signals, alerts, strategy orders, etc. As such, it's imperative to understand how to inspect the conditions a script uses to ensure proper execution.

As numbers

One possible way to debug a script's conditions is to define *numeric values* based on them, which allows programmers to inspect them using numeric approaches, such as those outlined in the previous section.

Let's look at a simple example. This script calculates the ratio between the ohlc4 price and the `lengthInput`-bar moving average. It assigns a condition to the `priceAbove` variable that returns `true` whenever the value of the ratio exceeds 1 (i.e., the price is above the average).

To inspect the occurrences of the condition, we created a `debugValue` variable assigned to the result of an expression that uses the ternary ?: operator to return 1 when `priceAbove` is `true` and 0 otherwise. The script plots the variable's value in all available locations:

```
//@version=6
indicator("Conditions as numbers demo", "MA signal")

//@variable The number of bars in the moving average calculation.
int lengthInput = input.int(20, "Length", 1)

//@variable The ratio of the `ohlc4` price to its `lengthInput`-bar moving average.
float ratio = ohlc4 / ta.sma(ohlc4, lengthInput)

//@variable The condition to inspect. Is `true` when `ohlc4` is above its moving average, `false` otherwise.
```

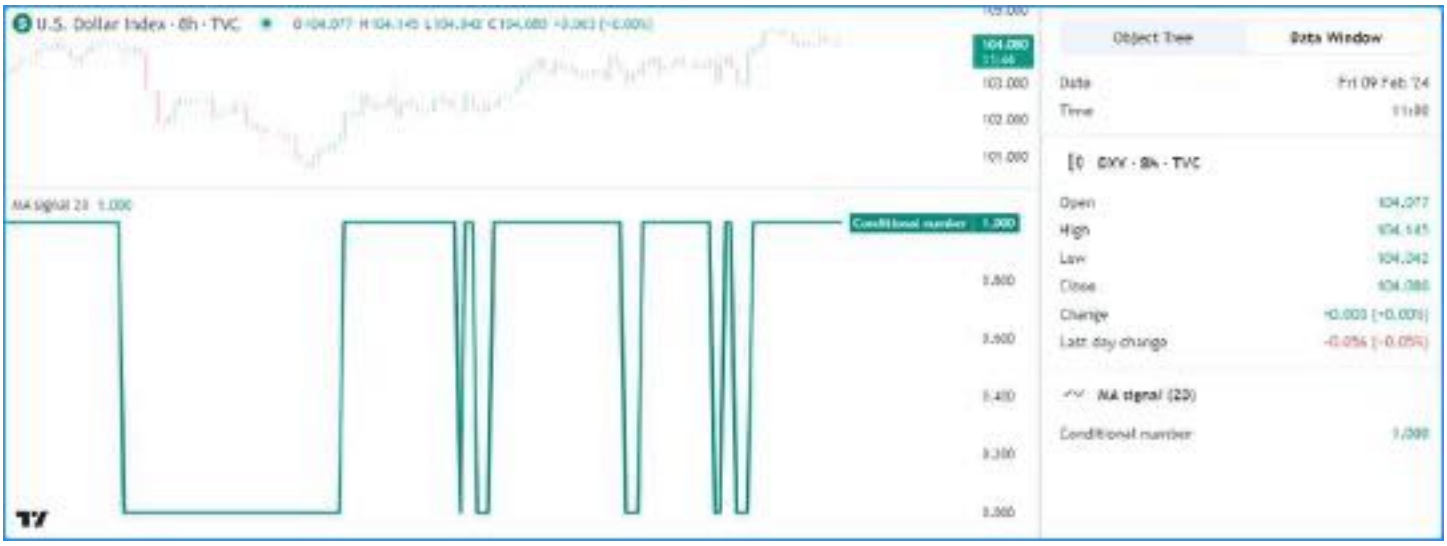


Figure 332: image

```
bool priceAbove = ratio > 1.0
//@variable Returns 1 when the `priceAbove` condition is `true`, 0 otherwise.
int debugValue = priceAbove ? 1 : 0

// Plot the `debugValue.
plot(debugValue, "Conditional number", color.teal, 3)
```

Note that:

- Representing "bool" values using numbers also allows scripts to display conditional shapes or characters at specific y-axis locations with plotshape() and plotchar(), and it facilitates conditional debugging with plotarrow(). See the next section to learn more.

Plotting conditional shapes

The plotshape() and plotchar() functions provide utility for debugging conditions, as they can plot shapes or characters at absolute or relative chart locations whenever they contain a `true` or non-na `series` argument.

These functions can also display *numeric* representations of the `series` in the script's status line and the Data Window, meaning they're also helpful for debugging numbers. We show a simple, practical way to debug numbers with these functions in the Tips section.

The chart locations of the plots depend on the `location` parameter, which is location.abovebar by default.

Let's inspect a condition using these functions. The following script calculates an RSI with a `lengthInput` length and a `crossBelow` variable whose value is the result of a condition that returns `true` when the RSI crosses below 30. It calls plotshape() to display a circle near the top of the pane each time the condition occurs:

```
//@version=6
indicator("Conditional shapes demo", "RSI cross under 30")

//@variable The length of the RSI.
int lengthInput = input.int(14, "Length", 1)

//@variable The calculated RSI value.
float rsi = ta.rsi(close, lengthInput)

//@variable Is `true` when the `rsi` crosses below 30, `false` otherwise.
bool crossBelow = ta.crossunder(rsi, 30.0)

// Plot the `rsi`.
plot(rsi, "RSI", color.rgb(136, 76, 146), linewidth = 3)
// Plot the `crossBelow` condition as circles near the top of the pane.
```

Figure 333: image

```
plotshape(crossBelow, "RSI crossed below 30", shape.circle, location.top, color.red, size = size.small)
```

Note that:

- The status line and Data Window show a value of 1 when `crossBelow` is `true` and 0 when it's `false`.

Suppose we'd like to display the shapes at *precise* locations rather than relative to the chart pane. We can achieve this by using conditional numbers and location.absolute in the plotshape() call.

In this example, we've modified the previous script by creating a `debugNumber` variable that returns the `rsi` value when `crossBelow` is `true` and na otherwise. The plotshape() function uses this new variable as its `series` argument and location.absolute as its `location` argument:

Figure 334: image

```
//@version=6
indicator("Conditional shapes demo", "RSI cross under 30")

//@variable The length of the RSI.
int lengthInput = input.int(14, "Length", 1)

//@variable The calculated RSI value.
float rsi = ta.rsi(close, lengthInput)
```

```
//@variable Is `true` when the `rsi` crosses below 30, `false` otherwise.
bool crossBelow = ta.crossunder(rsi, 30.0)
//@variable Returns the `rsi` when `crossBelow` is `true`, `na` otherwise.
float debugNumber = crossBelow ? rsi : na

// Plot the `rsi`.
plot(rsi, "RSI", color.rgb(136, 76, 146), linewidth = 3)
// Plot circles at the `debugNumber`.
plotshape(debugNumber, "RSI when it crossed below 30", shape.circle, location.absolute, color.red, size = size
```

Note that:

- Since we passed a *numeric* series to the function, our conditional plot now shows the values of the `debugNumber` in the status line and Data Window instead of 1 or 0.

Another handy way to debug conditions is to use plotarrow(). This function plots an arrow with a location relative to the *main chart prices* whenever the `series` argument is nonzero and not na. The length of each arrow varies with the `series` value supplied. As with plotshape() and plotchar(), plotarrow() can also display numeric results in the status line and the Data Window.

This example shows an alternative way to inspect our `crossBelow` condition using plotarrow(). In this version, we've set `overlay` to `true` in the indicator() function and added a plotarrow() call to visualize the conditional values. The `debugNumber` in this example measures how far the `rsi` dropped below 30 each time the condition occurs:

Figure 335: image

```
//@version=6
indicator("Conditional shapes demo", "RSI cross under 30", true)

//@variable The length of the RSI.
int lengthInput = input.int(14, "Length", 1)

//@variable The calculated RSI value.
float rsi = ta.rsi(close, lengthInput)

//@variable Is `true` when the `rsi` crosses below 30, `false` otherwise.
bool crossBelow = ta.crossunder(rsi, 30.0)
//@variable Returns `rsi - 30.0` when `crossBelow` is `true`, `na` otherwise.
float debugNumber = crossBelow ? rsi - 30.0 : na

// Plot the `rsi`.
plot(rsi, "RSI", color.rgb(136, 76, 146), display = display.data_window)
// Plot circles at the `debugNumber`.
plotarrow(debugNumber, "RSI cross below 30 distnce")
```

Note that:

- We set the `display` value in the plot() of the `rsi` to display.data_window to preserve the chart's scale.

To learn more about plotshape(), plotchar(), and plotarrow(), see this manual's Text and shapes page.

Conditional colors

An elegant way to visually represent conditions in Pine is to create expressions that return color values based on `true` or `false` states, as scripts can use them to control the appearance of drawing objects or the results of `plot*()`, fill(), bgcolor(), or barcolor() calls.

For example, this script calculates the change in close prices over `lengthInput` bars and declares two "bool" variables to identify when the price change is positive or negative.

The script uses these "bool" values as conditions in ternary expressions to assign the values of three "color" variables, then uses those variables as the `color` arguments in plot(), bgcolor(), and barcolor() to debug the results:

Figure 336: image

```
//@version=6
indicator("Conditional colors demo", "Price change colors")

//@variable The number of bars in the price change calculation.
int lengthInput = input.int(10, "Length", 1)

//@variable The change in `close` prices over `lengthInput` bars.
float priceChange = ta.change(close, lengthInput)

//@variable Is `true` when the `priceChange` is a positive value, `false` otherwise.
bool isPositive = priceChange > 0
//@variable Is `true` when the `priceChange` is a negative value, `false` otherwise.
bool isNegative = priceChange < 0

//@variable Returns a color for the `priceChange` plot to show when `isPositive`, `isNegative`, or neither occ
color plotColor = isPositive ? color.teal : isNegative ? color.maroon : chart.fg_color
//@variable Returns an 80% transparent color for the background when `isPositive` or `isNegative`, `na` otherw
color bgColor = isPositive ? color.new(color.aqua, 80) : isNegative ? color.new(color.fuchsia, 80) : na
//@variable Returns a color to emphasize chart bars when `isPositive` occurs. Otherwise, returns the `chart.bg
color barColor = isPositive ? color.orange : chart.bg_color

// Plot the `priceChange` and color it with the `plotColor`.
plot(priceChange, "Price change", plotColor, style = plot.style_area)
// Highlight the pane's background with the `bgColor`.
```

```
bgcolor(bgColor, title = "Background highlight")
// Emphasize the chart bars with positive price change using the `barColor`.
barcolor(barColor, title = "Positive change bars")
```

Note that:

- The barcolor() function always colors the main chart's bars, regardless of whether the script occupies another chart pane, and the chart will only display the results if the bars are visible.

See the Colors, Fills, Backgrounds, and Bar coloring pages for more information about working with colors, filling plots, highlighting backgrounds, and coloring bars.

Using drawings

Pine Script™'s drawing types provide flexible ways to visualize conditions on the chart, especially when the conditions are within local scopes.

Consider the following script, which calculates a custom `filter` with a smoothing parameter (`alpha`) that changes its value within an if structure based on recent volume conditions:

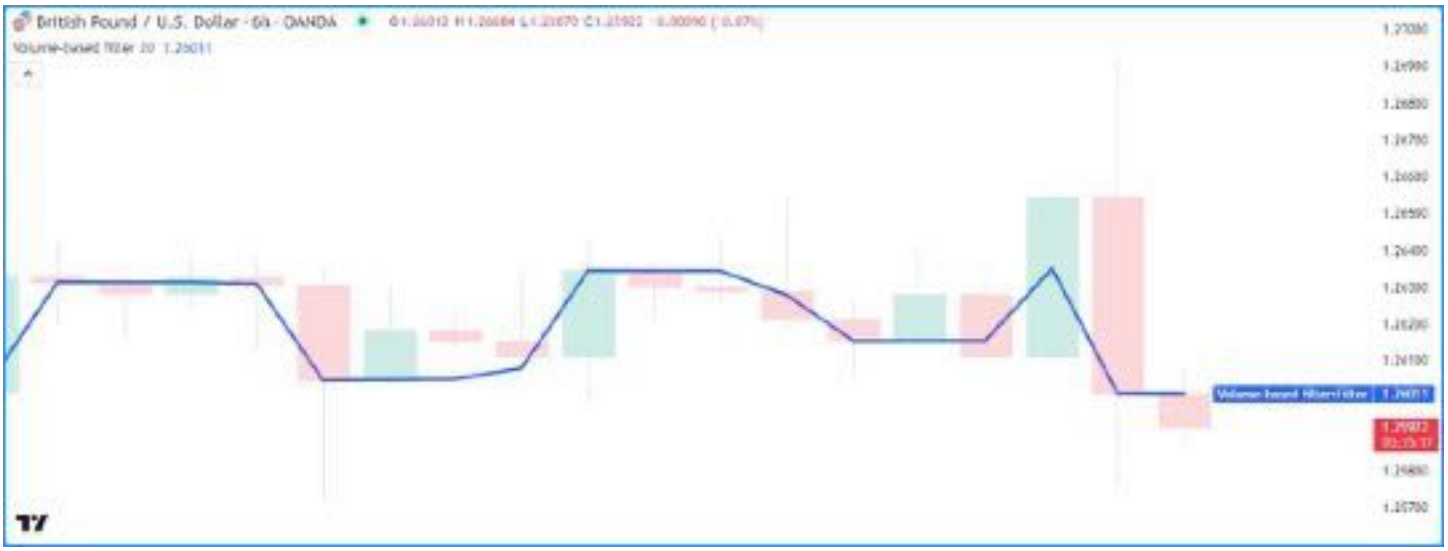


Figure 337: image

```
//@version=6
indicator("Conditional drawings demo", "Volume-based filter", true)

//@variable The number of bars in the volume average.
int lengthInput = input.int(20, "Volume average length", 1)

//@variable The average `volume` over `lengthInput` bars.
float avgVolume = ta.sma(volume, lengthInput)

//@variable A custom price filter based on volume activity.
float filter = close
//@variable The smoothing parameter of the filter calculation. Its value depends on multiple volume conditions
float alpha = na

// Set the `alpha` to 1 if `volume` exceeds its `lengthInput`-bar moving average.
if volume > avgVolume
    alpha := 1.0
// Set the `alpha` to 0.5 if `volume` exceeds its previous value.
else if volume > volume[1]
    alpha := 0.5
// Set the `alpha` to 0.01 otherwise.
else
```

```
    alpha := 0.01

// Calculate the new `filter` value.
filter := (1.0 - alpha) * nz(filter[1], filter) + alpha * close

// Plot the `filter`.
plot(filter, "Filter", linewidth = 3)
```

Suppose we'd like to inspect the conditions that control the `alpha` value. There are several ways we could approach the task with chart visuals. However, some approaches will involve more code and careful handling.

For example, to visualize the if structure's conditions using plotted shapes or background colors, we'd have to create additional variables or expressions in the global scope for the `plot*()` or bgcolor() functions to access.

Alternatively, we can use drawing types to visualize the conditions concisely without those extra steps.

The following is a modification of the previous script that calls label.new() within specific branches of the conditional structure to draw labels on the chart whenever those branches execute. These simple changes allow us to identify those conditions on the chart without much extra code:

Figure 338: image

```
//@version=6
indicator("Conditional drawings demo", "Volume-based filter", true, max_labels_count = 500)

//@variable The number of bars in the volume average.
int lengthInput = input.int(20, "Volume average length", 1)

//@variable The average `volume` over `lengthInput` bars.
float avgVolume = ta.sma(volume, lengthInput)

//@variable A custom price filter based on volume activity.
float filter = close
//@variable The smoothing parameter of the filter calculation. Its value depends on multiple volume conditions
float alpha = na

// Set the `alpha` to 1 if `volume` exceeds its `lengthInput`-bar moving average.
if volume > avgVolume
    // Add debug label.
    label.new(chart.point.now(high), "alpha = 1", color = color.teal, textcolor = color.white)
    alpha := 1.0
// Set the `alpha` to 0.5 if `volume` exceeds its previous value.
else if volume > volume[1]
    // Add debug label.
```